

Kế hoạch xây dựng chức năng chuyển tiền sát production

Stack: Java 21 · Spring Boot 3 · PostgreSQL · Kafka · Resilience4j · Docker Compose

Phạm vi: tiền giả, nhưng ngữ nghĩa nghiệp vụ và độ tin cậy giống hệ thống thật.

Kiến trúc: ứng dụng chuyển tiền standalone (DB riêng) + Fake Bank (app nhỏ, hệ thống ngoài giả lập) + dự án thương mại có sẵn đóng vai client.

Mục tiêu: học nghiệp vụ chuyển tiền, không cần hoàn thành một app hoàn chỉnh.

Quan sát (observability): tạm để mức tối thiểu (xem Giai đoạn 6), làm sâu sau.

Tra cứu: mục lục có số trang ở ngay bên dưới; bảng tra cứu nhanh theo chủ đề nằm sau mục lục. Số trang khớp với số in ở chân mỗi trang.

Mục lục

Tra cứu nhanh theo chủ đề	4
Đọc trước: cách dùng tài liệu này	5
Đừng đọc hết một lượt	5
Phạm vi tối thiểu có giá trị	5
Ước lượng thời gian (thô)	5
Năm việc đầu tiên	5
Thuật ngữ	6
Trạng thái kiểm chứng của tài liệu	6
0. Đổi tư duy: từ CRUD sang sổ cái	8
Bốn bất biến (invariants) phải luôn đúng	8
1. Kiến trúc tổng thể	9
1.1 Ba hệ thống, ba vai trò	9
1.2 Nguyên tắc ranh giới (vi phạm là mất giá trị học tập)	9
1.3 Ba loại chuyển tiền và bút toán tương ứng	9
1.4 Lộ trình xây dựng	10
1.5 Cấu trúc dự án gợi ý	10
1.6 Thư viện Spring Boot dự kiến	10
2. Các giai đoạn thực hiện (đi từ CRUD, nâng cấp từng bậc, mỗi bậc kèm tài liệu schema)	12
2.0 Cách đọc phần này	12
2.0.1 Bản đồ tổng quan	13
2.0.2 Nguyên tắc thiết kế schema chung	13
2.0.3 Quy ước migration và bảng migration theo bậc	13
2.0.4 Sơ đồ quan hệ đích của wallet-service	14
Giai đoạn 0: Chốt định nghĩa "đúng" và dựng môi trường (1-2 ngày)	14
Tổng kết giai đoạn 0	16
Giai đoạn 1: Từ CRUD đến chuyển tiền nội bộ đúng đắn	16
Bậc 1.1: CRUD ngây thơ	18
Bậc 1.2: Bọc trong transaction	19
Bậc 1.3: Khóa dòng khi đọc số dư	20
Bậc 1.4: Khóa có thứ tự	20
Bậc 1.5: Sổ cái ghi từng thay đổi (ledger)	20
Bậc 1.6: Bút toán kép và điểm ghi sổ duy nhất	21
Bậc 1.7: Thứ tự bút toán theo từng tài khoản	23
Bậc 1.8: Đưa ràng buộc xuống database	24
Bậc 1.9: Transfer có trạng thái và lỗi nghiệp vụ có lưu vết	27
Bậc 1.10: Kiểm chứng	31
Tổng kết giai đoạn 1	33
Giai đoạn 2: Idempotency (gửi trùng không tạo hiệu ứng trùng)	33

Bậc 2.1: Chứng minh lỗi	34
Bậc 2.2: Unique key trên transfers (lớp bảo vệ thô)	34
Bậc 2.3: Lưu response để trả lại kết quả cũ	35
Bậc 2.4: Xử lý đồng thời cùng key	37
Bậc 2.5: Idempotency theo nghiệp vụ	37
Bậc 2.6: Lưu cả lỗi nghiệp vụ và dọn dẹp	38
Tổng kết giai đoạn 2	38
Giai đoạn 3: Xử lý nền, outbox và Kafka	39
Bậc 3.1: State machine có kiểm soát	39
Bậc 3.2: Tách xử lý ra nền bằng worker quét DB (chưa cần Kafka)	40
Bậc 3.3: Gửi event lên Kafka một cách ngẫu nhiên	40
Bậc 3.4: Transactional outbox	40
Bậc 3.5: Consumer idempotent	42
Bậc 3.6: Thử tự, cấu hình Kafka và xử lý lỗi	43
Bậc 3.7: Ma trận "chết ở đâu thì chuyện gì xảy ra"	44
Tổng kết giai đoạn 3	44
Giai đoạn 4: Fake Bank và chuyển tiền liên ngân hàng (tùy chọn theo tiến độ)	44
Bậc 4.1: Fake Bank đơn giản và gọi bank đồng bộ trong transaction (bản ngẫu nhiên)	45
Bậc 4.2: Tách transaction, dùng tài khoản clearing (saga)	48
Bậc 4.3: Timeout, retry và idempotency key gửi bank	48
Bậc 4.4: Trạng thái UNKNOWN và inquiry	50
Bậc 4.5: Callback và inbox chống trùng	51
Bậc 4.6: Circuit breaker và bulkhead	52
Bậc 4.7: Nạp tiền (INBOUND) và ma trận lỗi tổng	53
Tổng kết giai đoạn 4	53
Giai đoạn 5: Bảo vệ hệ thống	54
Bậc 5.1: Rate limit	55
Bậc 5.2: Hạn mức nghiệp vụ (bản sai rồi bản đúng)	55
Bậc 5.3: Phân quyền	57
Bậc 5.4: Đóng băng và cảnh báo bất thường	57
Tổng kết giai đoạn 5	57
Giai đoạn 6: Observability tối thiểu (tạm không đi sâu Prometheus)	58
Bậc 6.1: Log có ID xuyên suốt	58
Bậc 6.2: Lịch sử trạng thái làm "observability nghiệp vụ"	59
Bậc 6.3: Bộ SQL sức khỏe hệ thống	59
Bậc 6.4 (tùy chọn khi sẵn sàng): 3 chỉ số Prometheus	59
Tổng kết giai đoạn 6	59
Giai đoạn 7: Recovery và Reconciliation	60
Bậc 7.1: Sửa tay và viết runbook	61
Bậc 7.2: Recovery worker theo bảng luật	61
Bậc 7.3: Reconciliation nội bộ	61
Bậc 7.4: Reconciliation với ngân hàng và bút toán điều chỉnh	63
Tổng kết giai đoạn 7	64
Giai đoạn 8: Kiểm thử độ tin cậy (phá hệ thống có chủ đích)	65
Bậc 8.1: Biến ma trận thành test	65
Bậc 8.2: Property-based test	66
Bậc 8.3: Fault injection hạ tầng và load test	66
Bậc 8.4: Postmortem	66
Tổng kết giai đoạn 8	66
Giai đoạn 9: Tích hợp với dự án thương mại (client của wallet-service)	66
Bậc 9.1: Gọi wallet đồng bộ, không idempotency (bản ngẫu nhiên)	67
Bậc 9.2: Key ổn định và state machine thanh toán	68
Bậc 9.3: Nhận kết quả bằng event	69
Bậc 9.4: Xử lý UNKNOWN và job quét	69
Bậc 9.5: Hoàn tiền khi hủy đơn	69
Tổng kết giai đoạn 9	70
3. Danh sách vấn đề khó cần suy nghĩ	71
Về đúng đắn	71
Về lỗi mạng và tính không chắc chắn	71
Về dữ liệu và vận hành	71
4. Kịch bản kiểm thử bắt buộc	72

5. Thứ tự làm và mốc hoàn thành	73
6. Checklist tổng	74
7. Logic kiểm thử: mô tả từng kịch bản để chạy được mà không phải tự viết test	75
7.1 Vì sao test khó, và cách tách nhỏ	75
7.2 Mẫu một thẻ kịch bản	75
7.3 Bốn viên gạch dùng chung	76
7.4 Quy trình chạy một kịch bản	77
7.5 Các thẻ kịch bản	78
7.6 Chạy ngay không cần ứng dụng: bốn kịch bản pgbench	81
7.7 Khi một kịch bản đồ: nhìn vào đâu	82
7.8 Chạy thẻ nào ở giai đoạn nào	83
8. Giao diện quan sát nghiệp vụ (Ops Console): xem kết quả thay vì mò vào database	84
8.1 Nguyên tắc thiết kế	84
8.2 Các màn hình	84
8.3 Các view và quyền đọc	86
8.4 Câu SQL cho từng màn hình	88
8.5 Scenario Lab: chạy kịch bản bằng nút bấm	89
8.6 Dựng bằng cách nào	90
8.7 Lộ trình dựng theo từng giai đoạn	90

Tra cứu nhanh theo chủ đề

Tôi muốn...	Xem
Hiểu ledger / bút toán kép là gì	Đối tư duy (tr. 8) · Bút toán từng loại chuyển tiền (tr. 9)
Tra một thuật ngữ (idempotency, saga, outbox...)	Thuật ngữ (tr. 6)
Biết nên làm gì trước, mất bao lâu	Năm việc đầu tiên (tr. 5) · Phạm vi tối thiểu (tr. 5) · Ước lượng thời gian (tr. 5) · Thử tự làm (tr. 73)
Hiểu kiến trúc ba hệ thống	Ba hệ thống (tr. 9) · Nguyên tắc ranh giới (tr. 9)
Xem đáp án schema (migration theo bậc)	Bảng migration theo bậc (tr. 13) · Sổ cái (tr. 21) · Transfer có trạng thái (tr. 27) · Fake Bank (tr. 45) · Phía thương mại (tr. 68)
Biết một bảng/cột là gì, vì sao có	Tài khoản, transfer (tr. 27) · Sổ cái (tr. 21) · Idempotency (tr. 35) · Outbox (tr. 40) · Gọi bank (tr. 48) · Đối soát (tr. 61)
Tự chấm schema của mình	Rubric đủ 15 tiêu chí (tr. 31) · Lỗi thường gặp (tr. 24) · Đánh đổi thiết kế (tr. 27)
Câu SQL kiểm tra bất biến / sức khỏe	Bất biến (bản đầy đủ) (tr. 14) · Từng câu theo bậc (tr. 31) · SQL sức khỏe (tr. 59)
Race condition, double-spend, deadlock	Khóa dòng (tr. 20) · Khóa có thứ tự (tr. 20)
Idempotency (chống gửi trùng)	Cà giai đoạn (tr. 33) · Trả lại kết quả cũ (tr. 35) · Cùng key đồng thời (tr. 37) · Theo nghiệp vụ (tr. 37)
State machine / trạng thái giao dịch	Nội bộ (tr. 39) · Chuyển ra ngân hàng (tr. 48)
Outbox, dual write, consumer idempotent	Dual write (tr. 40) · Outbox (tr. 40) · Consumer (tr. 42)
Cấu hình Kafka, DLT, thứ tự message	Kafka (tr. 43)
Chết giữa chừng thì sao (crash matrix)	Ma trận crash (tr. 44)
Gọi ngân hàng: timeout, retry, UNKNOWN	Retry/Resilience4j (tr. 48) · UNKNOWN + inquiry (tr. 50) · Circuit breaker (tr. 52)
Callback từ ngân hàng, ma trận lỗi bank	Callback/inbox (tr. 51) · Ma trận lỗi (tr. 53)
Rate limit, hạn mức, phân quyền	Rate limit (tr. 55) · Hạn mức nguyên tử (tr. 55) · Phân quyền (tr. 57)
Recovery và đối soát	Recovery worker (tr. 61) · Đối soát nội bộ (tr. 61) · Đối soát với bank (tr. 63)
Kiểm thử chaos, load, postmortem	Giai đoạn 8 (tr. 65) · Kịch bản test (tr. 72)
Nổi vào dự án thương mại	Giai đoạn 9 (tr. 66)
Các vấn đề khó cần suy nghĩ	Mục 3 (tr. 71)
Viết logic kiểm thử (mất mạng, đồng thời, sập process) mà không tự viết test	Chương 7 (tr. 75) · Thẻ kịch bản (tr. 78) · pgbench chạy ngay (tr. 81)
Xem kết quả trên giao diện thay vì mò DB	Chương 8 (tr. 84) · Các màn hình (tr. 84) · View và quyền đọc (tr. 86) · Scenario Lab (tr. 89)
Checklist hoàn thành	Mục 6 (tr. 74)

Đọc trước: cách dùng tài liệu này

Đừng đọc hết một lượt

Tài liệu dài vì nó vừa là bản đồ vừa là đáp án để **tra cứu**, không phải bài đọc liền. Cách dùng:

Bạn đang...	Đọc
Lần đầu mở tài liệu (30-45 phút)	Mục 0 (tư duy ledger), 1.1 đến 1.3 (ba hệ thống, bút toán) và bảng Thuật ngữ bên dưới
Sắp bắt đầu một giai đoạn	Chỉ giai đoạn đó trong mục 2 (mỗi bậc đã kèm đủ tài liệu schema)
Vừa tự thiết kế xong schema của một bậc, muốn đối chiếu	Khối Tài liệu schema ở cuối chính bậc đó (migration đáp án, từ điển cột, rubric)
Thấy một bảng hoặc cột khó hiểu	Bảng migration ở mục 2.0.3 cho biết bảng đó sinh ra ở bậc nào; từ điển cột nằm trong khối Tài liệu schema của bậc đó
Test đo hoặc gặp lỗi lạ	Mục 3 (vấn đề khó) và 4 (kịch bản test)

Phạm vi tối thiểu có giá trị

- **Lỗi:** Giai đoạn 0 đến 3 (ledger, idempotency, xử lý bất đồng bộ với outbox và Kafka). Chỉ làm đến đây vẫn học được phần lớn khái niệm nền.
- **Mở rộng theo hứng thú:** Giai đoạn 9 (nổi dự án thương mại, học phía client) **hoặc** Giai đoạn 4 (Fake Bank, phần khó nhất về xử lý lỗi). Sau đó 5, 6, 7, 8.
- Không cần làm hết để có giá trị. Dừng ở giai đoạn nào cũng nên có test xanh và bất biến đúng.

Ước lượng thời gian (thô)

Giả định làm bán thời gian và đã biết Spring Boot cơ bản. Đây là ước lượng thô để lên lịch, thực tế phụ thuộc kinh nghiệm của bạn với Kafka và concurrency.

Giai đoạn	Ước lượng
0: Chốt định nghĩa "đúng", dựng môi trường	1-2 ngày
1: Từ CRUD đến ledger	1-2 tuần
2: Idempotency	3-5 ngày
3: Bất đồng bộ, outbox, Kafka	1-2 tuần
4: Fake Bank và liên ngân hàng	2 tuần
5: Bảo vệ hệ thống	3-5 ngày
6: Quan sát tối thiểu	2-3 ngày
7: Recovery và đối soát	1-2 tuần
8: Kiểm thử độ tin cậy	1 tuần
9: Nổi dự án thương mại	1 tuần

Năm việc đầu tiên

1. Đọc mục 0 và 1.3.
2. Dựng `docker-compose.yml` với PostgreSQL (Kafka để sau, đến Giai đoạn 3).
3. Tạo project Spring Boot + Flyway + Testcontainers, chạy một test rỗng với Postgres thật.
4. Viết `docs/invariants.md` với các câu SQL bất biến (chép từ bước 0.2 ở Giai đoạn 0, đã có đủ câu SQL).

5. Bắt đầu Giai đoạn 1, bậc 1.1: viết CRUD ngây thơ rồi viết 3 test làm nó hỏng.

Thuật ngữ

Thuật ngữ	Nghĩa ngắn gọn
Ledger / sổ cái	Sổ ghi mọi dòng tiền, chỉ thêm không sửa. Số dư suy ra từ đây
Double-entry / bút toán kép	Mỗi sự kiện tiền ghi ít nhất 2 dòng (một bên ra, một bên vào) và tổng luôn bằng 0
DEBIT / CREDIT	Trong tài liệu này: DEBIT = tiền ra khỏi tài khoản, CREDIT = tiền vào
Invariant / bất biến	Điều luôn phải đúng bất kể chuyện gì xảy ra (ví dụ tổng debit = tổng credit)
Transaction (DB)	Nhóm lệnh DB hoặc thành công cả nhóm, hoặc không lệnh nào có hiệu lực
Race condition	Hai luồng chạy xen kẽ cho kết quả sai (ví dụ cùng đọc số dư cũ rồi cùng trừ)
Khóa bi quan / lạc quan	Bi quan: khóa dòng trước khi sửa (FOR UPDATE). Lạc quan: không khóa, phát hiện xung đột lúc ghi (cột version)
Deadlock	Hai luồng giữ khóa của nhau và chờ nhau mãi; DB phải hủy một bên
Idempotency	Làm cùng một thao tác nhiều lần cho kết quả như làm một lần
Idempotency key	Mã client gửi kèm để server nhận ra "yêu cầu này đã xử lý rồi"
State machine / máy trạng thái	Quy định rõ trạng thái nào được chuyển sang trạng thái nào
Dual write	Ghi vào hai nơi (DB và Kafka) mà không có transaction chung, nên có thể ghi được một nơi
Outbox	Ghi sự kiện vào bảng trong cùng transaction với dữ liệu, một tiến trình khác gửi đi sau
Inbox / processed_messages	Bảng nhớ message đã xử lý để xử lý trùng không gây hiệu ứng trùng
At-least-once	Message chắc chắn đến, nhưng có thể đến nhiều lần (nên consumer phải idempotent)
DLT (dead letter topic)	Nơi chứa message xử lý thất bại nhiều lần để người xem sau
Saga	Quy trình nhiều bước, mỗi bước một transaction ngắn; lỗi giữa chừng thì chạy bước bù
Bút toán bù / reversal	Bút toán mới đảo ngược bút toán cũ, thay vì sửa hoặc xóa cái cũ
Clearing account	Tài khoản trung gian giữ tiền "đang bay" tới hoặc từ ngân hàng ngoài
UNKNOWN	Trạng thái "không biết kết quả" (ví dụ gọi bank bị timeout). Khác hẳn "thất bại"
Inquiry	API hỏi lại bank "giao dịch này kết quả ra sao"
Reconciliation / đối soát	So số của mình với số liệu của bên khác (hoặc với chính mình) để tìm chỗ lệch
Circuit breaker	Ngắt tạm thời các lời gọi tới dịch vụ đang lỗi để khỏi dồn thêm tải
Hot account	Tài khoản bị rất nhiều giao dịch cùng lúc, nghẽn vì khóa

Trạng thái kiểm chứng của tài liệu

Đã chạy thử:

- Toàn bộ migration của từng bậc (V0 đến V7_4, DB Fake Bank, phía thương mại) chạy sạch trên PostgreSQL 16, theo đúng thứ tự xuất hiện trong tài liệu.
- 11 thử nghiệm hành vi: bút toán cân được chấp nhận; bút toán không cân bị từ chối lúc COMMIT; UPDATE/DELETE ledger bị chặn; số dư âm bị chặn; ghi sổ trùng bước, trùng account_seq, trùng idempotency key, trùng nguồn đơn hàng đều bị chặn; câu hạn mức ON CONFLICT trả 0 dòng khi vượt hạn mức.
- Các câu SQL bất biến và SQL sức khỏe chạy được và bắt đúng dữ liệu cố tình làm sai.
- Request thứ hai cùng idempotency key tự chờ request đầu commit (đo được).

- Khóa không thứ tự gây deadlock (65 lần ghi nhận trong 6 giây, 96% giao dịch lỗi); khóa có thứ tự: 0 lỗi trong 2039 giao dịch.
- CRUD kiểu đọc-rồi-ghi mất cập nhật: chạy 200 lần rút 10 từ số dư 1000, số dư chỉ giảm 250 thay vì về 0.

Chưa kiểm chứng (hãy tự kiểm khi làm):

- Code Java mẫu chưa được biên dịch.
- Cấu hình Resilience4j: tên thuộc tính và thứ tự decorator khớp tài liệu chính thức, nhưng việc bật đồng thời `enable-exponential-backoff` và `enable-randomized-wait` trong một instance chưa xác minh được. Xem ghi chú ở Giai đoạn 4, bậc 4.3.
- Cấu hình Kafka chưa chạy thật. Ước lượng thời gian là thô.

0. Đổi tư duy: từ CRUD sang sổ cái

CRUD: `UPDATE accounts SET balance = balance - 100 ...` rồi `... + 100 ...`. Sai vì không có lịch sử, không audit được, lỗi giữa chừng là mất tiền, không giải thích được "tiền đi đâu".

Chuyển tiền thật dựa trên **ledger**:

- **Double-entry**: mỗi giao dịch tạo ít nhất 2 bút toán (DEBIT một bên, CREDIT bên kia), tổng luôn bằng 0.
- **Append-only**: không UPDATE/DELETE bút toán. Sai thì tạo bút toán đảo (reversal).
- **Số dư là kết quả của ledger**; cột `balance` trong `accounts` chỉ là cache để đọc nhanh.
- **Tiền là số nguyên** ở đơn vị nhỏ nhất (VND: đồng), kiểu `BIGINT` + `currency`. Không dùng `double` / `float`; nếu cần tính toán dùng `long` hoặc `BigDecimal` rồi quy đổi.

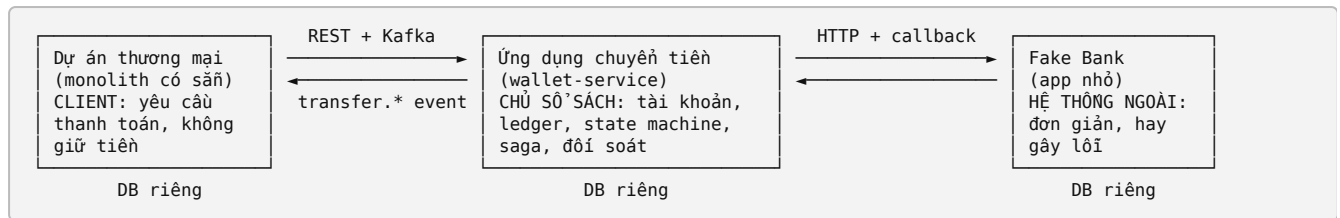
Bốn bất biến (invariants) phải luôn đúng

1. Tổng DEBIT = tổng CREDIT trên toàn hệ thống.
2. Số dư tài khoản người dùng không bao giờ âm.
3. Một yêu cầu chuyển tiền chỉ tạo hiệu ứng tiền tệ đúng một lần dù được gửi nhiều lần.
4. `accounts.balance` = tổng ledger của tài khoản đó.

Mọi test và mọi quyết định thiết kế nên quy về việc bảo vệ 4 điều này. Hãy viết chúng thành **câu SQL kiểm tra** ngay từ đầu.

1. Kiến trúc tổng thể

1.1 Ba hệ thống, ba vai trò



Hệ thống	Vai trò	Giữ tiền?	Độ phức tạp nghiệp vụ
Dự án thương mại (có sẵn)	Người yêu cầu: "trừ ví người mua, cộng ví người bán"	Không	Đã có, chỉ thêm phần gọi ví
Ứng dụng chuyển tiền (bạn xây)	Ví/ngân hàng thu nhỏ: tài khoản, ledger, chuyển nội bộ, điều phối chuyển ra/vào ngân hàng ngoài	Có	Cao, đây là nơi học nghiệp vụ
Fake Bank	Đóng vai ngân hàng khác, cố tình gây lỗi để bạn luyện xử lý lỗi và đối soát	Ít (số dư đơn giản)	Thấp

1.2 Nguyên tắc ranh giới (vi phạm là mất giá trị học tập)

- **Mỗi hệ thống một database riêng.** Không truy cập DB của nhau.
- Chỉ giao tiếp qua **API (REST) và event (Kafka)**. Không import code của nhau.
- Phía gọi luôn coi lời gọi là **có thể chậm, lỗi, hoặc không rõ kết quả**: timeout, retry có backoff + idempotency key, circuit breaker, hỏi lại trạng thái thay vì đoán.
- Hợp đồng API/event được ghi rõ và ổn định (có version).

1.3 Ba loại chuyển tiền và bút toán tương ứng

Quy ước trong tài liệu này: DEBIT = tiền ra khỏi tài khoản (balance giảm), CREDIT = tiền vào tài khoản (balance tăng). Đây là quy ước "kiểu ví" cho dễ hiểu; kế toán thật dùng chart of accounts và ý nghĩa debit/credit phụ thuộc loại tài khoản. Đủ dùng cho mục tiêu học.

Tài khoản hệ thống cần có:

Loại	Vai trò
SYSTEM_FEE	Nhận phí
SYSTEM_CLEARING	Giữ tiền "đang bay" tới/từ ngân hàng ngoài
SYSTEM_BANK	Một tài khoản cho mỗi ngân hàng đối tác, đại diện phần tiền đã đi ra hoặc đã đi vào hệ thống qua ngân hàng đó. Được phép có số dư bất kỳ

Loại 1: chuyển nội bộ (INTERNAL). Ví A → ví B, một DB, một transaction.

Ledger transaction (type)	DEBIT	CREDIT	Số tiền
PRINCIPAL	A	B	amount
FEE (nếu có phí)	A	SYSTEM_FEE	fee

Loại 2: chuyển ra ngân hàng ngoài (OUTBOUND). Saga nhiều bước.

Bước	Ledger transaction (type)	DEBIT	CREDIT	Số tiền
Giữ tiền	TO_CLEARING	A	SYSTEM_CLEARING	amount + fee
Bank báo thành công	SETTLE	SYSTEM_CLEARING	SYSTEM_BANK	amount
Bank báo thành công	FEE	SYSTEM_CLEARING	SYSTEM_FEE	fee
Bank báo thất bại	REFUND	SYSTEM_CLEARING	A	amount + fee

Loại 3: nạp tiền từ ngân hàng ngoài (INBOUND). Bank báo có tiền vào (callback).

Ledger transaction (type)	DEBIT	CREDIT	Số tiền
TOPUP	SYSTEM_BANK	Ví của người dùng	amount

Nạp tiền ban đầu cho môi trường dev: không set balance trực tiếp. Tạo một bút toán từ SYSTEM_BANK sang ví người dùng (giống loại 3) để các bất biến vẫn đúng ngay từ đầu.

1.4 Lộ trình xây dựng

- Ứng dụng chuyển tiền standalone:** một Spring Boot app, DB riêng, chia package `account`, `ledger`, `transfer`, `idempotency`, `outbox`. Làm Giai đoạn 0-3 ở đây (có Kafka, outbox). Test bằng test tự động hoặc Postman/curl.
- Fake Bank** (tùy chọn theo tiến độ): app nhỏ riêng, thêm chuyển ra/nạp vào (Giai đoạn 4).
- Nối dự án thương mại** làm client (Giai đoạn 9). Bước này **không phụ thuộc** bước 2, có thể làm trước nếu muốn.

Không cần tách nhiều microservice. Tách thêm chỉ khi thật sự có lý do học.

1.5 Cấu trúc dự án gợi ý

```

money-transfer/
├── docker-compose.yml      # postgres (nhiều DB), kafka (KRaft), toxiproxy (sau)
├── wallet-service/         # ứng dụng chuyển tiền (Spring Boot)
├── fake-bank/              # ngân hàng giả (Spring Boot, nhỏ)
├── common/                 # (tùy chọn) DTO/event schema dùng chung
└── docs/                  # sơ đồ, ADR (quyết định thiết kế), runbook

```

Dự án thương mại nằm ở repo hiện có của bạn, chỉ thêm module/client gọi sang `wallet-service`.

1.6 Thư viện Spring Boot dự kiến

Nhu cầu	Thư viện
Web/API	spring-boot-starter-web, validation
DB	spring-boot-starter-data-jpa (hoặc JdbcTemplate/jOOQ cho phần ledger), PostgreSQL driver
Migration	Flyway
Kafka	spring-kafka
Timeout/Retry/Circuit breaker/Rate limiter/Bulkhead	Resilience4j (<code>resilience4j-spring-boot3</code> , dùng bản từ 2.3.0, xem Giai đoạn 4 bậc 4.3)
Rate limit phân tán (tùy chọn)	Bucket4j + Redis
Test	JUnit 5, Testcontainers (Postgres + Kafka), Awaitility, jqwik
Security	spring-boot-starter-security (API key hoặc JWT đơn giản)
Health	spring-boot-starter-actuator

Dùng **Testcontainers** cho mọi test chạm DB/Kafka. Test concurrency trên H2 hoặc mock cho kết quả sai.

2. Các giai đoạn thực hiện (đi từ CRUD, nâng cấp từng bậc, mỗi bậc kèm tài liệu schema)

2.0 Cách đọc phần này

Bạn sẽ **không** xây hệ thống cuối cùng ngay. Bạn bắt đầu bằng một bản CRUD ngây thơ, **tự làm nó hỏng bằng test**, rồi nâng cấp từng bậc. Mỗi bậc giải quyết đúng **một** vấn đề, và để lại một chỗ hờ dẫn sang bậc kế tiếp. Cách này giúp bạn hiểu *vi sao* mỗi thành phần tồn tại thay vì chỉ chép đáp án.

Mỗi **bậc** có 5 dòng cố định:

- **Vấn đề:** cái gì đang sai, và cách chứng minh nó sai (một test hoặc thí nghiệm làm nó hỏng).
- **Nâng cấp:** thay đổi cụ thể (code, schema, thiết kế).
- **Giải quyết được:** vấn đề nào đã hết.
- **Còn hờ:** vấn đề nào vẫn còn, đây chính là lý do có bậc tiếp theo.
- **✓ Bàn giao:** sản phẩm để biết bạn đã xong bậc đó.

Mỗi **giai đoạn** mở đầu bằng khối **Trạng thái database trước khi bắt đầu** (sơ đồ ERD, danh sách bảng phải có, và một câu SQL để bạn tự kiểm tra), và kết thúc bằng bảng **Tổng kết** (bạn vừa làm gì, giải bài toán gì, còn hờ gì, đi tiếp ra sao) và phần **Tự kiểm tra**.

Nguyên tắc: **làm hỏng trước, sửa sau**. Ở mỗi bậc, viết test chứng minh lỗi tồn tại (test đỏ) rồi mới nâng cấp để test xanh. Nếu bạn không tái hiện được lỗi thì chưa hiểu *vi sao* cần bản sửa.

Tài liệu schema nằm ngay trong từng bậc. Bậc nào đụng tới database thì cuối bậc có khối **Tài liệu schema của bậc N.M**, để bạn không phải lật sang chỗ khác. Khối này gồm (không bậc nào có đủ mọi mục, chỉ những mục liên quan):

- **Migration:** đáp án, chạy được nguyên văn trên PostgreSQL 16, đặt tên theo Flyway.
- **Từ điển cột:** mỗi cột mới hoặc đổi là gì, vì sao tồn tại.
- **Index và ràng buộc:** chặn lỗi nào, phục vụ truy vấn nào.
- **Đánh đổi thiết kế:** vì sao chọn vậy, phương án khác là gì.
- **Lỗi hay gặp khi tự thiết kế.**
- **Tự chấm (rubric):** các tiêu chí của phần schema thuộc bậc này (bậc 1.10 gom đủ 15 tiêu chí).
- **Câu SQL bất biến** chạy được từ bậc này.

Bậc không đổi schema thì ghi rõ "không đổi so với bậc trước".

Cách đọc từ điển cột. Mỗi bảng có (1) một câu nói bảng này **là gì và vì sao cần**, (2) bảng cột với hai cột giải thích: **Là gì** (ý nghĩa), **Vì sao tồn tại** (nếu bỏ đi thì hỏng ở đâu). Khi tự thiết kế, hãy thử bỏ từng cột và hỏi "lỗi nào sẽ xảy ra?". Nếu không nghĩ ra lỗi, cột đó có thể thừa.

Cách tự học với schema (làm trước, đối chiếu sau):

1. Trước khi làm Giai đoạn 1, đọc mục 1.3 (bút toán của từng loại chuyển tiền). Trên giấy, tự trả lời: *với mỗi loại chuyển tiền, dòng nào được ghi vào bảng nào, và trạng thái nào được lưu ở đâu?*
2. Ở mỗi bậc có đổi schema, **trước khi** đọc khối "Tài liệu schema", tự trả lời: *bậc này cần thêm hoặc đổi bảng/cột nào, vì sao?* rồi viết migration của bạn. Chưa xem đáp án.
3. Đọc khối "Tài liệu schema" để đối chiếu: migration, từ điển cột, index, rubric. Khác với đáp án **không nhất thiết là sai**; hãy tự hỏi: thiết kế của mình có chặn được lỗi mà đáp án chặn không?
4. Hết bậc 1.9 bạn có schema giai đoạn 1 của riêng mình. Ở bậc 1.10, tự vẽ ERD (so với ERD ở đầu Giai đoạn 2), chạy các câu SQL bất biến và test song song 1000 request.

Quy ước SQL. Mọi lệnh SQL viết cho **PostgreSQL 16**. Dấu `:tên` (như `:amount` , `:from_id`) là **tham số có tên** của Spring (`NamedParameterJdbcTemplate` , `@Query`), không phải cú pháp riêng của PostgreSQL. Khi chạy tay trong `psql` , dùng `\set amount 30000` rồi viết `:amount` , hoặc thay thẳng bằng giá trị. **Schema chốt cho từng bậc nằm ngay trong khối "Tài liệu schema" của bậc đó** (V0 ở bậc 1.1, rồi mỗi bậc một migration), cùng ba tài khoản mẫu A, B, C có UUID cố định, nên lệnh ở bậc nào cũng chạy được nguyên văn trên schema của bậc đó.

2.0.1 Bản đồ tổng quan

| GD | Bài toán chính | Các bậc (đi từ đâu tới đâu) | Bảng mới (migration) |

Thứ tự làm: 0 → 1 → 2 → 3 → (4 hoặc 9, độc lập nhau) → 5 → 6 → 7 → 8. Xem mục 5.

2.0.2 Nguyên tắc thiết kế schema chung

- **Khóa chính UUID** cho thực thể nghiệp vụ (`accounts` , `transfers` ...), sinh ở ứng dụng (UUIDv7 nếu có thư viện, vì có thứ tự thời gian, index thân thiện). Dùng `BIGSERIAL/IDENTITY` cho bảng chỉ ghi thêm, khối lượng lớn (`ledger_entries` , `history`, `outbox`).
- **Tiền là BIGINT** ở đơn vị nhỏ nhất, luôn đi kèm `currency CHAR(3)` . Không `FLOAT/DOUBLE` .
- **Thời gian là TIMESTAMPTZ** , lưu UTC.
- **Trạng thái/loại là VARCHAR + CHECK** , không dùng ENUM của Postgres (đổi giá trị ENUM khó hơn khi migrate).
- **Ép ràng buộc ở DB**, không chỉ ở code: `CHECK` , `UNIQUE` , `FOREIGN KEY` , `NOT NULL` . Code sai thì DB vẫn chặn được.
- Đặt tên: bảng số nhiều `snake_case` , index `ix_*` , unique `uq_*` , check `ck_*` .

Ba cột lặp lại ở nhiều bảng (`id` , `created_at / updated_at` , `version`) được giải thích ngay ở lần đầu chúng xuất hiện (bậc 1.1, 1.5 và 1.9).

2.0.3 Quy ước migration và bảng migration theo bậc

- Mọi lệnh viết cho **PostgreSQL 16** (đã chạy thử nguyên văn trên PostgreSQL 16, theo đúng thứ tự xuất hiện trong tài liệu).
- Với Flyway, mỗi bậc đổi schema có một file `V<giai đoạn>_<bậc>__tên.sql` (số phiên bản tăng dần là đủ), ví dụ `V1_5__ledger_entries.sql` . Chạy theo thứ tự bậc.
- Vài migration có `TRUNCATE` . Lý do: thêm cột `NOT NULL` mới vào bảng đã có dòng cũ thì PostgreSQL từ chối nếu không có giá trị mặc định hợp lý. Ở môi trường học, xóa dữ liệu thử là xong. Ở production thật phải làm ba bước: thêm cột cho phép `NULL` , điền giá trị cho dòng cũ (backfill), rồi mới đặt `NOT NULL` .
- Các view cho giao diện quan sát (`ops_1` , `ops_2` , `ops_3` , mục 8.3) không thêm bảng; chạy sau bậc 1.9, sau bậc 4.5 và sau bậc 7.4.

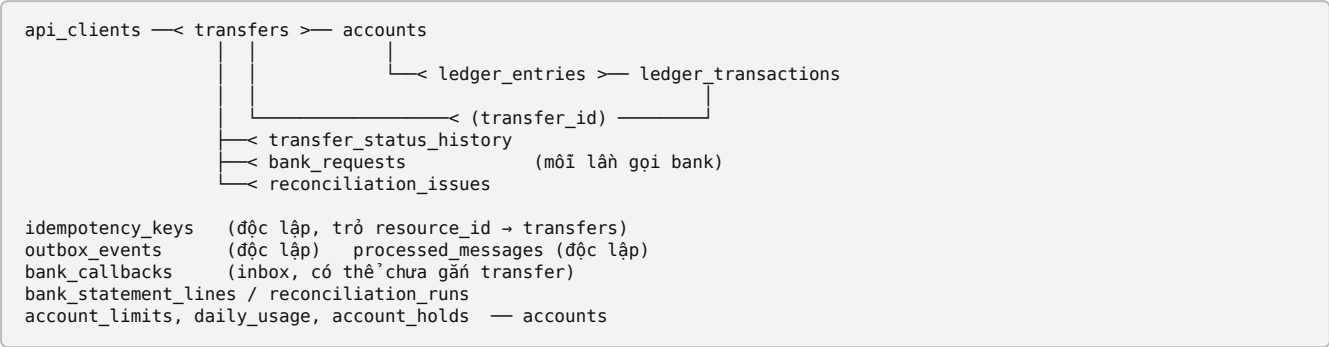
Bậc	File migration	Thêm hoặc đổi gì
1.1	V0__naive.sql + dữ liệu mẫu	<code>accounts</code> , <code>transfers</code> bản ngây thơ
1.5	V1_5__ledger_entries.sql	<code>ledger_entries</code> (trò thẳng <code>transfers</code>)
1.6	V1_6__ledger_transactions.sql	<code>ledger_transactions</code> ; <code>ledger_entries</code> trở về nó
1.7	V1_7__account_seq.sql	<code>accounts.entry_seq</code> , <code>ledger_entries.account_seq</code> + <code>UNIQUE</code>
1.8	V1_8__db_guards.sql	<code>CHECK</code> , <code>UNIQUE</code> (<code>transfer_id</code> , <code>type</code>) , hai trigger
1.9	V1_9__transfer_state.sql	<code>transfers</code> có vòng đời, <code>transfer_status_history</code> , <code>accounts</code> có loại
2.2	V2_2__idempotency_columns.sql	<code>api_clients</code> , <code>transfers.client_id/idempotency_key</code>

Bậc	File migration	Thêm hoặc đổi gì
2.3	V2_3__idempotency_keys.sql	idempotency_keys
2.5	V2_5__source_idempotency.sql	transfers.source_type/source_id + UNIQUE nghiệp vụ
3.4	V3_4__outbox.sql	outbox_events
3.5	V3_5__processed_messages.sql	processed_messages
4.1	V4_1__fake_bank.sql (DB Fake Bank)	bank_accounts , bank_transactions , bank_callback_outbox , chaos_rules
4.3	V4_3__bank_requests.sql	bank_requests
4.5	V4_5__bank_callbacks.sql	bank_callbacks
5.2	V5_2__limits.sql , V5_2_1__holds_optional.sql (tùy chọn)	account_limits , daily_usage ; accounts.held_balance , account_holds
7.3	V7_3__reconciliation.sql	reconciliation_runs , reconciliation_issues
7.4	V7_4__bank_statement.sql	bank_statement_lines
9.2	V9_2__order_payments.sql (DB thương mại)	orders.payment_status , order_payments

Điểm mấu chốt về thiết kế: **transfers** là ý định nghiệp vụ có trạng thái; **ledger_transactions** là bút toán thực sự. Một transfer có thể sinh nhiều ledger transaction (chuyển ra ngân hàng: giữ tiền, rồi quyết toán hoặc hoàn tiền).

2.0.4 Sơ đồ quan hệ đích của wallet-service

Đây là hình dạng bạn sẽ có khi xong Giai đoạn 7. Mỗi giai đoạn bên dưới có sơ đồ riêng cho thời điểm bắt đầu giai đoạn đó, nên đây chỉ là bản nhìn tổng thể.



Giai đoạn 0: Chốt định nghĩa "đúng" và dựng môi trường (1-2 ngày)

Bài toán: chưa định nghĩa thế nào là đúng thì sau này không có gì để kiểm tra bug.

Trạng thái database trước khi bắt đầu giai đoạn này

Chưa có gì. Giai đoạn này không đụng tới database nghiệp vụ.

(chưa có bảng nào; chỉ có database trống và Flyway)

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	không có
Giai đoạn này thêm	không có bảng nào

Migration đã phải chạy	Chưa có migration nghiệp vụ nào.
Dữ liệu cần có	Không cần.

Kiểm tra database đã đúng trạng thái chưa (chạy trong `psql`):

```
SELECT string_agg(table_name, ', ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

(một dòng trống: chưa có bảng nào)

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

0.1 Chốt phạm vi.

- Trong: tài khoản, chuyển nội bộ, chuyển ra/nạp vào ngân hàng ngoài, hoàn tiền, phí, hạn mức, đối soát.
- Ngoài (ghi rõ để khỏi sa đà): KYC, đa tiền tệ (chỉ VND), lãi suất, giao diện, đăng ký/dăng nhập.
- ✓ docs/scope.md

0.2 Chốt bất biến, viết thành SQL (mục 0). Bàn đầy đủ các câu SQL ở ngay bên dưới.

- ✓ docs/invariants.md và lớp test `InvariantChecker` gọi được sau mọi test.

Các câu SQL bất biến (bàn đầy đủ). Chép nguyên vào `docs/invariants.md`; `InvariantChecker` chạy chúng sau mọi test. Mỗi câu chỉ chạy được từ bậc ghi trong bảng, vì trước đó bảng/cột chưa tồn tại (khi đến bậc đó, câu được nhắc lại trong khối tài liệu schema).

Câu	Kiểm tra gì	Chạy được từ bậc
1	Tổng debit = tổng credit toàn hệ thống (bất biến 1)	1.5
1b	Bút toán nào không cân	1.6
2	Không ví <code>USER</code> nào âm (bất biến 2)	1.9
4	<code>accounts.balance</code> lệch tổng sổ cái (bất biến 4)	1.5
5	Transfer nội bộ <code>COMPLETED</code> nhưng không có bút toán	1.9
6	Khoảng trống trong <code>account_seq</code>	1.7
7	Bút toán rỗng	1.8

Bất biến số 3 ở mục 0 (một yêu cầu chỉ một hiệu ứng tiền) không có câu SQL riêng: nó được ép bằng các ràng buộc `UNIQUE` của Giai đoạn 2, nên vi phạm thì DB từ chối ngay.

```
-- (1) Tổng debit = tổng credit toàn hệ thống
SELECT
  SUM(CASE WHEN direction='DEBIT' THEN amount ELSE 0 END) AS total_debit,
  SUM(CASE WHEN direction='CREDIT' THEN amount ELSE 0 END) AS total_credit
FROM ledger_entries;

-- (1b) Ledger transaction nào không cân
SELECT ledger_transaction_id,
  SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END) AS diff
FROM ledger_entries GROUP BY ledger_transaction_id HAVING SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -
amount END) <> 0;

-- (2) Không tài khoản USER nào âm
SELECT id, balance FROM accounts WHERE type='USER' AND balance < 0;

-- (4) Balance cache lệch ledger
```

```

SELECT a.id, a.balance AS cached,
       COALESCE(SUM(CASE WHEN e.direction='CREDIT' THEN e.amount ELSE -e.amount END),0) AS from_ledger
FROM accounts a LEFT JOIN ledger_entries e ON e.account_id = a.id
GROUP BY a.id, a.balance
HAVING a.balance <> COALESCE(SUM(CASE WHEN e.direction='CREDIT' THEN e.amount ELSE -e.amount END),0);

-- (5) Transfer nội bộ COMPLETED nhưng không có bút toán
SELECT t.id FROM transfers t
LEFT JOIN ledger_transactions l ON l.transfer_id = t.id AND l.type = 'PRINCIPAL'
WHERE t.type = 'INTERNAL' AND t.status = 'COMPLETED' AND l.id IS NULL;

-- (6) Khoảng trống trong account_seq (dấu hiệu ghi sai)
SELECT account_id, account_seq FROM (
  SELECT account_id, account_seq,
         LAG(account_seq) OVER (PARTITION BY account_id ORDER BY account_seq) AS prev
  FROM ledger_entries) x
WHERE prev IS NOT NULL AND account_seq <> prev + 1;

-- (7) Bút toán RỘNG: có ledger_transactions nhưng không có dòng nào
-- (trigger cân bút toán chỉ chạy khi có dòng được thêm nên không bắt được trường hợp này)
SELECT l.id AS empty_ledger_transaction
FROM ledger_transactions l
LEFT JOIN ledger_entries e ON e.ledger_transaction_id = l.id
WHERE e.id IS NULL;

```

0.3 Chốt quy ước tiền và bút toán: VND BIGINT , DEBIT = tiền ra, CREDIT = tiền vào, các tài khoản hệ thống (mục 1.3).

- ✓ docs/ledger-conventions.md

0.4 Dựng môi trường. docker-compose.yml (PostgreSQL; Kafka KRaft có thể để đến Giai đoạn 3), Spring Boot skeleton, Flyway, Testcontainers.

- ✓ Một test rỗng chạy xanh với Postgres thật.

0.5 Viết hai ADR ngắn (bối cảnh, quyết định, đánh đổi): "Ứng dụng standalone, DB riêng" và "Ledger double-entry, append-only".

- ✓ docs/adr/001-*.md , 002-*.md

0.6 Chưa thiết kế schema cuối. Schema sẽ **tiến hóa** theo các bậc ở Giai đoạn 1. Mỗi bậc bạn tự nghĩ schema cần đổi gì, viết migration, rồi mới đối chiếu với khối "Tài liệu schema" ở cuối bậc đó.

Tổng kết giai đoạn 0

Bạn vừa làm	Định nghĩa phạm vi, bất biến, quy ước bút toán, dựng môi trường
Bài toán đã giải	"Đúng" có định nghĩa đo được (các câu SQL), không còn là cảm giác
Còn hờ	Chưa có dòng code nghiệp vụ nào
Hướng đi tiếp	GB 1: bắt đầu từ CRUD ngây thơ

Giai đoạn 1: Từ CRUD đến chuyển tiền nội bộ đúng đắn

Bài toán: hai người cùng rút từ một tài khoản, hoặc $A \rightarrow B$ và $B \rightarrow A$ cùng lúc, hoặc process chết giữa chừng. Hệ thống không được âm tiền, mất tiền, deadlock, hay để số dư không giải thích được.

Trạng thái database trước khi bắt đầu giai đoạn này

Database trống, chưa có bảng nghiệp vụ nào. Toàn bộ sơ đồ dưới đây là thứ bạn sẽ xây **trong** giai đoạn này (mọi bảng đều đánh dấu [MỚI]).

```

transfers [MÔI] >— accounts [MÔI] (transfers.from_account_id / to_account_id)
└─< transfer_status_history [MÔI]
    └─< ledger_transactions [MÔI] —< ledger_entries [MÔI] >— accounts (ledger_entries.account_id)
        (transfers còn tự trở lại chính nó: reversal_of_transfer_id)

```

Bảng đứng riêng, không khóa ngoại tới transfers:
(chưa có)

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	không có
Giai đoạn này thêm	accounts, transfers, transfer_status_history, ledger_transactions, ledger_entries (hình thành dần qua bậc 1.1 đến 1.9, mỗi bậc có khối tài liệu schema riêng)
Migration đã phải chạy	Chưa có migration nghiệp vụ nào. Flyway đã chạy được (bảng flyway_schema_history xuất hiện sau lần chạy đầu, không tính vào danh sách bảng).
Dữ liệu cần có	Không cần. Dữ liệu mẫu A, B, C nạp ở bậc 1.1.

Kiểm tra database đã đúng trạng thái chưa (chạy trong psql):

```

SELECT string_agg(table_name, ' ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';

```

Kết quả phải đúng là:

(một dòng trống: chưa có bảng nào)

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Schema tiến hóa qua các bậc (đáp án migration, ý nghĩa từng cột, index, rubric nằm ngay trong khối "Tài liệu schema" ở cuối mỗi bậc):

Bậc	Thay đổi schema	Migration
1.1	accounts(id, owner_ref, currency, balance, created_at), transfers(id, from_account_id, to_account_id, amount, created_at)	V0__naive.sql + dữ liệu mẫu
1.2-1.4	Không đổi (chỉ đổi cách dùng transaction và khóa)	không có
1.5	Thêm ledger_entries (trò thẳng transfer_id)	V1_5
1.6	Thêm ledger_transactions; ledger_entries trỏ về nó thay vì transfer_id	V1_6
1.7	Thêm accounts.entry_seq, ledger_entries.account_seq + UNIQUE (account_id, account_seq)	V1_7
1.8	Thêm CHECK, UNIQUE (transfer_id, type), trigger append-only và trigger cân bút toán	V1_8
1.9	Thêm transfers.type/status/fee_amount/failure_*/version/..., transfer_status_history, accounts.account_number/type/status/bank_code/...	V1_9
2.2	Thêm api_clients, transfers.client_id/idempotency_key + UNIQUE (client_id, idempotency_key)	V2_2
2.5	Thêm transfers.source_type/source_id + UNIQUE (client_id, source_type, source_id, type)	V2_5

Bậc 1.1: CRUD ngây thơ

- **Vấn đề:** chưa có gì. Đây là điểm xuất phát mà hầu hết mọi người viết đầu tiên.
- **Nâng cấp:** viết `POST /transfers` làm đúng ba lệnh:

```
UPDATE accounts SET balance = balance - :amount WHERE id = :from_id;
UPDATE accounts SET balance = balance + :amount WHERE id = :to_id;
INSERT INTO transfers (id, from_account_id, to_account_id, amount)
VALUES (gen_random_uuid(), :from_id, :to_id, :amount);
```

Schema để chạy các lệnh này là **VO ở khối tài liệu schema bên dưới** (hai bảng `accounts`, `transfers`, ba tài khoản mẫu). Ví dụ chạy tay trong `psql`, chuyển 30.000 từ A sang B:

```
\set from_id '00000000-0000-0000-0000-00000000000a'
\set to_id   '00000000-0000-0000-0000-00000000000b'
\set amount 30000
-- rồi chạy ba lệnh trên; sau đó: SELECT owner_ref, balance FROM accounts ORDER BY owner_ref;
-- kết quả đúng: A = 70000, B = 80000, C = 0, tổng = 150000
```

- **Giải quyết được:** chạy được trong lúc demo, một người dùng, mạng ổn.
- **Còn hờ:** bạn chưa biết nó sai ở đâu. Hãy **tự làm nó hỏng** bằng ba thí nghiệm:
 1. Ném exception (hoặc `kill -9`) giữa hai lệnh `UPDATE` → tiền của A mất, B không nhận.
 2. Hai request cùng rút 80 từ tài khoản có 100, theo kiểu app đọc số dư rồi ghi lại → mất cập nhật, số dư sai.
 3. Hỏi "vì sao số dư của A là 37.000?" → không có cách trả lời.
- **✓ Bàn giao:** ba test đồ chứng minh ba lỗi trên. `docs/why-crud-fails.md` ghi lại kết quả.

Tài liệu schema của bậc 1.1

Bậc này chỉ có **hai bảng** và ba tài khoản mẫu. Cố ý chưa có `CHECK`, trạng thái hay số cái: đó là các lỗ hổng bạn sẽ tự làm lộ ra rồi vá dần. Hãy tự viết hai bảng này trước, rồi so với đáp án.

Migration (`V0__naive.sql`):

```
-- V0 (bậc 1.1): CRUD ngây thơ. Cố ý chưa có CHECK, FK nghiệp vụ, trạng thái...
CREATE TABLE accounts (
  id          UUID PRIMARY KEY,
  owner_ref   VARCHAR(64) NOT NULL,
  currency    CHAR(3) NOT NULL DEFAULT 'VND',
  balance     BIGINT NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE transfers (
  id          UUID PRIMARY KEY,
  from_account_id UUID NOT NULL REFERENCES accounts(id),
  to_account_id  UUID NOT NULL REFERENCES accounts(id),
  amount       BIGINT NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Dữ liệu mẫu cố định:

```
-- Dữ liệu mẫu cố định để mọi lệnh SQL trong tài liệu chạy được nguyên văn
INSERT INTO accounts (id, owner_ref, balance) VALUES
('00000000-0000-0000-0000-00000000000a', 'user-A', 100000),
('00000000-0000-0000-0000-00000000000b', 'user-B', 50000),
('00000000-0000-0000-0000-00000000000c', 'user-C', 0);
```

Ba tài khoản A, B, C có UUID cố định nên mọi lệnh SQL trong tài liệu chạy được nguyên văn. Sau khi chuyển 30.000 từ A sang B, kết quả đúng là A = 70000, B = 80000, C = 0 và tổng vẫn là 150000.

Bảng `accounts`

Cột	Là gì	Vì sao tồn tại
<code>id</code> kiểu UUID	Mã định danh duy nhất của bản ghi, sinh ở ứng dụng	Sinh ở ứng dụng nên biết được id trước khi ghi DB (cần cho idempotency và outbox); UUID không đoán được nên an toàn để đưa ra API
<code>owner_ref</code>	Mã người dùng bên hệ thống thương mại (chuỗi)	Ví không có bảng người dùng riêng (tránh sa đà), chỉ cần biết "ví này của ai" bên ngoài. Ở bậc này bắt buộc có (<code>NOT NULL</code>); bậc 1.9 cho phép NULL để có tài khoản hệ thống
<code>currency</code>	Loại tiền (VND)	Không bao giờ được cộng tiền khác loại; cột này cho phép kiểm tra và mở rộng sau
<code>balance</code>	Số dư hiện tại, <code>BIGINT</code> ở đơn vị nhỏ nhất (đồng)	Ở bậc này nó là nguồn duy nhất của số dư, và đó chính là điểm yếu: không có lịch sử. Từ bậc 1.5 nó chỉ còn là cache của số cái
<code>created_at</code>	Thời điểm tạo (<code>TIMESTAMPZ</code> , UTC)	Điều tra sự cố ("lúc nào?"). <code>updated_at</code> sẽ được thêm ở bậc 1.9

Bảng transfers

Cột	Là gì	Vì sao tồn tại
<code>id</code>	Mã giao dịch (UUID)	Sinh ở ứng dụng nên biết được id trước khi ghi DB
<code>from_account_id</code>	Tài khoản tiền đi ra	<code>REFERENCES accounts</code> : DB chặn trỏ vào tài khoản không tồn tại. Bậc này bắt buộc có; bậc 1.9 cho phép NULL (tiền vào từ ngân hàng ngoài thì không có ví nguồn)
<code>to_account_id</code>	Tài khoản tiền đi vào	Như trên; bậc 1.9 cho phép NULL (tiền ra ngân hàng ngoài thì không có ví đích bên mình)
<code>amount</code>	Số tiền, <code>BIGINT</code> đơn vị nhỏ nhất	Số nguyên để không sai số. Chưa có <code>CHECK (amount > 0)</code> : số âm lọt qua được, bậc 1.8 sẽ chặn
<code>created_at</code>	Thời điểm tạo	Điều tra sự cố và sắp xếp lịch sử

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Khóa chính	UUID cho thực thể, <code>BIGSERIAL</code> cho bảng chỉ ghi	UUID an toàn để lộ ra ngoài, <code>BIGSERIAL</code> rẻ và có thứ tự	Toàn bộ UUID v4 (index phân mảnh)

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Lưu tiền bằng `DECIMAL/FLOAT` lẫn với số nguyên, hoặc quên `currency` .

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

☐ Tiền là số nguyên, có `currency` , không dùng float.

Bậc 1.2: Bọc trong transaction

- **Vấn đề:** thí nghiệm 1, lỗi giữa chừng làm mất tiền.
- **Nâng cấp:** bọc cả ba lệnh trong một transaction (`@Transactional`). Lỗi ở đâu thì rollback tất cả.
- **Giải quyết được:** tính **nguyên tử** (atomicity): hoặc cả hai bên cùng đổi, hoặc không bên nào đổi.
- **Còn hờ:** thí nghiệm 2 vẫn hỏng. Hai transaction cùng đọc số dư cũ rồi cùng trừ (**lost update / race condition**).
- **✓ Bàn giao:** thí nghiệm 1 chuyển xanh.

Tài liệu schema của bậc 1.2: không đổi so với bậc trước (schema vẫn như sau bậc 1.1). Bậc này chỉ đổi cách dùng transaction.

Bậc 1.3: Khóa dòng khi đọc số dư

- **Vấn đề:** thí nghiệm 2, hai luồng cùng vượt qua kiểm tra "đủ tiền" rồi cùng trừ.
- **Nâng cấp:** đọc số dư bằng `SELECT ... FOR UPDATE` (khóa bi quan) trước khi kiểm tra và trừ. Luồng thứ hai phải chờ luồng thứ nhất commit rồi mới đọc.
 - Cách khác cùng mục tiêu: `UPDATE accounts SET balance = balance - :amount WHERE id = :id AND balance >= :amount` và kiểm tra số dòng bị ảnh hưởng. Ghi vào ADR vì sao bạn chọn cách nào (bài này chọn `FOR UPDATE` vì sau này phải khóa nhiều tài khoản cùng lúc và đọc lại nhiều thông tin).
- **Giải quyết được:** double-spend một tài khoản.
- **Còn hờ:** khi khóa **hai** tài khoản, $A \rightarrow B$ và $B \rightarrow A$ chạy cùng lúc, mỗi bên giữ một khóa và chờ khóa còn lại (deadlock).
- **✓ Bàn giao:** test 1000 request song song cùng rút một tài khoản $\rightarrow$ không âm.

Tài liệu schema của bậc 1.3: không đổi so với bậc trước (schema vẫn như sau bậc 1.1). Bậc này chỉ thêm `SELECT ... FOR UPDATE` vào code.

Bậc 1.4: Khóa có thứ tự

- **Vấn đề:** deadlock giữa $A \rightarrow B$ và $B \rightarrow A$. Tái hiện bằng test chạy hai chiều hàng nghìn lần. (Đã đo thử: khóa nguồn rồi mới khóa đích với 16 kết nối song song, hầu hết giao dịch lỗi vì deadlock.)
- **Nâng cấp:** luôn khóa các tài khoản theo **thứ tự cố định** (ví dụ `ORDER BY id`).

```
public interface AccountRepository extends JpaRepository<Account, UUID> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id in :ids order by a.id")
    List<Account> lockAllOrdered(@Param("ids") Collection<UUID> ids);
}
```

- **Giải quyết được:** deadlock do khóa vòng tròn.
- **Còn hờ:** thí nghiệm 3, số dư đúng nhưng **không có lịch sử**. Sai một chỗ thì không truy ra được.
- **✓ Bàn giao:** docs/adr/003-locking.md (vì sao bi quan, vì sao có thứ tự, isolation level) + test hai chiều xanh. Thử bỏ `ORDER BY` để thấy deadlock lại xuất hiện.

Tài liệu schema của bậc 1.4: không đổi so với bậc trước (schema vẫn như sau bậc 1.1). Bậc này chỉ đổi thứ tự khóa trong code.

Bậc 1.5: Sổ cái ghi từng thay đổi (ledger)

- **Vấn đề:** thí nghiệm 3, số dư không giải thích được.
- **Nâng cấp:** thêm bảng `ledger_entries`. Mỗi lần số dư đổi, ghi một dòng (tài khoản nào, DEBIT hay CREDIT, bao nhiêu, số dư sau đó). Bảng chỉ **thêm**, không sửa.
- **Giải quyết được:** truy vết được một số dư từ đâu mà ra (cộng dồn entries phải bằng số dư).
- **Còn hờ:** một dòng DEBIT có thể tồn tại mà **không có** CREDIT tương ứng (tiền biến mất khỏi sổ). Các dòng chưa được nhóm lại thành một giao dịch.
- **✓ Bàn giao:** câu SQL "cache balance lệch ledger" (câu 4 trong khối tài liệu schema bên dưới) trả về 0 dòng sau mọi test.

Tài liệu schema của bậc 1.5

Thêm bảng sổ cái đầu tiên. Ở bậc này mỗi dòng sổ trò thẳng về `transfers`; bậc 1.6 sẽ đổi chỗ trò. Tự hỏi: vì sao một dòng sổ cần cả `direction` lẫn `amount` dương, mà không dùng số âm?

Migration (`V1_5__ledger_entries.sql`):

```
-- Bậc 1.5: số cái ghi từng thay đổi
CREATE TABLE ledger_entries (
  id          BIGSERIAL PRIMARY KEY,
  transfer_id UUID NOT NULL REFERENCES transfers(id),
  account_id  UUID NOT NULL REFERENCES accounts(id),
  direction   VARCHAR(6) NOT NULL,      -- 'DEBIT' | 'CREDIT'
  amount      BIGINT NOT NULL,
  currency    CHAR(3) NOT NULL,
  balance_after BIGINT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Bảng ledger_entries

Cột	Là gì	Vì sao tồn tại
id kiểu BIGSERIAL	Số tự tăng do DB cấp	Dùng cho bảng chỉ ghi thêm khối lượng lớn: nhỏ, nhanh, và có thứ tự , dùng để duyệt "đến đâu rồi" (outbox, sao kê)
transfer_id	Dòng sổ này thuộc giao dịch nào (tạm thời)	Để truy từ sổ ngược về giao dịch. Sẽ bị thay bằng ledger_transaction_id ở bậc 1.6, vì một giao dịch có thể sinh nhiều bút toán
account_id	Tài khoản bị tác động	Số dư tài khoản = tổng các dòng của nó
direction	DEBIT (tiền ra, balance giảm) / CREDIT (tiền vào, balance tăng)	Quy ước "kiểu ví". Tổng credit – tổng debit của một tài khoản = số dư
amount	Số tiền, luôn dương	Chiều đi nằm ở direction, nên không cần số âm (tránh nhầm dấu). CHECK (amount > 0) sẽ thêm ở bậc 1.8
currency	Loại tiền	Kiểm tra cân đối theo từng loại tiền khi mở rộng
balance_after	Số dư của tài khoản ngay sau dòng này	Sao kê hiển thị được "số dư sau giao dịch" mà không cộng dồn lại; đối chiếu nhanh với cache balance
created_at	Thời điểm ghi dòng sổ	Điều tra và sao kê theo thời gian

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Số dư	Cache balance + ledger là nguồn sự thật	Độc nhanh, đối soát được	Chỉ tính từ ledger (chậm khi lớn)

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Set số dư ban đầu trực tiếp bằng UPDATE accounts SET balance = ... , làm ledger lệch ngay từ đầu.

Câu SQL bất biến chạy được từ bậc này (số dư lưu sẵn phải bằng tổng số cái; bàn giao của bậc 1.5):

```
-- (4) Balance cache lệch ledger
SELECT a.id, a.balance AS cached,
       COALESCE(SUM(CASE WHEN e.direction='CREDIT' THEN e.amount ELSE -e.amount END),0) AS from_ledger
FROM accounts a LEFT JOIN ledger_entries e ON e.account_id = a.id
GROUP BY a.id, a.balance
HAVING a.balance <> COALESCE(SUM(CASE WHEN e.direction='CREDIT' THEN e.amount ELSE -e.amount END),0);
```

Bậc 1.6: Bút toán kép và điểm ghi sổ duy nhất

- Vấn đề:** DEBIT không có CREDIT; code ở nhiều chỗ tự ghi entries và tự sửa balance, mỗi chỗ một kiểu.
- Nâng cấp:**
 - Thêm ledger_transactions gom các dòng của một bút toán; quy tắc: **tổng debit = tổng credit** trong mỗi ledger_transaction.
 - Tạo LedgerPostingService.post(...) là nơi **duy nhất** được ghi ledger_entries và đổi accounts.balance.

```

public record Leg(UUID accountId, Direction direction, long amount) {}

@Service
@RequiredArgsConstructor
public class LedgerPostingService {
    private final LedgerTransactionRepository ledgerTxns;
    private final LedgerEntryRepository entries;

    /** Điểm DUY NHẤT được phép ghi số. Gọi trong transaction đang mở, tài khoản đã được khóa. */
    public void post(UUID transferId, String type, Map<UUID, Account> lockedAccounts, List<Leg> legs) {
        long sum = legs.stream()
            .mapToLong(l -> l.direction() == Direction.CREDIT ? l.amount() : -l.amount()).sum();
        if (sum != 0) throw new UnbalancedPostingException();

        LedgerTransaction txn = ledgerTxns.save(LedgerTransaction.of(transferId, type));
        for (Leg leg : legs) {
            Account acc = lockedAccounts.get(leg.accountId());
            acc.apply(leg.direction(), leg.amount()); // đổi balance
            entries.save(LedgerEntry.of(txn.getId(), acc, leg)); // ghi dòng số
        }
    }
}

```

- **Giải quyết được:** tổng tiền hệ thống luôn cân; quy tắc ghi sổ cài đặt đúng một chỗ. Từ giờ, mọi loại chuyển tiền (kể cả phí) chỉ là một danh sách `Leg`.
- **Còn hở:** hai luồng ghi đồng thời sai cách vẫn có thể cho `balance_after` sai mà không ai phát hiện; sổ chưa có thứ tự.
- **✓ Bàn giao:** unit test `post` từ chối bút toán không cân; bảng bút toán của từng loại chuyển tiền (mục 1.3) được cài đặt bằng danh sách `Leg`.

Tài liệu schema của bậc 1.6

Thêm "phong bì" `ledger_transactions` gom các dòng sổ của một sự kiện tiền; mỗi phong bì phải cân (debit = credit). `ledger_entries` đổi sang trỏ về phong bì thay vì trỏ về `transfers`.

Migration (`V1_6__ledger_transactions.sql`):

```

-- Bậc 1.6: bút toán kép. Dữ liệu thư' của bậc 1.5 không có ledger_transaction đề'gắn vào nên bỏ đi.
TRUNCATE ledger_entries;

CREATE TABLE ledger_transactions (
    id                UUID PRIMARY KEY,
    transfer_id       UUID NOT NULL REFERENCES transfers(id),
    type              VARCHAR(30) NOT NULL,
    description        VARCHAR(255),
    reverses_ledger_txn_id UUID REFERENCES ledger_transactions(id),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE ledger_entries DROP COLUMN transfer_id;
ALTER TABLE ledger_entries
    ADD COLUMN ledger_transaction_id UUID NOT NULL REFERENCES ledger_transactions(id);
CREATE INDEX ix_ledger_entries_txn ON ledger_entries (ledger_transaction_id);

```

Bảng `ledger_transactions`: là "phong bì" gom các dòng `ledger_entries` thuộc cùng một sự kiện tiền. Quy tắc: tổng debit = tổng credit **trong từng phong bì**. Ràng buộc `UNIQUE (transfer_id, type)` nhắc ở cột `type` được thêm ở bậc 1.8.

Cột	Là gì	Vì sao tồn tại
<code>transfer_id</code>	Bút toán thuộc transfer nào	Một transfer có nhiều bút toán; cần truy ngược từ sổ về ý định nghiệp vụ

Cột	Là gì	Vì sao tồn tại
type	PRINCIPAL , FEE , TO_CLEARING , SETTLE , REFUND , TOPUP , ADJUSTMENT	Mỗi bước của luồng là một loại. Kết hợp với transfer_id thành UNIQUE (transfer_id, type) : một bước chỉ được ghi sổ một lần , kể cả khi message bị giao lại hay recovery chạy lại
description	Mô tả	Hiện thị trên sao kê và khi điều tra
reverses_ledger_txn_id	Bút toán nào bị đảo bởi bút toán này	Khi sửa sai: tạo bút toán đảo trở về bút toán sai, không xóa

Bảng ledger_entries (cột đối ở bậc này; cột transfer_id của bậc 1.5 bị bỏ)

Cột	Là gì	Vì sao tồn tại
ledger_transaction_id	Thuộc bút toán nào	Gom các dòng thành nhóm để kiểm tra cân đối

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Cách ghi sổ	ledger_transactions + ledger_entries (mỗi tài khoản một dòng)	Có balance_after , account_seq theo tài khoản; một transfer sinh được nhiều bút toán	Một dòng/giao dịch có debit_account và credit_account (gọn hơn, kiểu TigerBeetle), khó gán số dư theo từng tài khoản

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Chỉ có bảng accounts.balance và bảng transactions một dòng/giao dịch, không có bút toán kép.

Câu SQL bất biến chạy được từ bậc này (toàn hệ thống cân; bút toán nào không cân):

```
-- (1) Tổng debit = tổng credit toàn hệ thống
SELECT
  SUM(CASE WHEN direction='DEBIT' THEN amount ELSE 0 END) AS total_debit,
  SUM(CASE WHEN direction='CREDIT' THEN amount ELSE 0 END) AS total_credit
FROM ledger_entries;

-- (1b) Ledger transaction nào không cân
SELECT ledger_transaction_id,
  SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END) AS diff
FROM ledger_entries GROUP BY ledger_transaction_id HAVING SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END) <> 0;
```

Lưu ý: migration có TRUNCATE ledger_entries vì dòng sổ cũ không có ledger_transaction để gán vào. Ở môi trường học thì xóa dữ liệu thử là xong; production phải backfill (xem mục 2.0.3).

Bậc 1.7: Thứ tự bút toán theo từng tài khoản

- Vấn đề:** nếu một đoạn code quên khóa, hai luồng cùng ghi, balance_after có thể sai nhưng vẫn "trông hợp lệ". Cần một cách để DB **tự tổ cáo**.
- Nâng cấp:** mỗi tài khoản có entry_seq (số thứ tự bút toán gần nhất). Khi ghi dòng mới: tăng entry_seq và gán vào ledger_entries.account_seq, kèm UNIQUE (account_id, account_seq) .
- Giải quyết được:** ghi song song sai sẽ va vào unique và **thất bại rõ ràng** thay vì lặng lẽ sai. Sao kê phân trang theo account_seq (không dùng OFFSET).
- Còn hờ:** mọi quy tắc trên chỉ nằm ở **code**. Một câu SQL chạy tay có thể phá tất cả.
- ✓ **Bàn giao:** câu SQL kiểm tra "khoảng trống trong account_seq " (câu 6 trong khối tài liệu schema bên dưới) trả về 0 dòng.

Tài liệu schema của bậc 1.7

Thêm số thứ tự bút toán theo từng tài khoản để DB **tự tổ cáo** khi hai luồng ghi đồng thời mà quên khóa.

Migration (`v1_7__account_seq.sql`):

```
-- Bậc 1.7: thứ tự bút toán theo tài khoản (bảng ledger_entries đang rỗng nên thêm cột NOT NULL được)
TRUNCATE ledger_entries;
ALTER TABLE accounts ADD COLUMN entry_seq BIGINT NOT NULL DEFAULT 0;
ALTER TABLE ledger_entries ADD COLUMN account_seq BIGINT NOT NULL;
ALTER TABLE ledger_entries
  ADD CONSTRAINT uq_ledger_entries_account_seq UNIQUE (account_id, account_seq);
```

Bảng accounts (cột mới)

Cột	Là gì	Vì sao tồn tại
entry_seq	Số thứ tự bút toán gần nhất của tài khoản	Mỗi bút toán mới lấy <code>entry_seq + 1</code> gán vào <code>ledger_entries.account_seq</code> . Nếu hai luồng ghi đồng thời mà không khóa đúng, DB báo trùng thay vì để sai lằng lẽ

Bảng ledger_entries (cột mới)

Cột	Là gì	Vì sao tồn tại
account_seq	Thứ tự dòng trong tài khoản (1, 2, 3...)	<code>UNIQUE (account_id, account_seq)</code> : phát hiện ghi song song sai; duyệt sao kê theo <code>account_seq</code> (không dùng <code>OFFSET</code>); khoảng trống trong dãy là dấu hiệu bug

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
<code>uq_ledger_entries_account_seq</code> (<code>account_id</code> , <code>account_seq</code>)	Sao kê theo tài khoản phân trang theo <code>account_seq</code> (không dùng <code>OFFSET</code>), đồng thời phát hiện ghi song song sai: hai luồng cùng lấy một số thứ tự thì một bên bị từ chối

Điểm dễ sai: `entry_seq` trong `accounts` + `account_seq` trong `ledger_entries` : khi khóa tài khoản, tăng `entry_seq` rồi gán cho dòng mới. `UNIQUE (account_id, account_seq)` sẽ **phát hiện ngay** nếu có hai luồng ghi đồng thời mà không khóa đúng cách (lỗi mà `balance` cache có thể che mắt).

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

- ☐ Có thứ tự bút toán theo tài khoản (`account_seq`) hoặc cơ chế tương đương để phát hiện ghi song song sai.

Câu SQL bất biến chạy được từ bậc này (khoảng trống trong `account_seq` là dấu hiệu ghi sai; bàn giao của bậc 1.7):

```
-- (6) Khoảng trống trong account_seq (dấu hiệu ghi sai)
SELECT account_id, account_seq FROM (
  SELECT account_id, account_seq,
    LAG(account_seq) OVER (PARTITION BY account_id ORDER BY account_seq) AS prev
  FROM ledger_entries) x
WHERE prev IS NOT NULL AND account_seq <> prev + 1;
```

Bậc 1.8: Đưa ràng buộc xuống database

- Vấn đề:** ai đó (hoặc bạn, lúc debug) `UPDATE ledger_entries`, `DELETE`, hoặc set số dư âm bằng SQL tay.
- Nâng cấp:** ép ở DB (migration và giải thích từng ràng buộc ở khối tài liệu schema bên dưới):
 - `CHECK (balance >= 0)` ở bậc này (chưa có cột `type`); sang bậc 1.9 có tài khoản hệ thống được phép âm thì thu hẹp thành `CHECK (type <> 'USER' OR balance >= 0)`.
 - Trigger chặn `UPDATE/DELETE` trên `ledger_entries` (append-only).

- Trigger `DEFERRABLE INITIALLY DEFERRED` kiểm tra mỗi `ledger_transaction` cân khi commit.
- `UNIQUE (transfer_id, type)` trên `ledger_transactions` để một bước không bị ghi sổ hai lần.
- `FOREIGN KEY, NOT NULL, CHECK (amount > 0)` khắp nơi.
- **Giải quyết được:** code sai vẫn bị DB chặn. Đây là "hàng phòng thủ cuối".
- **Còn hở:** `transfers` vẫn chỉ là một dòng ghi lại; chưa có trạng thái; giao dịch bị từ chối (thiếu tiền) thì rollback và **biến mất** không dấu vết. Ngoài ra trigger cân bút toán không bắt được một `ledger_transaction` rỗng (không có dòng nào), cần câu kiểm tra số 7 (khối tài liệu schema bên dưới).
- **✓ Bàn giao:** test "DB tự bảo vệ": cố tình insert số dư âm, ledger không cân, UPDATE/DELETE ledger, ghi trùng bước; DB từ chối tất cả.

Tài liệu schema của bậc 1.8

Bậc này **không thêm cột nào**. Nó đưa các quy tắc xuống database: DB phải từ chối được dữ liệu sai dù code sai hoặc ai đó gõ SQL tay. Tự liệt kê trước: *những quy tắc nào của sổ cái có thể ép bằng `CHECK`, `UNIQUE`, `trigger`?*

Migration (`V1_8__db_guards.sql`):

```
-- Bậc 1.8: đưa ràng buộc xuống database
ALTER TABLE accounts ADD CONSTRAINT ck_accounts_balance CHECK (balance >= 0); -- bậc 1.9 sẽ thu hẹp
ALTER TABLE transfers ADD CONSTRAINT ck_transfers_amount CHECK (amount > 0);
ALTER TABLE transfers ADD CONSTRAINT ck_transfers_diff_accounts CHECK (from_account_id <> to_account_id); --
bậc 1.9 thay bằng ck_transfers_internal

ALTER TABLE ledger_transactions ADD CONSTRAINT ck_ledger_txn_type CHECK (type IN
('PRINCIPAL', 'FEE', 'TO_CLEARING', 'SETTLE', 'REFUND', 'TOPUP', 'ADJUSTMENT'));
CREATE UNIQUE INDEX uq_ledger_txn_step ON ledger_transactions (transfer_id, type);

ALTER TABLE ledger_entries ADD CONSTRAINT ck_ledger_entries_direction CHECK (direction IN
('DEBIT', 'CREDIT'));
ALTER TABLE ledger_entries ADD CONSTRAINT ck_ledger_entries_amount CHECK (amount > 0);

CREATE FUNCTION fn_ledger_entries_immutable() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'ledger_entries is append-only';
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ledger_entries_immutable
BEFORE UPDATE OR DELETE ON ledger_entries
FOR EACH ROW EXECUTE FUNCTION fn_ledger_entries_immutable();

CREATE FUNCTION fn_ledger_txn_balanced() RETURNS trigger AS $$
DECLARE diff BIGINT;
BEGIN
    SELECT COALESCE(SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END), 0)
    INTO diff FROM ledger_entries WHERE ledger_transaction_id = NEW.ledger_transaction_id;
    IF diff <> 0 THEN
        RAISE EXCEPTION 'ledger_transaction % is not balanced (diff=%)', NEW.ledger_transaction_id, diff;
    END IF;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_ledger_txn_balanced
AFTER INSERT ON ledger_entries
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_ledger_txn_balanced();
```

Các ràng buộc vừa thêm và chúng chặn gì:

Ràng buộc	Chặn lỗi nào
<code>ck_accounts_balance CHECK (balance >= 0)</code>	Số dư âm do code sai hoặc SQL tay. Bậc 1.9 thu hẹp thành "chỉ ví <code>USER</code> mới không được âm" vì tài khoản hệ thống được phép âm

Ràng buộc	Chặn lỗi nào
<code>ck_transfers_amount</code> CHECK (<code>amount > 0</code>)	Số 0 và số âm. Số âm là cách gian lận kinh điển để "chuyển ngược"
<code>ck_transfers_diff_accounts</code> CHECK (<code>from <> to</code>)	Chuyển cho chính mình. Bậc 1.9 thay bằng <code>ck_transfers_internal</code> , vì khi đó <code>from / to</code> có thể NULL
<code>ck_ledger_txn_type</code> CHECK (<code>type IN ...</code>)	Loại bút toán lạ do gõ nhầm
<code>uq_ledger_txn_step</code> UNIQUE (<code>transfer_id, type</code>)	Ghi số trùng một bước của một giao dịch (message giao lại, recovery chạy lại). Đây là "chốt chặn cuối" ngoài idempotency ở tầng trên
<code>ck_ledger_entries_direction</code> , <code>ck_ledger_entries_amount</code>	Chiều tiền lạ; số tiền không dương
<code>trg_ledger_entries_immutable</code>	Mọi UPDATE / DELETE trên sổ cái (append-only)
<code>trg_ledger_txn_balanced</code> (deferred)	Bút toán không cân, kiểm tra lúc COMMIT

Điểm dễ sai:

- `uq_ledger_txn_step` là "chốt chặn cuối" chống ghi số trùng, ngoài idempotency ở tầng trên.
- Trigger cân bút toán là `DEFERRABLE INITIALLY DEFERRED` để bạn được insert từng dòng rồi kiểm tra một lần lúc commit.
- Trigger chặn UPDATE/DELETE không chặn TRUNCATE; dùng TRUNCATE để dọn dữ liệu giữa các test.

Lưu ý: trigger "bút toán phải cân" chỉ chạy khi có dòng `ledger_entries` được thêm vào. Một `ledger_transactions` **không có dòng nào** vẫn lọt. Câu kiểm tra số 7 bên dưới bắt trường hợp này.

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Chống ghi số trùng	<code>UNIQUE (transfer_id, type)</code>	Chặn ở DB, không phụ thuộc code	Chỉ kiểm tra ở code (dễ lọt khi message giao lại)

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Cho phép UPDATE/DELETE trên bảng ledger.
- Chỉ kiểm tra "số dư không âm" ở code mà không có CHECK ở DB.
- Tin rằng trigger cân bút toán là đủ: nó không bắt được một `ledger_transactions` rỗng (không có dòng nào). Cần câu kiểm tra số 7 (ở khối tài liệu của bậc 1.8).

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

- ☐ Ledger append-only, được **ép ở DB** (trigger hoặc quyền), không chỉ quy ước.
- ☐ Mỗi ledger transaction cân (`debit = credit`), có cơ chế ép ở DB hoặc có test bắt buộc.
- ☐ Chặn ghi số trùng ở DB (ví dụ `UNIQUE (transfer_id, type)`).
- ☐ Số dư không âm được ép bằng CHECK ở DB.

Câu SQL bất biến chạy được từ bậc này (bút toán rỗng; trigger không bắt được):

```
-- (7) Bút toán RỖNG: có ledger_transactions nhưng không có dòng nào
-- (trigger cân bút toán chỉ chạy khi có dòng được thêm nên không bắt được trường hợp này)
SELECT l.id AS empty_ledger_transaction
FROM ledger_transactions l
LEFT JOIN ledger_entries e ON e.ledger_transaction_id = l.id
WHERE e.id IS NULL;
```

Bậc 1.9: Transfer có trạng thái và lỗi nghiệp vụ có lưu vết

- **Vấn đề:** giao dịch bị từ chối không được ghi lại (mất audit); không biết một transfer đã đi qua những bước nào; phí chưa có chỗ đứng.
- **Nâng cấp:**
 - transfers có type, status (CREATED → VALIDATED → COMPLETED | FAILED), fee_amount, failure_code, failure_reason; thêm transfer_status_history; accounts có type, status, account_number. Migration bậc 1.9 ở khối tài liệu schema bên dưới (có TRUNCATE, đọc ghi chú ở đó). Xong bậc này, tạo view ops_1 và tài khoản ops_reader (mục 8.3) để nhìn kết quả bằng giao diện thay vì psql.
 - Luồng chuyển nội bộ trong **một** transaction:

```
1. Kiểm tra đầu vào (amount > 0, from ≠ to, currency khớp) -- trước transaction
2. BEGIN
3. Khóa from, to theo thứ tự id
4. Kiểm tra cả hai ACTIVE, đủ tiền (amount + fee)
5. INSERT transfers (CREATED) + history
6. post(PRINCIPAL: DEBIT from, CREDIT to)
7. nếu fee > 0: post(FEE: DEBIT from, CREDIT SYSTEM_FEE)
8. UPDATE transfers SET status='COMPLETED' + history
9. COMMIT
```

- Lỗi nghiệp vụ: transaction chính rollback, sau đó ghi transfers ở trạng thái FAILED (kèm failure_code) trong một transaction riêng lẻ.

Loại lỗi	Ví dụ	Xử lý
Đầu vào sai	amount ≤ 0, thiếu trường	400, không tạo transfer
Từ chối nghiệp vụ	thiếu tiền, tài khoản đóng băng	422, ghi transfer FAILED
Lỗi hạ tầng	mất kết nối DB	503, không ghi gì, client được retry

- Hợp đồng API và mã lỗi:

```
// POST /transfers
{ "type": "INTERNAL", "fromAccountId": "3f2c...", "toAccountId": "9a1b...",
  "amount": 50000, "currency": "VND", "description": "Thanh toán đơn #1234",
  "source": { "type": "ORDER", "id": "1234" } }
// 201
{ "id": "b7d0...", "status": "COMPLETED", "amount": 50000, "fee": 0 }
// lỗi
{ "error": { "code": "INSUFFICIENT_FUNDS", "message": "Số dư không đủ", "transferId": "b7d0..." } }
```

Mã lỗi	HTTP	Ý nghĩa
INVALID_AMOUNT	400	amount ≤ 0
ACCOUNT_NOT_FOUND	404	không có tài khoản
ACCOUNT_NOT_ACTIVE	422	đóng băng/đóng
INSUFFICIENT_FUNDS	422	không đủ số dư
CURRENCY_MISMATCH	422	khác loại tiền
INTERNAL_ERROR	500/503	lỗi hạ tầng, không lộ chi tiết

- Nạp tiền dev bằng một bút toán từ SYSTEM_BANK sang ví (không UPDATE balance trực tiếp, nếu không câu kiểm tra bất biến 4 sẽ báo lệch ngay).
- **Giải quyết được:** mọi giao dịch, kể cả bị từ chối, đều có vết. Có hợp đồng API rõ ràng.
- **Còn hờ:** client gửi lại cùng một yêu cầu thì **trừ tiền hai lần** (giai đoạn 2).
- **✓ Bàn giao:** API chạy được bằng curl; docs/api-contract.md; docs/transfer-flow.md.

Tài liệu schema của bậc 1.9

Bậc lớn nhất về schema: `transfers` có vòng đời, tài khoản có loại, và có bảng lịch sử chuyển trạng thái. Tự hỏi trước: *giao dịch bị từ chối cần lưu những gì để audit? tài khoản hệ thống khác ví người dùng ở điểm nào?*

Migration (`V1_9__transfer_state.sql`):

```
-- Bậc 1.9: transfer có trạng thái, tài khoản có loại.
-- Các cột NOT NULL mới không có giá trị mặc định hợp lý cho dòng cũ, nên xóa dữ liệu thư' (chỉ làm được trong
môi trường học).
-- Ở production thật phải: thêm cột cho phép NULL -> backfill -> mới đặt NOT NULL.
TRUNCATE accounts CASCADE;

-- accounts
ALTER TABLE accounts DROP CONSTRAINT ck_accounts_balance;
ALTER TABLE accounts ALTER COLUMN owner_ref DROP NOT NULL;
ALTER TABLE accounts
  ADD COLUMN account_number VARCHAR(20) NOT NULL UNIQUE,
  ADD COLUMN type           VARCHAR(30) NOT NULL,
  ADD COLUMN bank_code      VARCHAR(20),
  ADD COLUMN status         VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  ADD COLUMN version        BIGINT NOT NULL DEFAULT 0,
  ADD COLUMN updated_at     TIMESTAMPTZ NOT NULL DEFAULT now();
ALTER TABLE accounts
  ADD CONSTRAINT ck_accounts_type CHECK (type IN ('USER', 'SYSTEM_FEE', 'SYSTEM_CLEARING', 'SYSTEM_BANK')),
  ADD CONSTRAINT ck_accounts_status CHECK (status IN ('ACTIVE', 'FROZEN', 'CLOSED')),
  ADD CONSTRAINT ck_accounts_user_balance CHECK (type <> 'USER' OR balance >= 0),
  ADD CONSTRAINT ck_accounts_owner CHECK (type <> 'USER' OR owner_ref IS NOT NULL);
CREATE UNIQUE INDEX uq_accounts_owner_currency ON accounts (owner_ref, currency) WHERE type = 'USER';
CREATE UNIQUE INDEX uq_accounts_system_bank ON accounts (bank_code) WHERE type = 'SYSTEM_BANK';

-- transfers
ALTER TABLE transfers DROP CONSTRAINT ck_transfers_diff_accounts;
ALTER TABLE transfers ALTER COLUMN from_account_id DROP NOT NULL;
ALTER TABLE transfers ALTER COLUMN to_account_id DROP NOT NULL;
ALTER TABLE transfers
  ADD COLUMN type           VARCHAR(20) NOT NULL,
  ADD COLUMN status        VARCHAR(30) NOT NULL,
  ADD COLUMN fee_amount    BIGINT NOT NULL DEFAULT 0,
  ADD COLUMN currency      CHAR(3) NOT NULL,
  ADD COLUMN counterparty_bank_code VARCHAR(20),
  ADD COLUMN counterparty_account_number VARCHAR(50),
  ADD COLUMN counterparty_account_name VARCHAR(100),
  ADD COLUMN description   VARCHAR(255),
  ADD COLUMN reversal_of_transfer_id UUID REFERENCES transfers(id),
  ADD COLUMN failure_code  VARCHAR(50),
  ADD COLUMN failure_reason VARCHAR(255),
  ADD COLUMN retry_count   INT NOT NULL DEFAULT 0,
  ADD COLUMN version       BIGINT NOT NULL DEFAULT 0,
  ADD COLUMN updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  ADD COLUMN completed_at  TIMESTAMPTZ;
ALTER TABLE transfers
  ADD CONSTRAINT ck_transfers_type CHECK (type IN ('INTERNAL', 'OUTBOUND', 'INBOUND', 'ADJUSTMENT')),
  ADD CONSTRAINT ck_transfers_status CHECK (status IN (
    'CREATED', 'VALIDATED', 'FUNDS_RESERVED', 'SENT_TO_BANK', 'COMPLETED',
    'FAILED', 'REFUNDING', 'REFUNDED', 'UNKNOWN', 'PENDING_REVIEW')),
  ADD CONSTRAINT ck_transfers_fee CHECK (fee_amount >= 0),
  ADD CONSTRAINT ck_transfers_internal CHECK (
    type <> 'INTERNAL' OR (from_account_id IS NOT NULL AND to_account_id IS NOT NULL AND from_account_id <>
to_account_id)),
  ADD CONSTRAINT ck_transfers_outbound CHECK (
    type <> 'OUTBOUND' OR (from_account_id IS NOT NULL AND counterparty_bank_code IS NOT NULL AND
counterparty_account_number IS NOT NULL)),
  ADD CONSTRAINT ck_transfers_inbound CHECK (
    type <> 'INBOUND' OR (to_account_id IS NOT NULL AND counterparty_bank_code IS NOT NULL));
CREATE INDEX ix_transfers_stuck ON transfers (status, updated_at)
  WHERE status NOT IN ('COMPLETED', 'FAILED', 'REFUNDED');
CREATE INDEX ix_transfers_from ON transfers (from_account_id, created_at DESC);
CREATE INDEX ix_transfers_to ON transfers (to_account_id, created_at DESC);

CREATE TABLE transfer_status_history (
  id BIGSERIAL PRIMARY KEY,
```

```

transfer_id  UUID NOT NULL REFERENCES transfers(id),
from_status  VARCHAR(30),
to_status    VARCHAR(30) NOT NULL,
actor        VARCHAR(30) NOT NULL,
reason       VARCHAR(255),
created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_status_history_transfer ON transfer_status_history (transfer_id, id);

```

Bảng accounts (cột mới hoặc đổi): Mỗi dòng là một "ngăn chứa tiền". Tài khoản hệ thống (phí, clearing, ngân hàng) nằm cùng bảng để mọi dòng tiền, kể cả phí và tiền đang bay, đều là bút toán giữa hai tài khoản và sổ luôn cân.

Cột	Là gì	Vì sao tồn tại
account_number	Số tài khoản hiển thị/nhập tay, duy nhất	Con người và hệ thống ngoài không dùng UUID; ngân hàng ngoài tìm tài khoản theo số này
type	USER / SYSTEM_FEE / SYSTEM_CLEARING / SYSTEM_BANK	Phân biệt ví thật với tài khoản kỹ thuật. Chỉ USER bị cấm âm; tài khoản hệ thống được phép có số dư bất kỳ
bank_code	Mã ngân hàng đối tác, chỉ có với SYSTEM_BANK	Mỗi ngân hàng đối tác cần đúng một tài khoản đại diện (UNIQUE có điều kiện); nhờ đó tiền ra/vào ngân hàng nào cũng truy được
status	ACTIVE / FROZEN / CLOSED	Đóng băng tài khoản nghi vấn mà vẫn giữ lịch sử
owner_ref	Đổi ở bậc này: cho phép NULL. Mã người dùng bên hệ thống thương mại (chuỗi), NULL với tài khoản hệ thống	Ví không có bảng người dùng riêng (tránh sa đà), chỉ cần biết "ví này của ai" bên ngoài. CHECK bắt buộc có với ví người dùng
balance	Ý nghĩa đã đổi từ bậc 1.5. Số dư hiện tại (cache)	Không phải nguồn sự thật (ledger mới là). Tồn tại để đọc số dư nhanh và để CHECK balance >= 0 chặn âm ngay ở DB. Luôn phải khớp tổng ledger (bất biến 4)
version	Số phiên bản tăng mỗi lần sửa	Khóa lạc quan: hai người cùng sửa một dòng thì người sau bị phát hiện thay vì ghi đè lặng lẽ
updated_at	Thời điểm sửa gần nhất (TIMESTAMPTZ , UTC)	Tìm giao dịch/tài khoản kẹt (updated_at cũ), điều tra sự cố

Bảng transfers (cột mới hoặc đổi): Khác với ledger (sự thật đã xảy ra, bất biến), transfers là "yêu cầu và tiến trình" nên có trạng thái và được cập nhật. Một transfer có thể sinh nhiều bút toán (giữ tiền, quyết toán, hoàn tiền).

Cột	Là gì	Vì sao tồn tại
from_account_id	Đổi ở bậc này: cho phép NULL. Tài khoản tiền đi ra	NULL với INBOUND (tiền đến từ ngân hàng ngoài, không có ví nguồn)
to_account_id	Đổi ở bậc này: cho phép NULL. Tài khoản tiền đi vào	NULL với OUTBOUND (tiền đi ra ngoài, không có ví đích bên mình)
type	INTERNAL / OUTBOUND / INBOUND / ADJUSTMENT	Mỗi loại có luồng, bút toán và ràng buộc khác nhau (xem các CHECK). ADJUSTMENT là cách duy nhất để sửa sai: tạo giao dịch mới, không sửa cũ
status	Trạng thái trong state machine	Biết giao dịch đang ở bước nào; recovery worker quét theo cột này; trạng thái cuối không đi tiếp
fee_amount	Phí	Tách khỏi amount để biết người nhận được bao nhiêu và nền tảng thu bao nhiêu; phục vụ bút toán FEE
currency	Loại tiền	Phải khớp tài khoản; tránh cộng nhầm loại tiền
counterparty_bank_code	Mã ngân hàng bên kia	Chỉ có với OUTBOUND / INBOUND : biết gửi đến/nhận từ ngân hàng nào

Cột	Là gì	Vì sao tồn tại
counterparty_account_number	Số tài khoản bên kia	Cần để gửi lệnh sang bank và để đối soát
counterparty_account_name	Tên chủ tài khoản bên kia	Hiện thị cho người dùng và đối chiếu tên khi bank trả về
description	Nội dung chuyển khoản	Người dùng thấy trên sao kê; bank cũng yêu cầu
reversal_of_transfer_id	Trò về giao dịch gốc nếu đây là hoàn tiền	Hoàn tiền là giao dịch mới trò về giao dịch cũ, không sửa/xóa cái cũ; vẫn truy vết được đầy đủ
failure_code , failure_reason	Mã lỗi máy đọc được / lý do cho người đọc	Mã để code và client xử lý (ví dụ INSUFFICIENT_FUNDS); lý do để con người hiểu. Giao dịch bị từ chối vẫn được lưu để audit
retry_count	Số lần recovery đã thử xử lý lại	Dừng thử vô hạn: quá ngưỡng thì chuyển PENDING_REVIEW cho người quyết định
completed_at	Thời điểm về trạng thái cuối	Tách riêng khỏi updated_at (cột này đổi liên tục) để tính thời gian xử lý và đối soát theo ngày
version	Số phiên bản tăng mỗi lần sửa	Khóa lạc quan: hai người cùng sửa một dòng thì người sau bị phát hiện thay vì ghi đè lặng lẽ
updated_at	Thời điểm sửa gần nhất	Recovery worker tìm giao dịch kẹt theo cột này; khác completed_at ở chỗ cột này đổi liên tục

Bảng transfer_status_history : Mỗi lần transfers.status đổi thì thêm một dòng. Chỉ thêm, không sửa. Đây là "camera hành trình" của giao dịch.

Cột	Là gì	Vì sao tồn tại
transfer_id	Giao dịch nào	Gom hành trình của một giao dịch bằng một câu SQL
from_status , to_status	Trạng thái trước / sau	Biết đúng bước nhảy. from_status NULL ở dòng đầu tiên (lúc mới tạo)
actor	Ai làm: API , KAFKA_CONSUMER , RECOVERY_WORKER , BANK_CALLBACK , ADMIN	Phân biệt "luồng bình thường đi tiếp" với "recovery phải cứu" hay "người sửa tay": quan trọng khi điều tra
reason	Lý do chuyển	Ví dụ "bank timeout, chuyển UNKNOWN"; giúp đọc hành trình không cần đọc log
created_at	Thời điểm chuyển trạng thái	Dựng lại hành trình theo thời gian

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
ix_transfers_stuck (status, updated_at) (partial: chưa ở trạng thái cuối)	Recovery worker quét giao dịch kẹt (Giai đoạn 7). Chỉ gồm dòng chưa xong nên nhỏ và nhanh
ix_transfers_from , ix_transfers_to (tài khoản, created_at DESC)	Lịch sử giao dịch của một tài khoản, mới nhất trước
ix_status_history_transfer (transfer_id, id)	Lấy hành trình một giao dịch đúng thứ tự
uq_accounts_owner_currency (owner_ref, currency) (chỉ USER)	Mỗi người dùng chỉ có một ví cho mỗi loại tiền
uq_accounts_system_bank (bank_code) (chỉ SYSTEM_BANK)	Mỗi ngân hàng đối tác đúng một tài khoản đại diện
ck_transfers_internal/outbound/inbound	Mỗi loại giao dịch phải có đủ các trường của nó (ví dụ INTERNAL cần cả hai ví và hai ví phải khác nhau)

Index *partial* (có WHERE) nhỏ và nhanh vì phần lớn dòng của các bảng này đã ở trạng thái cuối.

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Transfer và ledger	Tách hai khái niệm	Transfer có vòng đời/trạng thái, ledger thì bất biến	Gộp lại thì không mô tả được saga nhiều bước
Trạng thái	<code>VARCHAR + CHECK</code>	Dễ đổi khi mở rộng	ENUM Postgres (khó migrate)
Hoàn tiền	Transfer/ledger transaction mới trở về bản gốc	Không sửa dữ liệu cũ, vẫn truy vết được	UPDATE/DELETE giao dịch cũ (sai nguyên tắc ledger)

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Có `status` trên transfer nhưng không có bảng lịch sử chuyển trạng thái.
- Không tách được "một transfer, nhiều ledger transaction", nên hoàn tiền phải sửa dữ liệu cũ.

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

- ☐ Tách được `transfer` (có trạng thái) khỏi bút toán ledger; một transfer có thể có nhiều bút toán.
- ☐ Có lịch sử chuyển trạng thái của transfer.
- ☐ Hoàn tiền là bản ghi mới trở về giao dịch gốc.

Câu SQL bất biến chạy được từ bậc này (cần cột `type`, `status` mới có ở bậc này):

```
-- (2) Không tài khoản USER nào âm
SELECT id, balance FROM accounts WHERE type='USER' AND balance < 0;

-- (5) Transfer nội bộ COMPLETED nhưng không có bút toán
SELECT t.id FROM transfers t
LEFT JOIN ledger_transactions l ON l.transfer_id = t.id AND l.type = 'PRINCIPAL'
WHERE t.type = 'INTERNAL' AND t.status = 'COMPLETED' AND l.id IS NULL;
```

Lưu ý: migration có `TRUNCATE accounts CASCADE` vì các cột `NOT NULL` mới không có giá trị mặc định hợp lý cho dòng cũ. Nạp lại dữ liệu mẫu bằng một bút toán từ `SYSTEM_BANK` sang ví (không `UPDATE balance` trực tiếp).

Bậc 1.10: Kiểm chứng

- **Vấn đề:** cảm giác "chắc đúng" không đủ.
- **Nâng cấp:** lớp `InvariantChecker` chạy các bất biến (bậc 1.10 có đủ câu SQL) sau **mỗi** test tích hợp. Bộ test concurrency: 1000 request cùng rút từ một tài khoản; $A \rightarrow B$ và $B \rightarrow A$ song song hàng nghìn lần; chạy lặp nhiều lần để phát hiện flaky.
- **Giải quyết được:** bằng chứng tự động cho tính đúng đắn.
- **Còn hờ:** không có gì về trùng lặp yêu cầu.
- **✓ Bàn giao:** bộ test xanh ổn định. Đối chiếu schema của bạn với khối tài liệu schema bên dưới và **bảng rubric** ở đó (đạt mục 1-9).

Tài liệu schema của bậc 1.10

Bậc này không đổi schema. Đây là lúc **đối chiếu schema của bạn với đáp án**. Dùng câu sau để liệt kê cột của bạn rồi so với các từ điển ở bậc 1.1 đến 1.9:

```
SELECT table_name, column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name <> 'flyway_schema_history'
ORDER BY table_name, ordinal_position;
```

Kết quả đúng là **năm bảng**: `accounts` , `transfers` , `transfer_status_history` , `ledger_transactions` , `ledger_entries` . Khác với đáp án không nhất thiết là sai; hãy hỏi: *thiết kế của mình có chặn được lỗi mà đáp án chặn không?*

Rubric tự chấm đầy đủ và bậc nào giúp đạt từng tiêu chí. Cuối giai đoạn 1 bạn phải đạt các mục 1 đến 9 (riêng mục 7 cần idempotency nên đạt ở giai đoạn 2); các mục 10 đến 15 đạt ở các giai đoạn sau, như cột bên phải chỉ ra.

#	Tiêu chí	Đạt ở bậc
1	Tiền là số nguyên, có <code>currency</code> , không dùng float.	1.1
2	Ledger append-only, được ép ở DB (trigger hoặc quyền), không chỉ quy ước.	1.8
3	Mỗi ledger transaction cân (debit = credit), có cơ chế ép ở DB hoặc có test bắt buộc.	1.8
4	Tách được <code>transfer</code> (có trạng thái) khỏi bút toán ledger; một transfer có thể có nhiều bút toán.	1.9
5	Chặn ghi sổ trùng ở DB (ví dụ <code>UNIQUE (transfer_id, type)</code>).	1.8
6	Số dư không âm được ép bằng <code>CHECK</code> ở DB.	1.8
7	Idempotency có ít nhất hai lớp: key theo client + ràng buộc unique nghiệp vụ.	2.5
8	Có thứ tự bút toán theo tài khoản (<code>account_seq</code>) hoặc cơ chế tương đương để phát hiện ghi song song sai.	1.7
9	Có lịch sử chuyển trạng thái của transfer.	1.9
10	Lưu từng lần gọi bank riêng; idempotency key gửi bank giữ nguyên qua các lần retry.	4.3
11	Callback lưu dạng inbox với <code>UNIQUE (bank_code, event_id)</code> .	4.5
12	Outbox cùng transaction với dữ liệu; consumer có <code>processed_messages</code> .	3.5
13	Có nơi ghi sai lệch đối soát và bút toán điều chỉnh, không sửa dữ liệu cũ.	7.4
14	Có index phù hợp cho recovery, outbox, sao kê tài khoản.	1.9, 3.4, 7.3
15	Hoàn tiền là bản ghi mới trở về giao dịch gốc.	1.9

Toàn bộ câu SQL bất biến (đây chính là nội dung `InvariantChecker` chạy sau mỗi test):

```
-- (1) Tổng debit = tổng credit toàn hệ thống
SELECT
  SUM(CASE WHEN direction='DEBIT' THEN amount ELSE 0 END) AS total_debit,
  SUM(CASE WHEN direction='CREDIT' THEN amount ELSE 0 END) AS total_credit
FROM ledger_entries;

-- (1b) Ledger transaction nào không cân
SELECT ledger_transaction_id,
  SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END) AS diff
FROM ledger_entries GROUP BY ledger_transaction_id HAVING SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END) <> 0;

-- (2) Không tài khoản USER nào âm
SELECT id, balance FROM accounts WHERE type='USER' AND balance < 0;

-- (4) Balance cache lệch ledger
SELECT a.id, a.balance AS cached,
  COALESCE(SUM(CASE WHEN e.direction='CREDIT' THEN e.amount ELSE -e.amount END),0) AS from_ledger
FROM accounts a LEFT JOIN ledger_entries e ON e.account_id = a.id
GROUP BY a.id, a.balance
HAVING a.balance <> COALESCE(SUM(CASE WHEN e.direction='CREDIT' THEN e.amount ELSE -e.amount END),0);

-- (5) Transfer nội bộ COMPLETED nhưng không có bút toán
SELECT t.id FROM transfers t
LEFT JOIN ledger_transactions l ON l.transfer_id = t.id AND l.type = 'PRINCIPAL'
WHERE t.type = 'INTERNAL' AND t.status = 'COMPLETED' AND l.id IS NULL;
```

```
-- (6) Khoảng trống trong account_seq (dấu hiệu ghi sai)
SELECT account_id, account_seq FROM (
  SELECT account_id, account_seq,
    LAG(account_seq) OVER (PARTITION BY account_id ORDER BY account_seq) AS prev
  FROM ledger_entries) x
WHERE prev IS NOT NULL AND account_seq <> prev + 1;

-- (7) Bút toán RỖNG: có ledger_transactions nhưng không có dòng nào
-- (trigger cân bút toán chỉ chạy khi có dòng được thêm nên không bắt được trường hợp này)
SELECT l.id AS empty_ledger_transaction
FROM ledger_transactions l
LEFT JOIN ledger_entries e ON e.ledger_transaction_id = l.id
WHERE e.id IS NULL;
```

Tổng kết giai đoạn 1

Bạn vừa làm	Đi từ 3 câu <code>UPDATE</code> đến ledger kép, khóa có thứ tự, ràng buộc ở DB, transfer có trạng thái
Bài toán đã giải	Mất tiền khi lỗi giữa chừng (transaction), double-spend (khóa), deadlock (thứ tự khóa), số dư không giải thích được (ledger), mất cân sổ (bút toán kép), ghi sai lạng lẽ (<code>account_seq</code>), sổ bị sửa (ràng buộc DB), giao dịch bị từ chối không dấu vết (FAILED)
Còn hờ	Gửi trùng yêu cầu thì trừ tiền hai lần. Mọi thứ đồng bộ, chưa có event cho hệ thống khác
Hướng đi tiếp	GĐ 2: idempotency

Tự kiểm tra: thử lần lượt bỏ từng bậc (bỏ `ORDER BY` , bỏ `entry_seq` , bỏ trigger cân) rồi chạy test. Với mỗi lần, test nào đơ? Nếu không có test nào đơ, bạn thiếu test.

Giai đoạn 2: Idempotency (gửi trùng không tạo hiệu ứng trùng)

Bài toán: mạng rớt sau khi server đã trừ tiền nhưng response chưa tới client; client gửi lại. Người dùng bấm hai lần. Yêu cầu: nhiều lần gửi cùng một ý định chỉ có **một** hiệu ứng tiền, và người gửi lại nhận **đúng kết quả cũ**.

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **5 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.

```
api_clients  [MỚI] <— transfers >— accounts      (transfers.from_account_id / to_account_id)
               |
               |—> transfer_status_history
               |—> ledger_transactions <— ledger_entries >— accounts (ledger_entries.account_id)
               |
               | (transfers còn tự trở lại chính nó: reversal_of_transfer_id)
```

Bảng đứng riêng, không khóa ngoại tới transfers:
 idempotency_keys [MỚI] (trỏ api_clients; resource_id trỏ transfers nhưng KHÔNG có khóa ngoại)

Ký hiệu: $A \leftarrow B$ nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); $A \rightarrow B$ là khóa ngoại từ A tới B; $\dots$ là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	<code>accounts</code> , <code>transfers</code> , <code>transfer_status_history</code> , <code>ledger_transactions</code> , <code>ledger_entries</code>
Giai đoạn này thêm	<code>api_clients</code> , <code>idempotency_keys</code>
Cột thêm vào bảng cũ	<code>transfers</code> : <code>client_id</code> , <code>idempotency_key</code> , <code>source_type</code> , <code>source_id</code> (migration 2.2 và 2.5)
Migration đã phải chạy	V0 và các migration của bậc 1.5, 1.6, 1.7, 1.8, 1.9 (trong khối tài liệu schema của từng bậc). View <code>ops_1</code> (mục 8.3) là tùy chọn nhưng nên có.
Dữ liệu cần có	Ít nhất 2 tài khoản <code>USER</code> có tiền, có <code>account_number</code> và <code>type</code> . (Dữ liệu ở bậc 1.9 bị <code>TRUNCATE</code> , nên nạp lại.)

Kiểm tra database đã đúng trạng thái chưa (chạy trong `psql`):

```
SELECT string_agg(table_name, ', ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

```
accounts, ledger_entries, ledger_transactions, transfer_status_history, transfers
```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 2.1: Chứng minh lỗi

- **Vấn đề:** gửi cùng một `POST /transfers` hai lần → tiền bị trừ hai lần.
- **Nâng cấp:** viết test đỏ: gửi trùng tuần tự, rồi gửi trùng song song. Liệt kê nguồn gây trùng (response mất, bấm đúp, retry tự động của thư viện HTTP, hai instance cùng nhận).
- **Giải quyết được:** (chưa)
- **Còn hờ:** toàn bộ.
- **✓ Bàn giao:** test đỏ + `docs/idempotency.md` mục kịch bản.

Tài liệu schema của bậc 2.1: không đổi so với bậc trước (schema vẫn như sau bậc 1.9).

Bậc 2.2: Unique key trên transfers (lớp bảo vệ thô)

- **Vấn đề:** cần một cách để nhận ra "đây là cùng một yêu cầu".
- **Nâng cấp:** client gửi `Idempotency-Key`. Thêm cột `transfers.client_id`, `transfers.idempotency_key` và `UNIQUE (client_id, idempotency_key)`. Bảng `api_clients` cho `client_id`. Migration bậc 2.2 ở khối tài liệu schema bên dưới.
- **Giải quyết được:** lần gửi thứ hai không thể tạo transfer thứ hai (DB chặn).
- **Còn hờ:** lần thứ hai nhận về **lỗi vi phạm unique** (500), không phải kết quả cũ. Client không biết yêu cầu trước đã thành công.
- **✓ Bàn giao:** test gửi trùng không còn trừ tiền hai lần.

Tài liệu schema của bậc 2.2

Cần cách nhận ra "đây là cùng một yêu cầu": thêm client gọi vào và hai cột nhận diện trên `transfers`. Tự hỏi: vì sao `UNIQUE (idempotency_key)` một mình là sai?

Migration (`V2_2__idempotency_columns.sql`):

```
-- Bậc 2.2: nhận diện "cùng một yêu cầu". Cột NOT NULL mới nên xóa dữ liệu thư' (xem ghi chú ở bậc 1.9).
TRUNCATE accounts CASCADE;

CREATE TABLE api_clients (
  id          UUID PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  api_key_hash CHAR(64) NOT NULL,
  status       VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_api_clients_status CHECK (status IN ('ACTIVE','DISABLED'))
);

ALTER TABLE transfers
  ADD COLUMN client_id      UUID NOT NULL REFERENCES api_clients(id),
  ADD COLUMN idempotency_key VARCHAR(100) NOT NULL;
CREATE UNIQUE INDEX uq_transfers_idem ON transfers (client_id, idempotency_key);
```

Bảng `api_clients` : Mỗi hệ thống gọi vào ví (ví dụ dự án thương mại) là một client. Bảng này tồn tại để biết **ai** đang gọi, từ đó cô lập dữ liệu và gán idempotency key theo từng client.

Cột	Là gì	Vì sao tồn tại
<code>id</code> kiểu UUID	Mã định danh duy nhất của bản ghi, sinh ở ứng dụng	Sinh ở ứng dụng nên biết được id trước khi ghi DB (cần cho idempotency và outbox); UUID không đoán được nên an toàn để đưa ra API
<code>name</code>	Tên client, duy nhất	Để người đọc log/audit biết client nào; <code>UNIQUE</code> tránh tạo hai client trùng tên
<code>api_key_hash</code>	Băm SHA-256 của API key	Chỉ lưu băm , không lưu key thật: lộ DB cũng không lộ key. Dùng để xác thực request
<code>status</code>	ACTIVE / DISABLED	Khóa khẩn cấp một client mà không phải xóa dữ liệu
<code>created_at</code>	Thời điểm tạo	Điều tra, audit

Bảng `transfers` (cột mới)

Cột	Là gì	Vì sao tồn tại
<code>client_id</code>	Client tạo transfer	Phân quyền và là một phần của khóa idempotency (hai client dùng cùng key vẫn là hai yêu cầu khác nhau)
<code>idempotency_key</code>	Mã client gửi kèm để chống gửi trùng	Lớp bảo vệ idempotency ở mức bản ghi: <code>UNIQUE (client_id, idempotency_key)</code> làm cho transfer thứ hai cùng key không thể tồn tại

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
<code>uq_transfers_idem (client_id, idempotency_key)</code>	Chống tạo trùng transfer khi retry: transfer thứ hai cùng client và key không thể tồn tại . Phải kèm <code>client_id</code> , vì hai client khác nhau có thể tình cờ dùng cùng một key

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Không có `UNIQUE` cho idempotency, hoặc chỉ đặt `UNIQUE (idempotency_key)` mà không kèm `client_id`.

Lưu ý: migration có `TRUNCATE accounts CASCADE` vì `client_id` là `NOT NULL` mới (xem ghi chú ở bậc 1.9). Nạp lại dữ liệu mẫu và tạo một `api_clients` để thử.

Bậc 2.3: Lưu response để trả lại kết quả cũ

- Vấn đề:** client cần nhận **đúng kết quả** đã có, không phải một lỗi.
- Nâng cấp:** thêm bảng `idempotency_keys` (migration ở khối tài liệu schema bên dưới) lưu `request_hash`, `response_code`, `response_body`. Thiết kế hành vi bằng ma trận:

Tình huống với (client, key)	Hành vi	HTTP
Key chưa có	Xử lý bình thường, lưu key	201
Đã có, hoàn tất, cùng request	Trả lại đúng response đã lưu (kể cả lỗi 422)	như lần đầu
Đã có, hoàn tất, khác request	Từ chối: lỗi lập trình phía client	422 IDEMPOTENCY_KEY_REUSED
Đang được xử lý bởi request khác	Chờ hoặc trả 409	409 REQUEST_IN_PROGRESS
Đã hết hạn (quá TTL)	Coi như key mới	201

request_hash = hash của method + path + body chuẩn hóa (sắp xếp khóa JSON). Ghi rõ những trường nào nằm trong hash.

- **Giải quyết được:** gửi lại nhận đúng kết quả cũ; dùng nhầm key cho request khác bị phát hiện.
- **Còn hở:** hai request cùng key đến **đồng thời**?
- **✓ Bàn giao:** RequestHasher + test (hai body khác thứ tự khóa cho cùng hash) + test phản hồi theo ma trận.

Tài liệu schema của bậc 2.3

Chỉ UNIQUE trên transfers mới báo "trùng" chứ không **trả lại kết quả cũ**. Bảng này nhớ phản hồi đã trả.

Migration (V2_3__idempotency_keys.sql):

```
CREATE TABLE idempotency_keys (
  client_id      UUID NOT NULL REFERENCES api_clients(id),
  idempotency_key VARCHAR(100) NOT NULL,
  request_hash   CHAR(64) NOT NULL,           -- hash của method + path + body chuẩn hóa
  status         VARCHAR(20) NOT NULL,        -- IN_PROGRESS | COMPLETED
  resource_type  VARCHAR(30),                 -- 'TRANSFER'
  resource_id    UUID,
  response_code  INT,
  response_body  JSONB,
  locked_at      TIMESTAMPTZ,                 -- đề'nhận lại key kệt IN_PROGRESS
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at     TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (client_id, idempotency_key),
  CONSTRAINT ck_idem_status CHECK (status IN ('IN_PROGRESS','COMPLETED'))
);
CREATE INDEX ix_idem_expires ON idempotency_keys (expires_at);
```

Bảng idempotency_keys : Lớp idempotency thứ nhất (xem 3 lớp ở Giai đoạn 2). Tồn tại vì chỉ UNIQUE trên transfers mới báo "trùng" chứ không **trả lại kết quả cũ** cho client.

Cột	Là gì	Vì sao tồn tại
client_id , idempotency_key	Khóa chính ghép	Key chỉ duy nhất trong phạm vi một client
request_hash	Băm của method + path + body chuẩn hóa	Phát hiện client dùng lại key cho một yêu cầu khác (lỗi lập trình): cùng key khác hash thì từ chối 422 thay vì trả nhầm kết quả
status	IN_PROGRESS / COMPLETED	Phân biệt "đang xử lý" (trả 409 hoặc chờ) với "xong rồi" (trả lại kết quả cũ)
resource_type , resource_id	Tài nguyên đã tạo (TRANSFER + id)	Biết key này dẫn tới transfer nào để trả lại/tra cứu
response_code , response_body	Phản hồi đã trả lần đầu	Lần gửi lại nhận đúng phản hồi đó, kể cả lỗi 422
locked_at	Thời điểm bắt đầu xử lý	Nhận ra key kệt IN_PROGRESS do process chết, để recovery giải phóng
expires_at	Hạn của key	Bảng không phình mãi; hết hạn thì gửi lại được coi là yêu cầu mới (chủ ý)

Cột	Là gì	Vì sao tồn tại
created_at	Thời điểm nhận key lần đầu	Điều tra

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
Khóa chính (client_id, idempotency_key)	Key chỉ duy nhất trong phạm vi một client; INSERT ... ON CONFLICT DO NOTHING dựa vào đây để phát hiện trùng
ix_idem_expires (expires_at)	Job dọn dẹp ở bậc 2.6 xóa key hết hạn mà không phải quét cả bảng

Luồng: INSERT ... ON CONFLICT DO NOTHING . Chèn được thì xử lý tiếp; không chèn được thì đọc bản ghi cũ và so request_hash .

Bậc 2.4: Xử lý đồng thời cùng key

- **Vấn đề:** hai request cùng key chạy song song, cả hai cùng thấy "chưa có key" rồi cùng xử lý.
- **Nâng cấp:** chọn ranh giới transaction:
 - **Phương án A (khuyến nghị): một transaction.** INSERT idempotency_keys ... ON CONFLICT DO NOTHING nằm cùng transaction với nghiệp vụ. Trong PostgreSQL request thứ hai **tự chờ** ở unique index tới khi request thứ nhất commit/rollback (đã đo thử: chờ đúng bằng thời gian request đầu còn giữ transaction), rồi INSERT trả 0 dòng và bạn đọc lại bản ghi để trả kết quả cũ. Trạng thái IN_PROGRESS hầu như không ai thấy.
 - **Phương án B: hai giai đoạn.** Chèn key IN_PROGRESS commit trước, xử lý, rồi COMPLETED . Cần khi việc xử lý vượt một transaction; đổi lại phải xử lý key kẹt IN_PROGRESS khi process chết (dùng locked_at , Recovery ở GD 7 nhận lại).
- **Giải quyết được:** 50 request song song cùng key → đúng 1 lần trừ tiền, 50 phản hồi giống nhau.
- **Còn hờ:** client sinh **key mới** nhưng vẫn là cùng ý định (ví dụ thanh toán hai lần cho một đơn hàng).
- **✓ Bàn giao:** docs/adr/004-idempotency-tx.md + test 50 request song song.

Tài liệu schema của bậc 2.4: không đổi so với bậc trước (schema vẫn như sau bậc 2.3). Cột locked_at (đã có từ bậc 2.3) chỉ dùng nếu bạn chọn phương án B (hai giai đoạn).

Bậc 2.5: Idempotency theo nghiệp vụ

- **Vấn đề:** hai key khác nhau, cùng đơn hàng → vẫn có hai khoản thanh toán.
- **Nâng cấp:** thêm source_type , source_id vào transfers và UNIQUE (client_id, source_type, source_id, type) (chỉ khi source_id không null). Migration bậc 2.5 ở khối tài liệu schema bên dưới.
- **Giải quyết được:** một đơn hàng chỉ có một khoản thanh toán cùng loại, bất kể client gửi bao nhiêu key.
- **Còn hờ:** rác trong bảng key; lỗi 422 chưa được lưu để phát lại.
- **✓ Bàn giao:** test gửi cùng đơn hàng hai key khác nhau → lần hai bị chặn.

Ba lớp bảo vệ bạn vừa dựng, mỗi lớp chặn một loại trùng khác nhau:

Lớp	Cơ chế	Chặn được
1	idempotency_keys (client_id, key)	Cùng một request gửi lại
2	UNIQUE (client_id, idempotency_key) trên transfers	Lọt lớp 1 do lỗi code/đưa
3	UNIQUE (client_id, source_type, source_id, type)	Key mới nhưng cùng ý định nghiệp vụ

Tài liệu schema của bậc 2.5

Hai key khác nhau nhưng cùng một ý định nghiệp vụ (cùng đơn hàng) vẫn phải chỉ có một khoản thanh toán.

Migration (`V2_5__source_idempotency.sql`):

```
-- Bậc 2.5: idempotency theo nghiệp vụ (hai cột cho phép NULL nên không cần xóa dữ liệu)
ALTER TABLE transfers
  ADD COLUMN source_type VARCHAR(30),
  ADD COLUMN source_id VARCHAR(64);
CREATE UNIQUE INDEX uq_transfers_source ON transfers (client_id, source_type, source_id, type)
WHERE source_id IS NOT NULL;
```

Bảng transfers (cột mới)

Cột	Là gì	Vì sao tồn tại
source_type , source_id	Nguồn gốc nghiệp vụ (ví dụ ORDER + mã đơn)	Idempotency nghiệp vụ : UNIQUE (client_id, source_type, source_id, type) đảm bảo một đơn hàng chỉ có một khoản thanh toán dù client sinh bao nhiêu key. Cũng để truy từ giao dịch ngược về đơn hàng

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
uq_transfers_source (client_id, source_type, source_id, type) (chỉ khi source_id không NULL)	Một đơn hàng chỉ có một khoản thanh toán cùng loại, bất kể client sinh bao nhiêu key. Hai cột cho phép NULL nên giao dịch không gắn đơn hàng không bị ảnh hưởng

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Idempotency	Bảng idempotency_keys + unique trên transfers + unique nghiệp vụ theo source	Ba lớp bảo vệ, mỗi lớp chặn một loại trùng	Chỉ Redis với TTL (mất khi Redis mất)

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

- ☐ Idempotency có ít nhất hai lớp: key theo client + ràng buộc unique nghiệp vụ.

Trạng thái schema: sau bậc này transfers đã có đủ cột của schema giai đoạn 1 và 2. Từ đây mỗi bảng mới được thêm riêng ở các bậc sau.

Bậc 2.6: Lưu cả lỗi nghiệp vụ và dọn dẹp

- **Vấn đề:** request bị 422 (thiếu tiền) rồi gửi lại: có nên xử lý lại không? Bảng key phình mãi.
- **Nâng cấp:** lưu key cả khi bị từ chối (response_code=422) để lần sau nhận đúng lỗi. Job @Scheduled xóa key có expires_at < now() (TTL 24-72h). Ghi vào ADR: hết TTL thì gửi lại được coi là yêu cầu mới, đó là chủ ý.
- **Giải quyết được:** phát lại nhất quán cả với lỗi; bảng key không phình.
- **Còn hở:** mọi thứ vẫn đồng bộ trong một request.
- **✓ Bàn giao:** test phát lại lỗi 422 nhận cùng phản hồi; job dọn dẹp chạy được.

Tài liệu schema của bậc 2.6: không đổi so với bậc trước (schema vẫn như sau bậc 2.5). Dừng lại expires_at và ix_idem_expires đã có từ bậc 2.3.

Tổng kết giai đoạn 2

Bạn vừa làm	Từ "gửi trùng thì trừ hai lần" đến 3 lớp idempotency, có ma trận hành vi, có xử lý đồng thời
Bài toán đã giải	Retry/bấm đúp/gửi trùng không tạo hiệu ứng tiền trùng; client nhận nhất quán khi gửi lại; trùng ý định nghiệp vụ bị chặn

Còn hờ	Khi cần nhiều bước (chuyển ra bank) hoặc thông báo cho hệ thống khác, hệ thống chưa chịu được crash giữa các bước
Hướng đi tiếp	GD 3: xử lý nền, outbox, Kafka

Tự kiểm tra: vì sao chỉ có lớp 1 là chưa đủ? Cho một ví dụ thực tế mà lớp 3 cứu được nhưng lớp 1 thì không.

Giai đoạn 3: Xử lý nền, outbox và Kafka

Bài toán: (a) chuyển ra ngân hàng ngoài không thể làm trong một transaction đồng bộ; (b) cần thông báo cho hệ thống khác khi giao dịch xong; (c) không được mất hay xử lý đôi message khi crash. Đây là nơi học **at-least-once + idempotent** thay cho "exactly-once".

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **7 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.

```
api_clients —< transfers >— accounts      (transfers.from_account_id / to_account_id)
                                     |
                                     <— transfer_status_history
                                     <— ledger_transactions >— ledger_entries accounts (ledger_entries.account_id)
                                     (transfers còn tự trở lại chính nó: reversal_of_transfer_id)
```

Bảng đứng riêng, không khóa ngoại tới transfers:

- idempotency_keys (trở api_clients; resource_id trở transfers nhưng KHÔNG có khóa ngoại)
- outbox_events **[MỚI]** (aggregate_id = id của transfer, liên kết lỏng)
- processed_messages **[MỚI]** (message_id = outbox_events.event_id, liên kết lỏng)

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trở về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	accounts, transfers, transfer_status_history, ledger_transactions, ledger_entries, api_clients, idempotency_keys
Giai đoạn này thêm	outbox_events, processed_messages
Migration đã phải chạy	Như trên, cộng migration của bậc 2.2, 2.3 (idempotency_keys) và 2.5.
Dữ liệu cần có	Ít nhất 1 dòng api_clients (ví dụ client commerce) và 2 tài khoản USER có tiền; thêm 1 tài khoản SYSTEM_FEE nếu tính phí.

Kiểm tra database đã đúng trạng thái chưa (chạy trong `psql`):

```
SELECT string_agg(table_name, ' ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

```
accounts, api_clients, idempotency_keys, ledger_entries, ledger_transactions, transfer_status_history,
transfers
```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 3.1: State machine có kiểm soát

- Vấn đề:** khi xử lý nhiều bước và nhiều luồng, trạng thái transfer dễ bị đổi sai thứ tự hoặc bị ghi đè.
- Nâng cấp:** định nghĩa bảng chuyển trạng thái và **một** hàm `transition(id, from, to, actor, reason)` thực thi `UPDATE transfers SET status = :to_status WHERE id = :id AND status = :from_status`.

Từ	Đến	Ai kích hoạt	Điều kiện
(mới)	CREATED	API	Đầu vào hợp lệ
CREATED	VALIDATED	Worker	Tài khoản hợp lệ
VALIDATED	COMPLETED	Worker	Đủ tiền, đã post số
CREATED / VALIDATED	FAILED	Worker	Bị từ chối nghiệp vụ

Quy tắc: 0 dòng bị ảnh hưởng nghĩa là ai đó đã xử lý rồi (an toàn cho việc xử lý trùng); mọi chuyển ghi `transfer_status_history`; trạng thái cuối không đi tiếp.

- **Giải quyết được:** chuyển trạng thái an toàn, có lịch sử, không ghi đè.
- **Còn hờ:** vẫn chạy đồng bộ trong request.
- **✓ Bàn giao:** unit test từ chối chuyển không hợp lệ.

Tài liệu schema của bậc 3.1: không đổi so với bậc trước (schema vẫn như sau bậc 2.5). State machine dựa vào `transfers.status` và `ck_transfers_status` đã có từ bậc 1.9; mọi chuyển trạng thái ghi vào `transfer_status_history`.

Bậc 3.2: Tách xử lý ra nền bằng worker quét DB (chưa cần Kafka)

- **Vấn đề:** request bị giữ lâu; không có chỗ cho các bước dài (gọi bank).
- **Nâng cấp:** `POST /transfers` chỉ tạo transfer `CREATED` rồi trả **202 Accepted** + `transferId`; `GET /transfers/{id}` để client hỏi. Một worker `@Scheduled` quét `CREATED` (bằng `FOR UPDATE SKIP LOCKED`) và xử lý. Phản hồi `202` cũng được lưu vào `idempotency_keys` (gửi lại cùng key nhận lại đúng `202` cùng `transferId`).
- **Giải quyết được:** tách API khỏi xử lý; chịu được xử lý lâu.
- **Còn hờ:** hệ thống khác muốn biết khi nào xong phải hỏi liên tục (polling); độ trễ cao.
- **✓ Bàn giao:** luồng bất đồng bộ chạy được đầu-cuối bằng worker quét DB.

Tài liệu schema của bậc 3.2: không đổi so với bậc trước (schema vẫn như sau bậc 2.5). Worker quét dựa vào `ix_transfers_stuck` (bậc 1.9).

Bậc 3.3: Gửi event lên Kafka một cách ngây thơ

- **Vấn đề:** cần phát sự kiện `transfer.completed` cho hệ thống khác.
- **Nâng cấp (cố tình sai):** sau `COMMIT` gọi `kafkaTemplate.send(...)`. Tái hiện lỗi bằng cách `kill -9 process` giữa commit và send.
- **Giải quyết được:** (tạm) event đến được trong ngày đẹp trời.
- **Còn hờ: dual write:** DB đã commit nhưng event mất (hoặc gửi event trước rồi commit thất bại → event nói dối). Không có transaction chung giữa DB và Kafka.
- **✓ Bàn giao:** test crash chứng minh mất event.

Tài liệu schema của bậc 3.3: không đổi so với bậc trước (schema vẫn như sau bậc 2.5).

Bậc 3.4: Transactional outbox

- **Vấn đề:** dual write.
- **Nâng cấp:** ghi `outbox_events` **trong cùng transaction** với dữ liệu; một relay đọc outbox và gửi Kafka rồi đánh dấu `published_at`.

```

@Scheduled(fixedDelay = 200)
@Transactional
public void publish() {
    List<OutboxEvent> batch = outboxRepo.lockNextBatch(100); // FOR UPDATE SKIP LOCKED, ORDER BY id
    for (OutboxEvent e : batch) {
        kafkaTemplate.send(e.getTopic(), e.getMessageKey(), e.getPayload()).get(5, TimeUnit.SECONDS);
        e.markPublished();
    }
}

```

Phong bì event chung: `eventId (= outbox_events.event_id)`, `eventType`, `occurredAt`, `aggregateId`, `version`, `payload`.

Topic	Key	Producer	Consumer
transfer.requested	transferId	Relay	Worker chuyển tiền
transfer.completed / transfer.failed	transferId	Relay	Hệ thống khác
*.DLT	giữ nguyên	Error handler	Công cụ xem/replay

- **Giải quyết được:** không mất event: nếu DB đã commit thì event chắc chắn sẽ được gửi.
- **Còn hờ:** relay có thể gửi xong rồi chết trước khi `markPublished` → event **gửi lại**. Consumer nhận trùng.
- **✓ Bàn giao:** test kill relay giữa chừng không mất event; `docs/events.md`.

Tài liệu schema của bậc 3.4

Giải bài toán "vừa ghi DB vừa gửi Kafka" (dual write): ghi sự kiện **cùng transaction** với dữ liệu, một relay gửi sau.

Migration (`V3_4__outbox.sql`):

```

CREATE TABLE outbox_events (
  id          BIGSERIAL PRIMARY KEY,
  event_id    UUID NOT NULL UNIQUE,          -- id ổn định của message, consumer dùng để 'chống trùng'
  aggregate_type VARCHAR(30) NOT NULL,        -- 'TRANSFER'
  aggregate_id UUID NOT NULL,
  event_type  VARCHAR(50) NOT NULL,
  topic       VARCHAR(100) NOT NULL,
  message_key VARCHAR(100) NOT NULL,          -- thường là transferId để 'giữ thứ tự trong partition'
  payload     JSONB NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ,
  attempts   INT NOT NULL DEFAULT 0
);
CREATE INDEX ix_outbox_unpublished ON outbox_events (id) WHERE published_at IS NULL;

```

Bảng outbox_events : Giải bài toán "vừa ghi DB vừa gửi Kafka" (dual write): ghi sự kiện **cùng transaction** với dữ liệu, một relay gửi sau.

Cột	Là gì	Vì sao tồn tại
id kiểu BIGSERIAL	Số tự tăng do DB cấp	Dùng cho bảng chỉ ghi thêm khối lượng lớn: nhỏ, nhanh, và có thứ tự , dùng để duyệt "đến đâu rồi" (outbox, sao kê)
event_id	Mã duy nhất ổn định của sự kiện (UUID)	Gửi kèm message; consumer dùng nó để chống xử lý trùng (<code>processed_messages</code>). Ổn định qua mọi lần relay gửi lại
aggregate_type , aggregate_id	Đối tượng sinh sự kiện (<code>TRANSFER</code> + id)	Tra cứu "sự kiện nào của giao dịch này" khi điều tra
event_type	Loại sự kiện (<code>transfer.completed</code> ...)	Consumer biết cách xử lý
topic	Topic Kafka đích	Relay biết gửi đi đâu

Cột	Là gì	Vì sao tồn tại
message_key	Khóa message Kafka (thường là transferId)	Cùng key vào cùng partition, giữ thứ tự các sự kiện của một giao dịch
payload	Nội dung sự kiện (JSONB)	Dữ liệu gửi đi
published_at	Thời điểm relay gửi xong (NULL = chưa gửi)	Chia sự kiện "đã gửi / chưa gửi"; index một phần theo cột này giúp relay lấy nhanh; sự kiện chưa gửi lâu = outbox ùn
attempts	Số lần relay đã thử gửi	Phát hiện sự kiện gửi mãi không được
created_at	Thời điểm ghi sự kiện	Đo độ trễ từ lúc ghi đến lúc gửi; sự kiện chưa gửi quá lâu là dấu hiệu outbox ùn

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
event_id UNIQUE	Mỗi sự kiện đúng một id, ổn định qua mọi lần relay gửi lại
ix_outbox_unpublished (id) (partial: published_at IS NULL)	Relay lấy nhanh các sự kiện chưa gửi theo thứ tự id

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Không có index cho truy vấn nóng (recovery quét kẹt, outbox chưa gửi), dẫn đến full scan khi dữ liệu lớn.

Bậc 3.5: Consumer idempotent

- Vấn đề:** at-least-once nghĩa là message sẽ bị giao trùng; consumer xử lý hai lần thì trừ tiền hai lần.
- Nâng cấp:** bảng processed_messages (consumer_group, message_id). Luồng consumer:

```
[Consumer transfer.requested]
BEGIN
  INSERT processed_messages(consumer, event_id) -- trùng khóa → bỏ qua, ack luôn
  transition CREATED→VALIDATED
  khóa tài khoản, kiểm tra, post PRINCIPAL (+FEE)
  transition VALIDATED→COMPLETED
  INSERT outbox(transfer.completed)
COMMIT → ack offset
```

Điểm mấu chốt: processed_messages, nghiệp vụ và outbox **cùng một transaction**, nên "đã xử lý" và "kết quả xử lý" không bao giờ lệch nhau.

- Giải quyết được:** xử lý trùng không tạo hiệu ứng trùng.
- Còn hờ:** thứ tự event, và lỗi liên tục (message đọc).
- ✓ **Bàn giao:** test cho consumer nhận trùng cùng một message hai lần chỉ xử lý một lần.

Tài liệu schema của bậc 3.5

Phía nhận: nhớ message nào đã xử lý để xử lý trùng không gây hiệu ứng trùng.

Migration (V3_5__processed_messages.sql):

```
CREATE TABLE processed_messages (
  consumer_group VARCHAR(100) NOT NULL,
  message_id      UUID NOT NULL,          -- = outbox_events.event_id
  processed_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (consumer_group, message_id)
);
```

Bảng processed_messages

Cột	Là gì	Vì sao tồn tại
consumer_group	Consumer nào đã xử lý	Cùng một message có thể được hiều consumer xử lý độc lập; khóa ghép theo từng nhóm
message_id	event_id của message	Khóa chính ghép với trên: thêm trùng nghĩa là đã xử lý, bỏ qua. Ghi cùng transaction với nghiệp vụ nên không bao giờ lệch
processed_at	Lúc xử lý	Dọn dẹp bản ghi cũ; điều tra

Ghi `processed_messages` **trong cùng transaction** với xử lý nghiệp vụ. Trùng khóa nghĩa là message đã xử lý, bỏ qua.

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Quên bảng `processed_messages` cho consumer Kafka.

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

☐ Outbox cùng transaction với dữ liệu; consumer có `processed_messages`.

Bậc 3.6: Thứ tự, cấu hình Kafka và xử lý lỗi

- **Vấn đề:** event của cùng một transfer đến sai thứ tự; một message luôn lỗi chặn cả partition; mất message do cấu hình.
- **Nâng cấp:**

Thiết lập	Giá trị gợi ý	Vì sao
Producer <code>acks</code>	<code>all</code>	Không mất message khi broker chết
Producer <code>enable.idempotence</code>	<code>true</code>	Tránh trùng do retry của producer
Message key	<code>transferId</code>	Cùng transfer vào cùng partition, giữ thứ tự
Số partition (dev)	3-6	Đủ để thấy song song mà không phức tạp
Consumer <code>enable.auto.commit</code>	<code>false</code>	Chỉ ack khi DB đã commit
Concurrency	$\leq$ số partition	Hơn cũng không có tác dụng
Error handler	<code>DefaultErrorHandler</code> + <code>DeadLetterPublishingRecoverer</code> + <code>ExponentialBackOff</code>	Retry có kiểm soát

Phân loại lỗi: **tạm thời** (mất DB) → retry theo backoff; **vĩnh viễn** (dữ liệu sai) → thẳng DLT; **nghiệp vụ** (thiếu tiền) không phải lỗi consumer, xử lý thành `FAILED` rồi ack. Nhiều relay `SKIP LOCKED` chạy song song có thể làm hai event cùng key lệch thứ tự: chạy **một** relay (hoặc chia relay theo hash key) và ghi vào ADR.

Khác RabbitMQ: Kafka là **log**, không xóa message khi đọc; consumer tự quản lý offset; thứ tự chỉ đảm bảo trong một partition; không có "requeue một message", retry phải thiết kế bằng retry topic hoặc publish lại.

- **Giải quyết được:** thứ tự theo transfer; message đọc không chặn hệ thống.
- **Còn hờ:** chưa kiểm chứng bằng crash thật.
- **✓ Bàn giao:** cấu hình Kafka được ghi lý do trong ADR; test message độc chuyển vào DLT.

Tài liệu schema của bậc 3.6: không đổi so với bậc trước (schema vẫn như sau bậc 3.5).

Bậc 3.7: Ma trận "chết ở đâu thì chuyện gì xảy ra"

- **Vấn đề:** cần biết chắc mọi điểm crash đều hội tụ về trạng thái đúng.
- **Nâng cấp:** viết bảng và biến mỗi dòng thành test (chạy ở GD 8):

Process chết ở điểm	Trạng thái DB lúc đó	Ai xử lý tiếp	Kết quả
Trước commit của API	Không có gì	Client retry cùng key	Xử lý bình thường
Sau commit API, trước khi trả 202	transfer CREATED + outbox chờ	Relay; client retry nhận lại 202 cũ	Không mất, không trùng
Relay gửi Kafka xong, chưa đánh dấu	outbox chưa published	Relay gửi lại	Consumer chống trùng bằng processed_messages
Consumer chết trước commit DB	Rollback toàn bộ	Kafka giao lại (offset chưa ack)	Xử lý lại từ đầu, an toàn
Consumer commit DB xong, chưa ack	Đã COMPLETED	Kafka giao lại	processed_messages chặn, chỉ ack

- **Giải quyết được:** bạn biết chính xác hệ thống hội tụ ra sao.
- **Còn hờ:** giao dịch kẹt do lỗi **kéo dài** (không phải crash một lần) chưa có ai quét.
- **✓ Bàn giao:** docs/crash-matrix.md .

Tài liệu schema của bậc 3.7: không đổi so với bậc trước (schema vẫn như sau bậc 3.5).

Tổng kết giai đoạn 3

Bạn vừa làm	Từ xử lý đồng bộ → worker nền → Kafka ngẫu thư (rồi tự làm hỏng) → outbox → consumer idempotent → cấu hình + ma trận crash
Bài toán đã giải	Không mất và không xử lý đôi message khi crash; DB và Kafka nhất quán mà không cần transaction phân tán
Còn hờ	Chưa có hệ thống ngoài để gây lỗi thật; giao dịch kẹt lâu chưa có ai quét; chưa có bảo vệ tài
Hướng đi tiếp	GD 4 (Fake Bank, phần khó nhất) hoặc GD 9 (nổi dự án thương mại), độc lập nhau

Tự kiểm tra: vì sao không thể "vừa ghi DB vừa gửi Kafka" trong hai bước độc lập? Vì sao consumer phải ghi processed_messages chung transaction với nghiệp vụ?

Giai đoạn 4: Fake Bank và chuyển tiền liên ngân hàng (tùy chọn theo tiến độ)

Bài toán: khi tiền ra khỏi hệ thống, bạn phụ thuộc vào một hệ thống bạn không kiểm soát: nó chậm, lỗi, trả lời mất, gọi callback trùng. Bài toán khó nhất: **timeout không có nghĩa là thất bại**; bạn không biết tiền đã đi chưa mà cũng không được đoán.

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **9 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.

```
api_clients —< transfers —> accounts      (transfers.from_account_id / to_account_id)
|
|—< transfer_status_history
|—< ledger_transactions —< ledger_entries —> accounts (ledger_entries.account_id)
|—< bank_requests [MỚI] (mỗi lần gọi bank một dòng)
|—< bank_callbacks [MỚI] (inbox; transfer_id có thể NULL)
| (transfers còn tự trở lại chính nó: reversal_of_transfer_id)
|
Bảng đứng riêng, không khóa ngoại tới transfers:
idempotency_keys (trở api_clients; resource_id trở transfers nhưng KHÔNG có khóa ngoại)
outbox_events (aggregate_id = id của transfer, liên kết lỏng)
processed_messages (message_id = outbox_events.event_id, liên kết lỏng)
```

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Database thứ hai (Fake Bank), **chưa tồn tại** khi bắt đầu giai đoạn 4, bạn dựng ở bậc 4.1:

bank_accounts [MỚI] ... bank_transactions [MỚI] —< bank_callback_outbox [MỚI] chaos_rules
(nối lỏng qua account_number, (callback gửi về`wallet-service, (công tắc gây lỗi,
không có khóa ngoại) có retry, có thể`có`trùng/trẻ] đứng riêng)

Bảng đã có khi bắt đầu	accounts , transfers , transfer_status_history , ledger_transactions , ledger_entries , api_clients , idempotency_keys , outbox_events , processed_messages
Giai đoạn này thêm	bank_requests , bank_callbacks ; và database Fake Bank: bank_accounts , bank_transactions , bank_callback_outbox , chaos_rules
Migration đã phải chạy	Như trên, cộng migration của bậc 3.4 và 3.5 (outbox_events , processed_messages).
Dữ liệu cần có	Như trên, cộng tài khoản hệ thống SYSTEM_CLEARING và một tài khoản SYSTEM_BANK cho mỗi ngân hàng (có bank_code).

Kiểm tra database đã đúng trạng thái chưa (chạy trong psql):

```
SELECT string_agg(table_name, ' , ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

```
accounts, api_clients, idempotency_keys, ledger_entries, ledger_transactions, outbox_events,
processed_messages, transfer_status_history, transfers
```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 4.1: Fake Bank đơn giản và gọi bank đồng bộ trong transaction (bản ngậy thơ)

- **Vấn đề:** chưa có "bên ngoài" để luyện.
 - **Nâng cấp:** dựng Fake Bank (app nhỏ, DB riêng, schema ở khối tài liệu schema của bậc này) với ba API và một callback:
 - POST /bank/transfers { "clientRequestId", "toAccountNumber", "amount", "currency" } → { "bankReference", "status": "SUCCESS|PENDING" }. Cùng clientRequestId thì trả lại kết quả cũ (bank cũng idempotent).
 - GET /bank/transfers/{bankReference} (inquiry).
 - GET /bank/report?date= (báo cáo ngày, để đối soát).
 - Callback về wallet: POST /callbacks/bank/{bankCode} với eventId , eventType (TRANSFER_SUCCEEDED , TRANSFER_FAILED , INCOMING_FUNDS), ký HMAC trong X-Signature ; bank retry đến khi nhận 200 .
- Lúc này bank luôn nhanh và luôn thành công. Viết OUTBOUND bản ngậy thơ: trong **một** transaction, khóa tài khoản, trừ tiền, **gọi HTTP sang bank**, rồi commit.
- **Giải quyết được:** chuyển ra bank chạy được trong ngày đẹp trời.
 - **Còn hờ:** làm Fake Bank chậm 30 giây → transaction giữ khóa và connection 30 giây → các giao dịch khác của tài khoản đó đều chờ, cả hệ thống ngهن. Bank timeout nhưng mình đã trừ tiền.
 - **✓ Bàn giao:** docs/bank-contract.md + test đồ với Fake Bank chậm.

Tài liệu schema của bậc 4.1

Bậc này thêm **một database mới**: DB của Fake Bank (project `fake-bank`, Flyway riêng). Database của wallet-service **không đổi**. Fake Bank cố ý nhỏ: nó chỉ cần đủ để từ chối được ("tài khoản không tồn tại", "không đủ tiền"), nhớ giao dịch đã nhận, gửi callback và gây lỗi theo công tắc.

Migration của DB Fake Bank (`V4_1__fake_bank.sql`):

```
CREATE TABLE bank_accounts (
  id          UUID PRIMARY KEY,
  account_number VARCHAR(50) NOT NULL UNIQUE,
  holder_name  VARCHAR(100) NOT NULL,
  balance      BIGINT NOT NULL DEFAULT 0 CHECK (balance >= 0),
  status       VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE bank_transactions (
  id          UUID PRIMARY KEY,
  bank_reference VARCHAR(50) NOT NULL UNIQUE,      -- mã giao dịch bank cấp
  client_request_id VARCHAR(100) NOT NULL UNIQUE,  -- idempotency key do wallet-service gửi
  type        VARCHAR(30) NOT NULL,                -- RECEIVE_FROM_WALLET | SEND_TO_WALLET
  account_number VARCHAR(50) NOT NULL,
  amount      BIGINT NOT NULL CHECK (amount > 0),
  currency    CHAR(3) NOT NULL,
  status      VARCHAR(20) NOT NULL,                -- PENDING | SUCCESS | FAILED
  failure_code VARCHAR(50),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ
);
CREATE INDEX ix_bank_tx_completed ON bank_transactions (completed_at); -- phục vụ GET /bank/report?date=

-- Callback gửi về wallet-service (có retry, và có thể có tình gửi trùng/trễ)
CREATE TABLE bank_callback_outbox (
  id          BIGSERIAL PRIMARY KEY,
  bank_transaction_id UUID NOT NULL REFERENCES bank_transactions(id),
  event_id    UUID NOT NULL UNIQUE,
  event_type  VARCHAR(50) NOT NULL,
  payload     JSONB NOT NULL,
  attempts    INT NOT NULL DEFAULT 0,
  next_attempt_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  delivered_at TIMESTAMPTZ
);
CREATE INDEX ix_bank_cb_due ON bank_callback_outbox (next_attempt_at) WHERE delivered_at IS NULL;

-- Công tắc gây lỗi, đôi lúc chạy mà không cần deploy lại
CREATE TABLE chaos_rules (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(50) NOT NULL UNIQUE,
  enabled     BOOLEAN NOT NULL DEFAULT FALSE,
  mode        VARCHAR(40) NOT NULL,
  -- DELAY | ERROR_5XX | LOSE_RESPONSE | CALLBACK_FAIL_AFTER_SUCCESS
  -- | CALLBACK_DUPLICATE | CALLBACK_LATE | DOWNTIME | REJECT_ACCOUNT
  probability NUMERIC(4,3) NOT NULL DEFAULT 1.0 CHECK (probability BETWEEN 0 AND 1),
  params      JSONB, -- ví dụ {"delay_ms":[100,30000]}
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Báo cáo giao dịch (`GET /bank/report?date=...`) chỉ là truy vấn trên `bank_transactions`, không cần bảng riêng.

Từ điển cột của Fake Bank:

bank_accounts : tài khoản ở ngân hàng giả, đủ để bank có thể từ chối ("tài khoản không tồn tại", "không đủ tiền").

Cột	Là gì	Vì sao tồn tại
<code>account_number</code> , <code>holder_name</code>	Số tài khoản, tên chủ	Bank tra tài khoản đích; trả tên cho bên gọi đối chiếu

Cột	Là gì	Vì sao tồn tại
balance	Số dư	Cho phép mô phỏng từ chối do thiếu tiền (CHECK >= 0)
status	Trạng thái tài khoản	Mô phỏng tài khoản bị khóa

bank_transactions : sổ giao dịch của bank; nguồn của API inquiry và báo cáo ngày.

Cột	Là gì	Vì sao tồn tại
bank_reference	Mã giao dịch bank cấp, duy nhất	Mã bên mình dùng để inquiry và đối soát
client_request_id	Mã idempotency do wallet gửi, duy nhất	Bank cũng phải chống trùng: cùng mã gửi lại thì trả kết quả cũ, không chuyển lần nữa
type	RECEIVE_FROM_WALLET / SEND_TO_WALLET	Hai chiều tiền
account_number , amount , currency	Tài khoản, số tiền, loại tiền	Nội dung lệnh
status	PENDING / SUCCESS / FAILED	Kết quả mà inquiry/callback/báo cáo sẽ trả
failure_code	Lý do thất bại	Mô phỏng từ chối vĩnh viễn
completed_at	Lúc hoàn tất	Báo cáo theo ngày lọc theo cột này (có index)

bank_callback_outbox : hàng đợi callback bank sẽ gửi về wallet (có retry, và cố ý gửi trùng/trễ).

Cột	Là gì	Vì sao tồn tại
bank_transaction_id	Giao dịch nào	Callback thuộc giao dịch nào
event_id	Mã sự kiện, duy nhất	Chính là event_id mà wallet dùng để chống trùng; bank gửi lại cùng id khi retry
event_type , payload	Loại và nội dung	Dữ liệu callback
attempts , next_attempt_at	Số lần đã thử / lần thử kế tiếp	Mô phỏng retry có backoff của bank; index theo next_attempt_at cho job gửi
delivered_at	Wallet đã trả 200 lúc nào	NULL = chưa giao được

chaos_rules : công tắc gây lỗi, đổi lúc chạy.

Cột	Là gì	Vì sao tồn tại
name	Tên luật, duy nhất	Gọi bật/tắt theo tên
enabled	Đang bật không	Bật/tắt mà không cần deploy lại
mode	Kiểu gây lỗi: DELAY , ERROR_5XX , LOSE_RESPONSE , CALLBACK_FAIL_AFTER_SUCCESS , CALLBACK_DUPLICATE , CALLBACK_LATE , DOWNTIME , REJECT_ACCOUNT	Mỗi kiểu tương ứng một dòng trong ma trận lỗi
probability	Xác suất xảy ra (0 đến 1)	Lỗi thật không xảy ra mọi lần
params	Tham số riêng (JSONB)	Ví dụ khoảng độ trễ {"delay_ms": [100,30000]}

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
bank_transactions.bank_reference UNIQUE , client_request_id UNIQUE	Bank cũng chống trùng: gửi lại cùng client_request_id thì trả kết quả cũ, không chuyển lần nữa
ix_bank_tx_completed (completed_at)	Phục vụ GET /bank/report?date=
ix_bank_cb_due (next_attempt_at) (partial: chưa giao)	Job gửi callback lấy nhanh các callback đến hạn
bank_callback_outbox.event_id UNIQUE	Mỗi sự kiện một id; retry gửi lại cùng id để wallet chống trùng được

Bậc 4.2: Tách transaction, dùng tài khoản clearing (saga)

- **Vấn đề:** không được giữ transaction DB mở trong lúc gọi mạng.
- **Nâng cấp:** chia thành các transaction ngắn, tiền "đang bay" nằm ở SYSTEM_CLEARING :

```

TX1 (consumer transfer.requested):
  validate → khóa → post TO_CLEARING (A → CLEARING, amount+fee)
  transition VALIDATED→FUNDS_RESERVED → outbox(bank.outbound.requested) → COMMIT

(gọi mạng, NGOÀI transaction) consumer bank.outbound.requested:
  ghi bank_requests(SENT) → gọi Fake Bank → ghi kết quả vào bank_requests

TX2a (bank báo SUCCESS): post SETTLE (CLEARING→BANK) + FEE (CLEARING→FEE)
  transition →COMPLETED → outbox(transfer.completed)
TX2b (bank báo FAILED rõ ràng): transition →FAILED→REFUNDING → post REFUND (CLEARING→A)
  → REFUNDED → outbox(transfer.failed)

```

Bảng trạng thái OUTBOUND:

Từ	Đến	Điều kiện
VALIDATED	FUNDS_RESERVED	Đủ tiền, đã post TO_CLEARING
FUNDS_RESERVED	SENT_TO_BANK	Đã gọi bank
SENT_TO_BANK	COMPLETED	Bank xác nhận thành công
SENT_TO_BANK	FAILED → REFUNDING → REFUNDED	Bank từ chối rõ ràng, đã post REFUND

- **Giải quyết được:** không giữ khóa khi gọi mạng; tiền luôn ở một chỗ xác định trong sổ.
- **Còn hờ:** bank chậm/lỗi/timeout chưa được xử lý.
- **✓ Bàn giao:** sơ đồ tuần tự + test Fake Bank chậm không còn làm nghẽn các giao dịch khác.

Tài liệu schema của bậc 4.2: không đổi so với bậc trước (schema vẫn như sau bậc 3.5). Các loại bút toán TO_CLEARING , SETTLE , REFUND đã nằm trong ck_ledger_txn_type từ bậc 1.8; tài khoản SYSTEM_CLEARING là một dòng trong accounts .

Bậc 4.3: Timeout, retry và idempotency key gửi bank

- **Vấn đề:** bank trả 5xx thoát qua hoặc timeout. Retry thế nào cho an toàn?
- **Nâng cấp:** mỗi lần gọi ghi một dòng bank_requests (attempt_no tăng dần). bank_request_id gửi bank là **cố định** (ví dụ transferId) qua mọi lần retry. Retry có exponential backoff + jitter, chỉ với lỗi tạm thời; lỗi vĩnh viễn (tài khoản đích sai) không retry.

```

resilience4j:
  timelimiter:
    instances:
      bank: { timeout-duration: 3s }
  retry:
    instances:
      bank:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        enable-randomized-wait: true # jitter
        retry-exceptions: [java.io.IOException, org.springframework.web.client.HttpServerErrorException]
        ignore-exceptions: [com.example.BankRejectedException] # lỗi vĩnh viễn

```

Timeout đặt cả ở HTTP client (connect/read) lẫn tầng bao ngoài.

Hai lưu ý khi cấu hình Resilience4j:

1. Dùng phiên bản **từ 2.3.0 trở lên**. Bản cũ hơn ném lỗi "The intervalFunction was configured twice" nếu bạn khai báo `wait-duration` ở config mặc định rồi bật `exponential backoff` ở instance ([issue #2378](https://github.com/resilience4j/resilience4j/issues/2378) (<https://github.com/resilience4j/resilience4j/issues/2378>), đã sửa từ 2.3.0).
2. Việc bật **đồng thời** `enable-exponential-backoff` và `enable-randomized-wait` trong một instance chưa được xác minh từ tài liệu. Nếu app báo lỗi lúc khởi động, hoặc bạn không thấy jitter trong log, chuyển sang khai báo bằng Java: `IntervalFunction.ofExponentialRandomBackoff(...)`.

- **Giải quyết được:** lỗi thoát qua tự lành; retry không gây chuyển tiền đôi vì bank nhận ra cùng key.
- **Còn hở:** timeout kéo dài: bạn **không biết** tiền đã đi hay chưa.
- **✓ Bàn giao:** test Fake Bank trả 5xx ngẫu nhiên → cuối cùng **COMPLETED**, bank chỉ ghi một giao dịch.

Tài liệu schema của bậc 4.3

Mỗi lần gọi bank là một dòng riêng: ghi đề lên transfer sẽ mất lịch sử các lần thử, và "gọi bank" là điểm mơ hồ nhất của cả hệ thống.

Migration (`V4_3__bank_requests.sql`):

```

-- Mỗi lần gọi sang bank là một dòng (retry sinh dòng mới)
CREATE TABLE bank_requests (
  id                UUID PRIMARY KEY,
  transfer_id       UUID NOT NULL REFERENCES transfers(id),
  bank_code         VARCHAR(20) NOT NULL,
  operation         VARCHAR(20) NOT NULL,          -- TRANSFER | INQUIRY
  attempt_no        INT NOT NULL,
  bank_request_id   VARCHAR(100) NOT NULL,          -- idempotency key gửi cho bank; GIỮ NGUYÊN qua các lần retry
  bank_reference     VARCHAR(100),                  -- mã giao dịch phía bank (nếu đã nhận được)
  status            VARCHAR(20) NOT NULL,           -- SENT | SUCCEEDED | FAILED | TIMEOUT | UNKNOWN
  http_status       INT,
  error_code        VARCHAR(50),
  request_payload    JSONB,
  response_payload   JSONB,
  sent_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
  responded_at      TIMESTAMPTZ,
  CONSTRAINT ck_bank_requests_op CHECK (operation IN ('TRANSFER','INQUIRY')),
  CONSTRAINT ck_bank_requests_status CHECK (status IN ('SENT','SUCCEEDED','FAILED','TIMEOUT','UNKNOWN')),
  CONSTRAINT uq_bank_requests_attempt UNIQUE (transfer_id, operation, attempt_no)
);
CREATE INDEX ix_bank_requests_transfer ON bank_requests (transfer_id, sent_at);

```

Bảng `bank_requests` : Tồn tại vì "gọi bank" là điểm mơ hồ nhất: retry, timeout, kết quả không rõ. Ghi đề lên transfer sẽ mất lịch sử các lần thử.

Cột	Là gì	Vì sao tồn tại
transfer_id	Gọi bank cho giao dịch nào	Gom các lần thử
bank_code	Ngân hàng nào	Hỗ trợ nhiều ngân hàng
operation	TRANSFER (gửi lệnh) / INQUIRY (hỏi trạng thái)	Hai thao tác khác bản chất, đều cần ghi lại
attempt_no	Lần thử thứ mấy	UNIQUE (transfer_id, operation, attempt_no) : không ghi trùng một lần thử
bank_request_id	Mã idempotency gửi cho bank	Giữ nguyên qua mọi lần retry để bank nhận ra yêu cầu trùng và không chuyển hai lần. Đổi nó giữa các lần thử là bug nghiêm trọng
bank_reference	Mã giao dịch phía bank cấp	Có khi nhận được response/callback; dùng để inquiry và đối soát
status	SENT / SUCCEEDED / FAILED / TIMEOUT / UNKNOWN	Kết quả lần thử này. TIMEOUT / UNKNOWN là thông tin quyết định (không được coi là thất bại)
http_status, error_code	Mã HTTP / mã lỗi bank trả	Phân loại lỗi tạm thời hay vĩnh viễn khi điều tra
request_payload, response_payload	Nội dung gửi / nhận (JSONB)	Bảng chứng khi tranh chấp với bank; nhớ che dữ liệu nhạy cảm
sent_at, responded_at	Lúc gửi / lúc có phản hồi	Đo độ trễ của bank; responded_at NULL nghĩa là chưa có phản hồi

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
uq_bank_requests_attempt (transfer_id, operation, attempt_no)	Không ghi trùng một lần thử
ix_bank_requests_transfer (transfer_id, sent_at)	Xem các lần gọi bank của một giao dịch theo thời gian

Điểm quan trọng: bank_request_id **không đổi** giữa các lần retry của cùng một giao dịch, để bank nhận ra yêu cầu trùng.

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Gọi bank	Bảng bank_requests riêng	Mỗi lần gọi là một sự kiện, phục vụ điều tra/ UNKNOWN	Ghi đè một cột status trên transfer (mất lịch sử)

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Ghi đè kết quả gọi bank lên transfer, không lưu từng lần gọi.

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

☐ Lưu từng lần gọi bank riêng; idempotency key gửi bank giữ nguyên qua các lần retry.

Bậc 4.4: Trạng thái UNKNOWN và inquiry

- Vấn đề:** Fake Bank ở chế độ "xử lý xong nhưng mất response": bên gọi thấy timeout dù tiền đã đi.

- **Nâng cấp:** timeout kéo dài → `transition → UNKNOWN`, **không đụng ledger**. Gọi inquiry `GET /bank/transfers/{ref}` (hoặc gửi lại cùng `bank_request_id`, bank trả kết quả cũ) để chốt: thành công → `COMPLETED`; thất bại → `REFUNDING`. Quy tắc bất khả xâm phạm:
 - Không hoàn tiền khi chưa chắc bank thất bại.
 - Không tạo lệnh chuyển **mới** (key mới) cho cùng giao dịch.
- **Giải quyết được:** phân biệt "thất bại" và "không biết"; không hoàn tiền nhầm, không chuyển đổi.
- **Còn hở:** bank gọi callback trùng/trễ/mâu thuẫn.
- **✓ Bàn giao:** test "mất response" → cuối cùng `COMPLETED`, không hoàn tiền. Giải thích vì sao hoàn tiền ngay là **sai** và hậu quả với ledger.

Tài liệu schema của bậc 4.4: không đổi so với bậc trước (schema vẫn như sau bậc 4.3). Trạng thái `UNKNOWN` đã nằm trong `ck_transfers_status` (bậc 1.9) và `ck_bank_requests_status` (bậc 4.3); lần hỏi inquiry là một dòng `bank_requests` có `operation = INQUIRY`.

Bậc 4.5: Callback và inbox chống trùng

- **Vấn đề:** callback đến trùng (cùng `eventId`), trễ, hoặc mâu thuẫn (báo thành công rồi báo thất bại).
- **Nâng cấp:** bảng `bank_callbacks` làm **inbox**: lưu callback **trước** (chữ ký sai thì lưu với `signature_valid=false` và bỏ qua), trả 200, xử lý sau. `UNIQUE (bank_code, event_id)` chặn trùng. Callback đến muộn khi giao dịch đã ở trạng thái cuối thì không đổi gì. Mâu thuẫn không tự sửa: `PENDING_REVIEW` + tạo `reconciliation_issue`.
- **Giải quyết được:** callback trùng/trễ/sai thứ tự không phá được state machine.
- **Còn hở:** bank sập cả tiếng.
- **✓ Bàn giao:** test callback trùng hai lần chỉ xử lý một lần.

Tài liệu schema của bậc 4.5

Callback được **lưu trước, xử lý sau** (inbox): xử lý lỗi thì dữ liệu vẫn còn để xử lý lại; bank gửi trùng thì bị chặn bởi unique.

Migration (`V4_5__bank_callbacks.sql`):

```
-- Inbox: mọi callback từ bank được lưu TRƯỚC khi xử lý
CREATE TABLE bank_callbacks (
  id                BIGSERIAL PRIMARY KEY,
  bank_code         VARCHAR(20) NOT NULL,
  event_id          VARCHAR(100) NOT NULL,          -- id sự kiện do bank cấp, dùng chống trùng
  event_type        VARCHAR(50) NOT NULL,          -- TRANSFER_SUCCEEDED | TRANSFER_FAILED | INCOMING_FUNDS
  bank_reference     VARCHAR(100),
  transfer_id       UUID REFERENCES transfers(id), -- NULL nếu chưa gán được (ví dụ nạp tiền mới đến)
  payload           JSONB NOT NULL,
  signature_valid    BOOLEAN NOT NULL,
  status            VARCHAR(20) NOT NULL DEFAULT 'RECEIVED', -- RECEIVED | PROCESSED | IGNORED | FAILED
  error             VARCHAR(255),
  received_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  processed_at      TIMESTAMPTZ,
  CONSTRAINT uq_bank_callbacks_event UNIQUE (bank_code, event_id)
);
CREATE INDEX ix_bank_callbacks_pending ON bank_callbacks (id) WHERE status = 'RECEIVED';
```

Bảng `bank_callbacks` : Callback được **lưu trước, xử lý sau**: nếu xử lý lỗi, dữ liệu vẫn còn để xử lý lại; nếu bank gửi trùng, bị chặn bởi unique.

Cột	Là gì	Vì sao tồn tại
<code>id</code> kiểu <code>BIGSERIAL</code>	Số tự tăng do DB cấp	Dùng cho bảng chỉ ghi thêm khối lượng lớn: nhỏ, nhanh, và có thứ tự , dùng để duyệt "đến đâu rồi" (outbox, sao kê)

Cột	Là gì	Vì sao tồn tại
bank_code , event_id	Ngân hàng và mã sự kiện do bank cấp	UNIQUE (bank_code, event_id) chống callback trùng
event_type	TRANSFER_SUCCEEDED / TRANSFER_FAILED / INCOMING_FUNDS	Quyết định cách xử lý
bank_reference	Mã giao dịch phía bank	Ghép callback với transfer
transfer_id	Transfer liên quan, NULL nếu chưa ghép được	Nạp tiền (INCOMING_FUNDS) đến trước khi có transfer nên phải cho phép NULL
payload	Nội dung gốc	Giữ nguyên bằng chứng
signature_valid	Chữ ký HMAC có hợp lệ không	Lưu cả callback sai chữ ký (để điều tra) nhưng không xử lý
status	RECEIVED / PROCESSED / IGNORED / FAILED	Tiến độ xử lý; index một phần theo RECEIVED để worker lấy nhanh
error	Lỗi khi xử lý	Biết vì sao FAILED
received_at , processed_at	Lúc nhận / lúc xử lý xong	Đo độ trễ xử lý; phát hiện callback bị bỏ quên

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
uq_bank_callbacks_event (bank_code, event_id)	Chống callback trùng (inbox)
ix_bank_callbacks_pending (id) (partial: status = RECEIVED)	Worker xử lý lấy nhanh các callback chưa xử lý

Điểm quan trọng: callback được INSERT trước; trùng event_id thì bỏ qua, xử lý sau.

Đánh đổi thiết kế:

Quyết định	Lựa chọn trong đáp án	Lý do	Phương án khác
Callback	Bảng inbox bank_callbacks + unique (bank_code, event_id)	Lưu trước xử lý sau, chống trùng	Xử lý thẳng trong controller (mất khi lỗi giữa chừng)

Lỗi hay gặp khi tự thiết kế schema ở bậc này:

- Không có bảng inbox cho callback nên callback trùng bị xử lý hai lần.

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

☐ Callback lưu dạng inbox với UNIQUE (bank_code, event_id) .

Bậc 4.6: Circuit breaker và bulkhead

- Vấn đề:** bank sập N phút, mọi request đều chờ timeout, chiếm hết thread.
- Nâng cấp:**

```
circuitbreaker:
  instances:
    bank:
      sliding-window-size: 20
      failure-rate-threshold: 50
      wait-duration-in-open-state: 30s
      permitted-number-of-calls-in-half-open-state: 3
  bulkhead:
    instances:
      bank: { max-concurrent-calls: 20 }
```

Breaker mở **không** phải lý do hoàn tiền: giao dịch ở lại FUNDS_RESERVED và được thử lại sau khi bank sống lại. Thứ tự decorator mặc định của Resilience4j trong Spring Boot là Retry (CircuitBreaker (RateLimiter (TimeLimiter (Bulkhead (hàm gọi))))) : Retry bao ngoài cùng, nên mỗi lần retry đều đi qua breaker (breaker mở thì các lần retry bị từ chối ngay).

- **Giải quyết được:** lỗi của bank không kéo sập hệ thống của bạn.
- **Còn hờ:** chưa có luồng nạp tiền; ai thử lại các giao dịch kẹt?
- **✓ Bàn giao:** test bank sập 2 phút rồi bật lại → giao dịch hoàn tất; thread không cạn.

Tài liệu schema của bậc 4.6: không đổi so với bậc trước (schema vẫn như sau bậc 4.5).

Bậc 4.7: Nạp tiền (INBOUND) và ma trận lỗi tổng

- **Vấn đề:** tiền cũng đi vào từ bank; và cần tổng hợp hành vi mong đợi.
- **Nâng cấp:**
 - INBOUND: callback INCOMING_FUNDS → INSERT bank_callbacks (dedupe) → worker xác thực chữ ký, tìm tài khoản theo số → tx: tạo transfer INBOUND , post TOPUP (SYSTEM_BANK → ví), COMPLETED , outbox.
 - Ma trận hành vi (mỗi dòng thành một test ở GD 8):

Chế độ gây lỗi	Wallet thấy gì	Sự thật ở bank	Hành động đúng	Kết cục
Chậm quá timeout	Timeout	Có thể đã xử lý	UNKNOWN → inquiry	Theo kết quả inquiry
Trà 5xx	Lỗi 5xx	Chưa xử lý	Retry cùng bank_request_id	COMPLETED / FAILED
Xử lý xong, mất response	Timeout	Đã xử lý	inquiry/gửi lại cùng key	COMPLETED , không hoàn tiền
Từ chối vĩnh viễn	4xx	Không xử lý	Không retry → FAILED → hoàn tiền	REFUNDED
Callback trùng	2 callback cùng eventId	Bình thường	Bỏ qua nhờ unique	Không đổi
Callback trễ	Đến muộn	Bình thường	Không đổi nếu đã ở trạng thái cuối	Không đổi
Callback mâu thuẫn	Thành công rồi thất bại	Bank lỗi	PENDING_REVIEW + issue	Xử lý thủ công
Bank sập	Toàn timeout	Không truy cập được	Breaker mở, giữ FUNDS_RESERVED	COMPLETED sau đó

- **Giải quyết được:** hai chiều tiền vào/ra; hành vi mọi kịch bản lỗi được ghi rõ.
- **Còn hờ:** vẫn chưa có ai tự quét giao dịch kẹt; chưa có đối soát.
- **✓ Bàn giao:** docs/bank-chaos-matrix.md .

Tài liệu schema của bậc 4.7: không đổi so với bậc trước (schema vẫn như sau bậc 4.5). Nạp tiền dùng transfers.type = INBOUND (to_account_id CÓ, from_account_id NULL, bậc 1.9) và bút toán TOPUP từ SYSTEM_BANK .

Tổng kết giai đoạn 4

Bạn vừa làm	Từ "gọi bank trong transaction" → saga nhiều transaction ngắn → retry an toàn → UNKNOWN + inquiry → inbox callback → breaker → nạp tiền
Bài toán đã giải	Tiền đi qua hệ thống không đáng tin mà không mất, không trùng; phân biệt "thất bại" và "không biết"; lỗi bank không lan sang hệ thống bạn
Còn hờ	Ai quét giao dịch kẹt UNKNOWN / FUNDS_RESERVED ? Ai phát hiện lệch giữa bạn và bank? Chưa có bảo vệ tài

Hướng đi tiếp	GD 5 (bảo vệ), rồi GD 7 (recovery và đối soát) trả lời hai câu hỏi "ai quét" và "ai phát hiện lệch"
----------------------	---

Tự kiểm tra: với dòng "xử lý xong nhưng mất response", vì sao hoàn tiền ngay là sai? Ledger sẽ thế nào nếu bạn làm vậy rồi bank vẫn giữ tiền?

Giai đoạn 5: Bảo vệ hệ thống

Bài toán: một client lỗi hoặc độc hại có thể làm sập hệ thống hoặc rút cạn tiền. (Có thể làm song song với GD 4.)

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **11 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.

```

api_clients —< transfers >— accounts      (transfers.from_account_id / to_account_id)
    |
    |—< transfer_status_history
    |—< ledger_transactions —< ledger_entries >— accounts      (ledger_entries.account_id)
    |—< bank_requests          (mỗi lần gọi bank một dòng)
    |—< bank_callbacks          (inbox; transfer_id có thể NULL)
    |—< account_holds [MỚI] >— accounts
    |   (transfers còn tự trở lại chính nó: reversal_of_transfer_id)
accounts —1 account_limits [MỚI]          (mỗi tài khoản một dòng hạn mức)
accounts —< daily_usage [MỚI]             (mỗi tài khoản, mỗi ngày một dòng)

Bảng đứng riêng, không khóa ngoại tới transfers:
idempotency_keys (trở api_clients; resource_id trở transfers nhưng KHÔNG có khóa ngoại)
outbox_events   (aggregate_id = id của transfer, liên kết lỏng)
processed_messages (message_id = outbox_events.event_id, liên kết lỏng)

```

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trở về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Database Fake Bank (dựng ở giai đoạn 4, giai đoạn này không đổi):

bank_accounts ... bank_transactions —< bank_callback_outbox chaos_rules (nối lỏng qua account_number, (callback gửi về wallet-service, (công tắc gây lỗi, không có khóa ngoại) có retry, có thể có tình trùng/trễ) đứng riêng)
--

Bảng đã có khi bắt đầu	accounts, transfers, transfer_status_history, ledger_transactions, ledger_entries, api_clients, idempotency_keys, outbox_events, processed_messages, bank_requests, bank_callbacks
Giai đoạn này thêm	account_limits, daily_usage, account_holds
Cột thêm vào bảng cũ	accounts: held_balance, cộng ràng buộc ck_accounts_available (bậc 5.2, phần tùy chọn)
Migration đã phải chạy	Như trên, cộng migration của bậc 4.3 và 4.5 (bank_requests, bank_callbacks) và database Fake Bank (bậc 4.1, ERD ở trên). View ops_2 nên có (mục 8.3).
Dữ liệu cần có	Như trên. Các dòng account_limits sẽ được tạo trong bậc 5.2 cho từng tài khoản USER.

Kiểm tra database đã đúng trạng thái chưa (chạy trong `psql`):

```

SELECT string_agg(table_name, ' ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';

```

Kết quả phải đúng là:

```

accounts, api_clients, bank_callbacks, bank_requests, idempotency_keys, ledger_entries, ledger_transactions,
outbox_events, processed_messages, transfer_status_history, transfers

```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 5.1: Rate limit

- **Vấn đề:** vòng lặp lỗi của một client gửi 10.000 request/giây; hệ thống nghẽn cho tất cả.
- **Nâng cấp:** token bucket theo đối tượng, trả 429 + Retry-After :

Đối tượng	API	Giới hạn gợi ý
API client	Mọi API	200 req/s
Tài khoản nguồn	POST /transfers	10 req/phút
IP	Mọi API	50 req/s

Bắt đầu bằng Resilience4j RateLimiter (một instance); Bucket4j + Redis nếu cần nhiều instance.

- **Giải quyết được:** một client không làm nghẽn cả hệ thống.
- **Còn hờ:** một client hợp lệ vẫn có thể rút hết tiền trong vài giây.
- **✓ Bàn giao:** docs/rate-limit.md + test bắn 1000 request/giây → phần lớn bị 429, giao dịch hợp lệ vẫn qua.

Tài liệu schema của bậc 5.1: không đổi so với bậc trước (schema vẫn như sau bậc 4.5).

Bậc 5.2: Hạn mức nghiệp vụ (bản sai rồi bản đúng)

- **Vấn đề:** cần giới hạn "tối đa 50 triệu/ngày". Bản ngây thơ: đọc tổng đã chuyển trong ngày → so sánh → ghi.
Sai: hai giao dịch song song cùng đọc giá trị cũ rồi cùng vượt.
- **Nâng cấp:** bảng account_limits , daily_usage (migration ở khối tài liệu schema bên dưới) và cập nhật nguyên tử trong cùng transaction với việc post số:

```
INSERT INTO daily_usage (account_id, usage_date, total_amount, txn_count)
VALUES (:acc, :day, :amt, 1)
ON CONFLICT (account_id, usage_date) DO UPDATE
SET total_amount = daily_usage.total_amount + EXCLUDED.total_amount,
    txn_count     = daily_usage.txn_count + 1
WHERE daily_usage.total_amount + EXCLUDED.total_amount <= :daily_limit
AND daily_usage.txn_count + 1 <= :daily_count_limit
RETURNING total_amount;    -- không có dòng trả về = vượt hạn mức
```

Nhánh INSERT (lần đầu trong ngày) không bị điều kiện WHERE chặn, nên phải kiểm amount <= per_txn_limit và amount <= daily_limit ở code trước. Ghi vào ADR: giao dịch thất bại/hoàn tiền thì có hoàn lại daily_usage không?

- **Giải quyết được:** hạn mức không bị vượt do race (đã thử: lần đầu 60/100 được chấp nhận, lần hai cộng thành 120 trả 0 dòng).
- **Còn hờ:** ai được phép chuyển từ tài khoản nào?
- **✓ Bàn giao:** test 100 request song song cùng vượt hạn mức ngày → đúng số lượng hợp lệ đi qua.

Tài liệu schema của bậc 5.2

Hạn mức cần hai thứ: cấu hình ("tối đa bao nhiêu") và bộ đếm ("đã dùng bao nhiêu hôm nay"). Bộ đếm phải cập nhật nguyên tử, nên nó là một bảng riêng chứ không tính lại từ transfers .

Migration (V5_2__limits.sql):

```
CREATE TABLE account_limits (
  account_id      UUID PRIMARY KEY REFERENCES accounts(id),
  per_txn_limit   BIGINT NOT NULL,
  daily_limit     BIGINT NOT NULL,
  daily_count_limit INT NOT NULL,
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE daily_usage (
  account_id  UUID NOT NULL REFERENCES accounts(id),
  usage_date  DATE NOT NULL,
  total_amount BIGINT NOT NULL DEFAULT 0,
  txn_count   INT NOT NULL DEFAULT 0,
  PRIMARY KEY (account_id, usage_date)
);
```

Bảng `account_limits` và `daily_usage` : hạn mức nghiệp vụ theo tài khoản và bộ đếm đã dùng trong ngày.

Cột	Là gì	Vì sao tồn tại
<code>account_limits.per_txn_limit</code>	Tối đa mỗi giao dịch	Chặn một giao dịch quá lớn
<code>account_limits.daily_limit</code>	Tối đa tổng tiền mỗi ngày	Giới hạn thiệt hại nếu tài khoản bị chiếm
<code>account_limits.daily_count_limit</code>	Tối đa số giao dịch mỗi ngày	Chặn gửi dồn dập nhiều khoản nhỏ
<code>daily_usage.usage_date</code>	Ngày áp dụng	Khóa ghép với <code>account_id</code> : mỗi tài khoản một dòng mỗi ngày
<code>daily_usage.total_amount, txn_count</code>	Đã dùng bao nhiêu tiền / số giao dịch hôm nay	Cập nhật nguyên tử bằng <code>INSERT ... ON CONFLICT DO UPDATE ... WHERE</code> để hai giao dịch song song không cùng vượt hạn mức

Tùy chọn mở rộng: giữ tiền tạm (authorize/capture). Không bậc nào trong tài liệu này bắt buộc, nhưng schema đáp án có sẵn để bạn học thêm. Tiền bị giữ chưa trừ hẳn nhưng không được tiêu; số dư khả dụng = `balance - held_balance`.

Migration tùy chọn (`V5_2_1_holds_optional.sql`):

```
-- Giữ tiền tạm (authorize/capture), tùy chọn
ALTER TABLE accounts ADD COLUMN held_balance BIGINT NOT NULL DEFAULT 0;
ALTER TABLE accounts ADD CONSTRAINT ck_accounts_available
  CHECK (type <> 'USER' OR balance - held_balance >= 0);

CREATE TABLE account_holds (
  id          UUID PRIMARY KEY,
  account_id  UUID NOT NULL REFERENCES accounts(id),
  transfer_id UUID REFERENCES transfers(id),
  amount      BIGINT NOT NULL CHECK (amount > 0),
  status      VARCHAR(20) NOT NULL,      -- ACTIVE | CAPTURED | RELEASED | EXPIRED
  expires_at  TIMESTAMPTZ NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_holds_active_expiry ON account_holds (expires_at) WHERE status = 'ACTIVE';
```

Bảng `accounts` (cột mới, tùy chọn)

Cột	Là gì	Vì sao tồn tại
<code>held_balance</code>	Số tiền đang bị giữ tạm	Tiền chưa trừ hẳn nhưng không được tiêu (authorize/capture). Số dư khả dụng = <code>balance - held_balance</code>

Bảng `account_holds`

Cột	Là gì	Vì sao tồn tại
account_id	Tài khoản bị giữ tiền	Tính held_balance
transfer_id	Giao dịch gây ra việc giữ	Biết giữ tiền cho việc gì
amount	Số tiền giữ, > 0	Phần số dư khả dụng bị khóa
status	ACTIVE / CAPTURED / RELEASED / EXPIRED	CAPTURED : đã thu thật; RELEASED / EXPIRED : trả lại số dư khả dụng
expires_at	Hạn giữ	Tiền không bị giữ vĩnh viễn nếu bên kia quên thu/hủy; job quét hạn dùng index một phần

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
daily_usage khóa chính (account_id, usage_date)	Mỗi tài khoản một dòng mỗi ngày; ON CONFLICT của câu cập nhật nguyên tử dựa vào khóa này
ck_accounts_available (tùy chọn)	Số dư khả dụng không âm: balance - held_balance >= 0 với ví USER
ix_holds_active_expiry (expires_at) (partial: ACTIVE)	Job quét giữ tiền đã hết hạn

Bậc 5.3: Phân quyền

- **Vấn đề:** client A có thể chuyển tiền từ tài khoản của client B.
- **Nâng cấp:** API key → client_id ; client chỉ thao tác trên tài khoản do chính nó tạo (owner_ref thuộc client đó).
- **Giải quyết được:** cô lập giữa các client.
- **Còn hờ:** tài khoản bị nghi vấn vẫn nhận/chuyển được.
- **✓ Bàn giao:** test client A không đọc/chuyển được tài khoản của client B.

Tài liệu schema của bậc 5.3: không đổi so với bậc trước (schema vẫn như sau bậc 5.2). Phân quyền dùng api_clients (bậc 2.2).

Bậc 5.4: Đóng băng và cảnh báo bất thường

- **Vấn đề:** tài khoản nghi vấn cần chặn ngay.
- **Nâng cấp:** FROZEN chặn cả gửi lẫn nhận. Luật đơn giản: tài khoản đích lập < 24h + số tiền lớn → đánh dấu PENDING_REVIEW thay vì COMPLETED . Che số tài khoản trong log, không trả stack trace.
- **Giải quyết được:** có công tắc khẩn cấp và một lớp phát hiện cơ bản.
- **Còn hờ:** khi có sự cố, chưa có cách nhìn nhanh vào một giao dịch.
- **✓ Bàn giao:** test tài khoản FROZEN bị từ chối cả hai chiều.

Tài liệu schema của bậc 5.4: không đổi so với bậc trước (schema vẫn như sau bậc 5.2). Trạng thái FROZEN đã nằm trong ck_accounts_status và PENDING_REVIEW trong ck_transfers_status (bậc 1.9).

Tổng kết giai đoạn 5

Bạn vừa làm	Rate limit → hạn mức nguyên tử → phân quyền → đóng băng
Bài toán đã giải	Client xấu không làm sập hoặc rút cạn hệ thống; hạn mức không bị vượt do race
Còn hờ	Khi sự cố xảy ra, chưa có cách điều tra nhanh; chưa có ai tự sửa giao dịch kẹt
Hướng đi tiếp	GD 6 (quan sát tối thiểu), rồi GD 7

Tự kiểm tra: vì sao "đọc `daily_usage` , kiểm tra, rồi mới ghi" là sai dù nằm trong transaction? (Hai transaction cùng đọc giá trị cũ.)

Giai đoạn 6: Observability tối thiểu (tạm không đi sâu Prometheus)

Bài toán: khi một giao dịch có vấn đề, bạn phải trả lời "nó đã đi qua bước nào, lúc nào, kẹt ở đâu" trong vài phút. Bạn chưa quen phân tích metrics nên dùng công cụ để hiểu và đủ giá trị.

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **14 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.

api_clients —< transfers >— accounts (transfers.from_account_id / to_account_id)

<

transfer_status_history

<

ledger_transactions —< ledger_entries >— accounts (ledger_entries.account_id)

<

bank_requests (mỗi lần gọi bank một dòng)

<

bank_callbacks (inbox; transfer_id có thể NULL)

<

account_holds >— accounts

(transfers còn tự trở lại chính nó: reversal_of_transfer_id)

accounts —1 account_limits (mỗi tài khoản một dòng hạn mức)

accounts —< daily_usage (mỗi tài khoản, mỗi ngày một dòng)

Bảng đứng riêng, không khóa ngoại tới transfers:

idempotency_keys (trở api_clients; resource_id trở transfers nhưng KHÔNG có khóa ngoại)

outbox_events (aggregate_id = id của transfer, liên kết lỏng)

processed_messages (message_id = outbox_events.event_id, liên kết lỏng)

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trở về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	accounts , transfers , transfer_status_history , ledger_transactions , ledger_entries , api_clients , idempotency_keys , outbox_events , processed_messages , bank_requests , bank_callbacks , account_limits , daily_usage , account_holds
Giai đoạn này thêm	không có bảng nào (chỉ thêm log có transferId , view ops_2)
Migration đã phải chạy	Như trên, cộng migration của bậc 5.2 (hạn mức, giữ tiền).
Dữ liệu cần có	Có vài giao dịch ở nhiều trạng thái (COMPLETED , FAILED , UNKNOWN) để dòng thời gian có gì để xem.

Kiểm tra database đã đúng trạng thái chưa (chạy trong psql):

SELECT string_agg(table_name, ' , ' ORDER BY table_name COLLATE "C")

FROM information_schema.tables

WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';

Kết quả phải đúng là:

account_holds, account_limits, accounts, api_clients, bank_callbacks, bank_requests, daily_usage,

idempotency_keys, ledger_entries, ledger_transactions, outbox_events, processed_messages,

transfer_status_history, transfers

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 6.1: Log có ID xuyên suốt

- Vấn đề:** log của một giao dịch nằm rải rác qua API, relay, consumer, không nối lại được.
- Nâng cấp:** gắn transferId và idempotencyKey vào MDC; log dạng JSON nếu muốn.

```
MDC.put("transferId", transferId.toString());
log.info("transfer.state_changed from={} to={}", from, to);
```

- **Giải quyết được:** tìm mọi dòng log của một giao dịch bằng một từ khóa.
- **Còn hờ:** log mất khi xoay vòng; khó tổng hợp.
- **✓ Bàn giao:** tìm log theo `transferId` ra đủ các bước.

Bậc 6.2: Lịch sử trạng thái làm "observability nghiệp vụ"

- **Vấn đề:** log kỹ thuật không trả lời "giao dịch này ở đâu trong quy trình".
- **Nâng cấp:** dùng `transfer_status_history` (đã có từ bậc 1.9). Một câu SQL cho toàn bộ hành trình:

```
SELECT created_at, from_status, to_status, actor, reason
FROM transfer_status_history WHERE transfer_id = :id ORDER BY id;
```

- **Giải quyết được:** trả lời "chuyện gì đã xảy ra với giao dịch này" mà không cần dashboard.
- **Còn hờ:** chưa có cái nhìn tổng thể toàn hệ thống.
- **✓ Bàn giao:** kể lại hành trình một giao dịch bất kỳ chỉ từ log và history.

Bậc 6.3: Bộ SQL sức khỏe hệ thống

- **Vấn đề:** cần nhìn nhanh có gì bất thường không.
- **Nâng cấp:** lưu `docs/health-queries.sql` :

```
-- Transfer kẹt ở trạng thái trung gian quá 5 phút
SELECT id, status, updated_at FROM transfers
WHERE status NOT IN ('COMPLETED', 'FAILED', 'REFUNDED')
AND updated_at < now() - interval '5 minutes';

-- Số transfer theo trạng thái
SELECT status, count(*) FROM transfers GROUP BY status;

-- Outbox bị ùn
SELECT count(*) FROM outbox_events WHERE published_at IS NULL;
```

Thêm Actuator health (`/actuator/health`) cho DB và Kafka.

- **Giải quyết được:** phát hiện giao dịch kẹt và outbox ùn.
- **Còn hờ:** phát hiện xong vẫn phải sửa tay.
- **✓ Bàn giao:** file SQL + health endpoint.

Bậc 6.4 (tùy chọn khi sẵn sàng): 3 chỉ số Prometheus

Chỉ học đọc 3 thứ, mỗi thứ trả lời một câu hỏi cụ thể:

Chỉ số	Câu hỏi nó trả lời	Dấu hiệu xấu
Số transfer kẹt (gauge từ câu SQL trên)	Có giao dịch nào bị bỏ quên không?	> 0 kéo dài
Tỷ lệ lỗi gọi Fake Bank / trạng thái circuit breaker	Đối tác có đang hỏng không?	breaker <code>OPEN</code> , tỷ lệ lỗi tăng
Consumer lag của Kafka	Xử lý có theo kịp không?	lag tăng liên tục

Micrometer + Resilience4j tự xuất metrics của breaker/retry; bạn chỉ cần bật Actuator + Prometheus endpoint. Tracing (OpenTelemetry) để dành sau cùng.

Tổng kết giai đoạn 6

Bạn vừa làm	Log có ID → lịch sử trạng thái → SQL sức khỏe → health check
Bài toán đã giải	Điều tra được một giao dịch cụ thể; nhận ra giao dịch kẹt và outbox ùn

Còn hờ	Biết là kẹt nhưng vẫn phải sửa tay
Hướng đi tiếp	GD 7: biến các câu SQL "phát hiện" thành worker "tự sửa"

Tự kiểm tra: lấy một giao dịch bất kỳ, bạn kể lại được hành trình của nó chỉ từ log và `transfer_status_history` không?

Giai đoạn 7: Recovery và Reconciliation

Bài toán: dù thiết kế kỹ vẫn có giao dịch kẹt (process chết, bank sập lâu, message mất) và có chỗ lệch giữa bạn và bank. Cần **tự phục hồi** những gì an toàn, và **phát hiện + báo** những gì cần người quyết định.

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **14 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.

```
api_clients —< transfers >— accounts      (transfers.from_account_id / to_account_id)
      |
      |—< transfer_status_history
      |—< ledger_transactions —< ledger_entries >— accounts      (ledger_entries.account_id)
      |—< bank_requests      (mỗi lần gọi bank một dòng)
      |—< bank_callbacks      (inbox; transfer_id có thể NULL)
      |—< reconciliation_issues [MỚI] >— reconciliation_runs [MỚI] (cũng trỏ transfers, hai cột)
      |—< account_holds >— accounts
      |   (transfers còn tự trỏ lại chính nó: reversal_of_transfer_id)
      |
accounts —1 account_limits      (mỗi tài khoản một dòng hạn mức)
accounts —< daily_usage      (mỗi tài khoản, mỗi ngày một dòng)

Bảng đứng riêng, không khóa ngoại tới transfers:
idempotency_keys (trỏ api_clients; resource_id trỏ transfers nhưng KHÔNG có khóa ngoại)
outbox_events (aggregate_id = id của transfer, liên kết lỏng)
processed_messages (message_id = outbox_events.event_id, liên kết lỏng)
bank_statement_lines [MỚI] (bản sao báo cáo bank; our_reference = transferId, liên kết lỏng)
```

Ký hiệu: `A —< B` nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); `A >— B` là khóa ngoại từ A tới B; `...` là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	<code>accounts</code> , <code>transfers</code> , <code>transfer_status_history</code> , <code>ledger_transactions</code> , <code>ledger_entries</code> , <code>api_clients</code> , <code>idempotency_keys</code> , <code>outbox_events</code> , <code>processed_messages</code> , <code>bank_requests</code> , <code>bank_callbacks</code> , <code>account_limits</code> , <code>daily_usage</code> , <code>account_holds</code>
Giai đoạn này thêm	<code>bank_statement_lines</code> , <code>reconciliation_runs</code> , <code>reconciliation_issues</code>
Migration đã phải chạy	Như giai đoạn 6 (giai đoạn 6 không thêm bảng nào).
Dữ liệu cần có	Một số transfer cố ý bị kẹt (<code>FUNDS_RESERVED</code> , <code>SENT_TO_BANK</code> , <code>UNKNOWN</code>) để recovery có việc làm.

Kiểm tra database đã đúng trạng thái chưa (chạy trong `psql`):

```
SELECT string_agg(table_name, ' ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

```
account_holds, account_limits, accounts, api_clients, bank_callbacks, bank_requests, daily_usage,
idempotency_keys, ledger_entries, ledger_transactions, outbox_events, processed_messages,
transfer_status_history, transfers
```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 7.1: Sửa tay và viết runbook

- **Vấn đề:** giao dịch kẹt được phát hiện (GD 6) nhưng mỗi lần sửa tay bằng SQL rất dễ sai, dễ phá bất biến.
- **Nâng cấp:** với mỗi loại kẹt bạn gặp, viết runbook: *triệu chứng* → *cách xác nhận (SQL)* → *cách xử lý* → *cách phòng ngừa*. Đây cũng là bản nháp cho các luật tự động ở bậc sau.
- **Giải quyết được:** cách xử lý có hệ thống, không ứng biến.
- **Còn hờ:** vẫn tồn người, chậm.
- **✓ Bàn giao:** docs/runbook.md có ít nhất: giao dịch kẹt UNKNOWN, outbox ùn, DLT tăng, lịch đối soát.

Tài liệu schema của bậc 7.1: không đổi so với bậc trước (schema vẫn như sau bậc 5.2).

Bậc 7.2: Recovery worker theo bảng luật

- **Vấn đề:** tự động hóa các cách sửa an toàn.
- **Nâng cấp:** bảng quyết định (mỗi dòng là một luật, mỗi luật idempotent):

Trạng thái	Quá hạn	Hành động	Nếu vẫn không được
CREATED	> 1 phút	Phát lại event xử lý	Tăng <code>retry_count</code> ; quá ngưỡng → <code>PENDING_REVIEW</code>
VALIDATED	> 2 phút	Xử lý tiếp (khóa, post số)	như trên
FUNDS_RESERVED	> 2 phút	Gửi lại bank với cùng <code>bank_request_id</code>	như trên
SENT_TO_BANK	> 5 phút	Gọi inquiry	→ <code>UNKNOWN</code>
UNKNOWN	> 15 phút	Inquiry lặp lại theo backoff	Quá ngưỡng → <code>PENDING_REVIEW</code>
FAILED (bank từ chối) chưa có REFUND	> 2 phút	Post REFUND (an toàn nhờ <code>UNIQUE (transfer_id, type)</code>)	như trên
REFUNDING	> 2 phút	Hoàn tất REFUND	như trên
PENDING_REVIEW	(không)	Không tự xử lý , chỉ cảnh báo	Người quyết định
<code>idempotency_keys</code> <code>IN_PROGRESS</code>	<code>locked_at</code> cũ	Nhận lại hoặc giải phóng	

Cài đặt: @Scheduled, chọn ứng viên bằng `SELECT ... FOR UPDATE SKIP LOCKED LIMIT n`; mọi hành động dùng **lại** hàm chuyển trạng thái và hàm ghi sổ của luồng bình thường (không viết đường tắt); tăng `transfers.retry_count`; ghi history với `actor='RECOVERY_WORKER'`.

- **Giải quyết được:** giao dịch kẹt tự về trạng thái cuối.
- **Còn hờ:** những chỗ **lệch** mà không có trạng thái kẹt nào lộ ra (bank ghi khác mình).
- **✓ Bàn giao:** docs/recovery-rules.md + test tạo giao dịch kẹt ở từng trạng thái bằng tay, worker đưa về đúng trạng thái cuối.

Tài liệu schema của bậc 7.2: không đổi so với bậc trước (schema vẫn như sau bậc 5.2). Recovery worker dùng các thứ đã có: `ix_transfers_stuck`, `transfers.retry_count` (bậc 1.9), `idempotency_keys.locked_at` (bậc 2.3).

Bậc 7.3: Reconciliation nội bộ

- **Vấn đề:** balance cache có thể lệch ledger, sổ có thể mất cân do bug, mà không ai biết.
- **Nâng cấp:** job định kỳ: tạo `reconciliation_runs` (`INTERNAL`) → chạy các câu kiểm tra bất biến (bậc 1.10) → mỗi kết quả lệch tạo một `reconciliation_issues` (`LEDGER_UNBALANCED`, `BALANCE_CACHE_MISMATCH` ...) → đóng run với `summary`.
- **Giải quyết được:** phát hiện lệch nội bộ chủ động.
- **Còn hờ:** chưa so với bank.
- **✓ Bàn giao:** test cố tình làm lệch cache → job phát hiện.

Tài liệu schema của bậc 7.3

Hai bảng: một lần chạy đối soát và từng chỗ lệch tìm được. Đối soát **không sửa dữ liệu cũ**; muốn sửa số dư thì tạo một transfer loại ADJUSTMENT rồi gắn vào adjustment_transfer_id.

Migration (V7_3__reconciliation.sql):

```
CREATE TABLE reconciliation_runs (
  id          UUID PRIMARY KEY,
  run_type    VARCHAR(20) NOT NULL,      -- INTERNAL | BANK
  bank_code   VARCHAR(20),
  business_date DATE NOT NULL,
  status      VARCHAR(20) NOT NULL,      -- RUNNING | SUCCEEDED | FAILED
  started_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  finished_at TIMESTAMPTZ,
  summary     JSONB
);

CREATE TABLE reconciliation_issues (
  id          UUID PRIMARY KEY,
  run_id      UUID NOT NULL REFERENCES reconciliation_runs(id),
  issue_type  VARCHAR(40) NOT NULL,
  -- MISSING_AT_BANK | MISSING INTERNALLY | AMOUNT_MISMATCH | STATUS_MISMATCH
  -- | LEDGER_UNBALANCED | BALANCE_CACHE_MISMATCH
  transfer_id UUID REFERENCES transfers(id),
  bank_reference VARCHAR(100),
  expected_amount BIGINT,
  actual_amount   BIGINT,
  details         JSONB,
  status          VARCHAR(20) NOT NULL DEFAULT 'OPEN', -- OPEN | INVESTIGATING | RESOLVED | IGNORED
  resolution_note VARCHAR(500),
  resolved_by     VARCHAR(100),
  resolved_at     TIMESTAMPTZ,
  adjustment_transfer_id UUID REFERENCES transfers(id), -- bút toán điều chỉnh (nếu có)
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_recon_issues_open ON reconciliation_issues (status) WHERE status IN ('OPEN','INVESTIGATING');
```

Bảng reconciliation_runs

Cột	Là gì	Vì sao tồn tại
run_type	INTERNAL (số với số dư) / BANK (mình với bank)	Hai loại đối soát khác nhau
bank_code	Ngân hàng được đối soát	Chỉ có với BANK
business_date	Ngày nghiệp vụ được đối soát	Chạy lại cho một ngày cụ thể
status	RUNNING / SUCCEEDED / FAILED	Biết job có chạy xong không (job chết giữa chừng là sự cố)
started_at , finished_at	Thời gian chạy	Theo dõi job chậm dần
summary	Tóm tắt kết quả (JSONB)	Xem nhanh "chạy ra bao nhiêu lệch"

Bảng reconciliation_issues

Cột	Là gì	Vì sao tồn tại
run_id	Phát hiện ở lần chạy nào	Truy vết nguồn gốc phát hiện
issue_type	MISSING_AT_BANK , MISSING INTERNALLY , AMOUNT_MISMATCH , STATUS_MISMATCH , LEDGER_UNBALANCED , BALANCE_CACHE_MISMATCH	Mỗi loại lệch có cách xử lý khác nhau (xem Giai đoạn 7)
transfer_id , bank_reference	Giao dịch / mã bank liên quan	Biết lệch ở đâu để điều tra

Cột	Là gì	Vì sao tồn tại
expected_amount , actual_amount	Số tiền mong đợi / thực tế	Với lệch số tiền: biết lệch bao nhiêu
details	Chi tiết bổ sung (JSONB)	Chứa ngữ cảnh tùy loại lệch
status	OPEN / INVESTIGATING / RESOLVED / IGNORED	Hàng đợi xử lý của người; index một phần theo trạng thái chưa xong
resolution_note , resolved_by , resolved_at	Ghi chú, người xử lý, thời điểm	Audit: ai đã quyết định gì và vì sao
adjustment_transfer_id	Giao dịch điều chỉnh đã tạo để sửa	Sửa sai bằng bút toán mới (loại ADJUSTMENT), không sửa dữ liệu cũ; cột này nối lệch với bản sửa

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
ix_recon_issues_open (status) (partial: OPEN , INVESTIGATING)	Màn hình và việc xử lý sai lệch chỉ cần các lệch còn mở

Bậc 7.4: Reconciliation với ngân hàng và bút toán điều chỉnh

- **Vấn đề:** "bank báo thành công nhưng mình ghi thất bại" (và ngược lại) không thể thấy từ bên trong.
- **Nâng cấp:** lấy GET /bank/report?date= → lưu bank_statement_lines → ghép với transfers theo our_reference → phân loại lệch:

Loại lệch	Ý nghĩa	Xử lý
MISSING_INTERNALLY	Bank có, mình không có/đang FAILED	Điều tra; nếu bank đúng, đưa về COMPLETED
MISSING_AT_BANK	Mình COMPLETED , bank không có	Điều tra; có thể phải hoàn tiền bằng bút toán bù
AMOUNT_MISMATCH	Lệch số tiền	Bút toán điều chỉnh có lý do
STATUS_MISMATCH	Trạng thái mâu thuẫn	Người quyết định

Mọi sửa chữa là **bút toán ADJUSTMENT mới** (một transfer loại ADJUSTMENT), gán vào adjustment_transfer_id . Không sửa dữ liệu cũ. Thêm công cụ xem và replay DLT (replay cũng phải idempotent).

- **Giải quyết được:** lệch giữa bạn và bank được phát hiện và sửa có truy vết.
- **Còn hờ:** chưa chứng minh mọi thứ chạy khi hệ thống bị phá.
- **✓ Bàn giao:** test sửa tay một dòng trong Fake Bank → job phát hiện đúng loại lệch.

Tài liệu schema của bậc 7.4

Dữ liệu "bank nói gì" để so với "mình ghi gì". Các loại lệch MISSING_AT_BANK , MISSING_INTERNALLY , AMOUNT_MISMATCH , STATUS_MISMATCH (bảng reconciliation_issues của bậc 7.3) phát sinh từ bảng này.

Migration (V7_4__bank_statement.sql):

```
CREATE TABLE bank_statement_lines ( -- báo cáo giao dịch lấy từ bank (GET /bank/report)
  id BIGSERIAL PRIMARY KEY,
  bank_code VARCHAR(20) NOT NULL,
  statement_date DATE NOT NULL,
  bank_reference VARCHAR(100) NOT NULL,
  our_reference VARCHAR(100), -- thường là transferId ta gửi cho bank
  direction VARCHAR(10) NOT NULL, -- OUT | IN
  amount BIGINT NOT NULL,
  status VARCHAR(20) NOT NULL,
  raw JSONB,
  imported_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_statement_line UNIQUE (bank_code, bank_reference)
);
```

Bảng `bank_statement_lines` : Dữ liệu "bank nói gì" để so với "mình ghi gì" khi đối soát.

Cột	Là gì	Vì sao tồn tại
<code>id</code> kiểu BIGSERIAL	Số tự tăng do DB cấp	Dùng cho bảng chỉ ghi thêm khối lượng lớn: nhỏ, nhanh, và có thứ tự , dùng để duyệt "đến đâu rồi" (outbox, sao kê)
<code>bank_code</code> , <code>bank_reference</code>	Ngân hàng và mã giao dịch của bank	UNIQUE : nhập lại cùng báo cáo không tạo dòng trùng
<code>statement_date</code>	Ngày của báo cáo	Đối soát theo từng ngày làm việc
<code>our_reference</code>	Mã phía mình (thường là <code>transferId</code>) mà bank ghi nhận	Chìa khóa ghép dòng bank với transfer của mình
<code>direction</code>	OUT / IN	Tiền đi ra hay vào từ góc nhìn của mình
<code>amount</code> , <code>status</code>	Số tiền và trạng thái theo bank	So với số tiền/trạng thái của mình để phát hiện lệch
<code>raw</code>	Dòng gốc	Bảng chứng, khi cần xem lại
<code>imported_at</code>	Lúc nhập	Biết dữ liệu mới đến đâu

Index và ràng buộc của bậc này:

Tên	Chặn lỗi nào / phục vụ truy vấn nào
<code>uq_statement_line (bank_code, bank_reference)</code>	Ngập lại cùng một báo cáo không tạo dòng trùng

Đối soát **không sửa dữ liệu cũ**. Muốn sửa số dư thì tạo một `transfer` loại `ADJUSTMENT` với ledger transaction mới, rồi gắn vào `reconciliation_issues.adjustment_transfer_id`.

Tự chấm (rubric): đánh dấu những tiêu chí thiết kế của bạn đạt.

☐ Có nơi ghi sai lệch đối soát và bút toán điều chỉnh, không sửa dữ liệu cũ.

Tổng kết giai đoạn 7

Bạn vừa làm	Sửa tay có runbook → worker tự phục hồi theo luật → đối soát nội bộ → đối soát bank + bút toán điều chỉnh
Bài toán đã giải	Giao dịch kẹt tự về trạng thái cuối; lệch giữa bạn và bank được phát hiện thay vì tồn tại âm thầm
Còn hờ	Chưa chứng minh bằng cách phá
Hướng đi tiếp	GĐ 8: phá hệ thống có chủ đích

Tự kiểm tra: vì sao recovery worker không nên tự xử lý `PENDING_REVIEW` ? Vì sao mọi hành động recovery phải dừng lại hàm ghi sổ của luồng bình thường?

Giai đoạn 8: Kiểm thử độ tin cậy (phá hệ thống có chủ đích)

Bài toán: mọi cơ chế ở trên mới là "giả định chạy đúng". Giai đoạn này biến giả định thành bằng chứng.

Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **17 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu [MỚI] là thứ giai đoạn này sẽ thêm.

```
api_clients —< transfers >— accounts      (transfers.from_account_id / to_account_id)
    |
    |—< transfer_status_history
    |—< ledger_transactions —< ledger_entries >— accounts      (ledger_entries.account_id)
    |—< bank_requests      (mỗi lần gọi bank một dòng)
    |—< bank_callbacks      (inbox; transfer_id có thể NULL)
    |—< reconciliation_issues >— reconciliation_runs      (cũng trỏ transfers, hai cột)
    |—< account_holds >— accounts
    |   (transfers còn tự trỏ lại chính nó: reversal_of_transfer_id)
accounts —1 account_limits      (mỗi tài khoản một dòng hạn mức)
accounts —< daily_usage      (mỗi tài khoản, mỗi ngày một dòng)

Bảng đứng riêng, không khóa ngoại tới transfers:
idempotency_keys      (trỏ api_clients; resource_id trỏ transfers nhưng KHÔNG có khóa ngoại)
outbox_events      (aggregate_id = id của transfer, liên kết lỏng)
processed_messages      (message_id = outbox_events.event_id, liên kết lỏng)
bank_statement_lines      (bản sao báo cáo bank; our_reference = transferId, liên kết lỏng)
```

Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Bảng đã có khi bắt đầu	accounts, transfers, transfer_status_history, ledger_transactions, ledger_entries, api_clients, idempotency_keys, outbox_events, processed_messages, bank_requests, bank_callbacks, account_limits, daily_usage, account_holds, bank_statement_lines, reconciliation_runs, reconciliation_issues
Giai đoạn này thêm	không có bảng nào (chỉ thêm dữ liệu thử và kịch bản)
Migration đã phải chạy	Như trên, cộng migration của bậc 7.3 và 7.4 (đối soát). View ops_3 nên có.
Dữ liệu cần có	Dữ liệu mẫu cố định, reset được bằng một lệnh (mục 7.4, bước 1).

Kiểm tra database đã đúng trạng thái chưa (chạy trong psql):

```
SELECT string_agg(table_name, ', ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

```
account_holds, account_limits, accounts, api_clients, bank_callbacks, bank_requests, bank_statement_lines,
daily_usage, idempotency_keys, ledger_entries, ledger_transactions, outbox_events, processed_messages,
reconciliation_issues, reconciliation_runs, transfer_status_history, transfers
```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 8.1: Biến ma trận thành test

- Vấn đề:** các ma trận crash-matrix (GD 3) và bank-chaos-matrix (GD 4) mới chỉ là tài liệu.
- Nâng cấp:** mỗi dòng là một test. Test crash: dừng process tại điểm đã liệt kê → khởi động lại → chờ hội tụ (Awaitility) → InvariantChecker. Test chaos: bật từng chế độ của Fake Bank → chạy nhiều giao dịch → hệ thống ổn định → InvariantChecker + không còn giao dịch kẹt.
- Giải quyết được:** bằng chứng cho từng kịch bản đã liệt kê.

- **Còn hờ:** kịch bản do bạn nghĩ ra, còn kịch bản bạn chưa nghĩ tới.
- **✓ Bàn giao:** mỗi dòng ma trận có một test tương ứng.

Bậc 8.2: Property-based test

- **Vấn đề:** tự nghĩ kịch bản thì có điểm mù.
- **Nâng cấp:** jqwik sinh ngẫu nhiên chuỗi thao tác (chuyển, chuyển trùng key, thất bại, hoàn tiền); cuối cùng kiểm tra các bất biến.
- **Giải quyết được:** tìm lỗi trong tổ hợp bạn không nghĩ ra.
- **Còn hờ:** chỉ phá phần mềm, chưa phá hạ tầng.
- **✓ Bàn giao:** test property chạy hàng nghìn chuỗi ngẫu nhiên.

Bậc 8.3: Fault injection hạ tầng và load test

- **Vấn đề:** lỗi thật đến từ mạng, DB, Kafka; và hiệu năng chưa được đo.
- **Nâng cấp:** Toxiproxy giữa app và Postgres/Kafka/Fake Bank (thêm độ trễ, ngắt kết nối, chặn gói tin); dừng Kafka broker vài phút rồi bật lại. Load test (k6/Gatling):

Kịch bản	Mục đích	Cần ghi lại
Tải đều nhiều tài khoản	Throughput bình thường	TPS, p95 latency, tỷ lệ lỗi
Hot account	Tìm điểm nghẽn do khóa	TPS giảm bao nhiêu, thời gian chờ khóa
Fake Bank chậm	Breaker/bulkhead bảo vệ được không	Giao dịch kẹt, thời gian phục hồi

- **Giải quyết được:** biết giới hạn và điểm nghẽn thật.
- **Còn hờ:** kết quả cần được ghi lại để không quên.
- **✓ Bàn giao:** báo cáo load test.

Bậc 8.4: Postmortem

- **Vấn đề:** lỗi tìm được mà không ghi lại thì sẽ lặp.
- **Nâng cấp:** mỗi lỗi một file: *triệu chứng* → *nguyên nhân gốc* → *cách phát hiện* → *cách sửa* → *test/ràng buộc* nào chặn nó lần sau.
- **Giải quyết được:** tri thức tích lũy; đây là phần giá trị nhất của cả dự án.
- **Còn hờ:** chưa có người dùng thật.
- **✓ Bàn giao:** docs/postmortems/*.md .

Tổng kết giai đoạn 8

Bạn vừa làm	Ma trận → test → property → fault injection → load → postmortem
Bài toán đã giải	Có bằng chứng (không phải niềm tin) rằng bất biến giữ được dưới lỗi; biết điểm nghẽn hiệu năng
Còn hờ	Chưa có client thật; hợp đồng với hệ thống khác mới là lý thuyết
Hướng đi tiếp	GĐ 9 nếu chưa làm

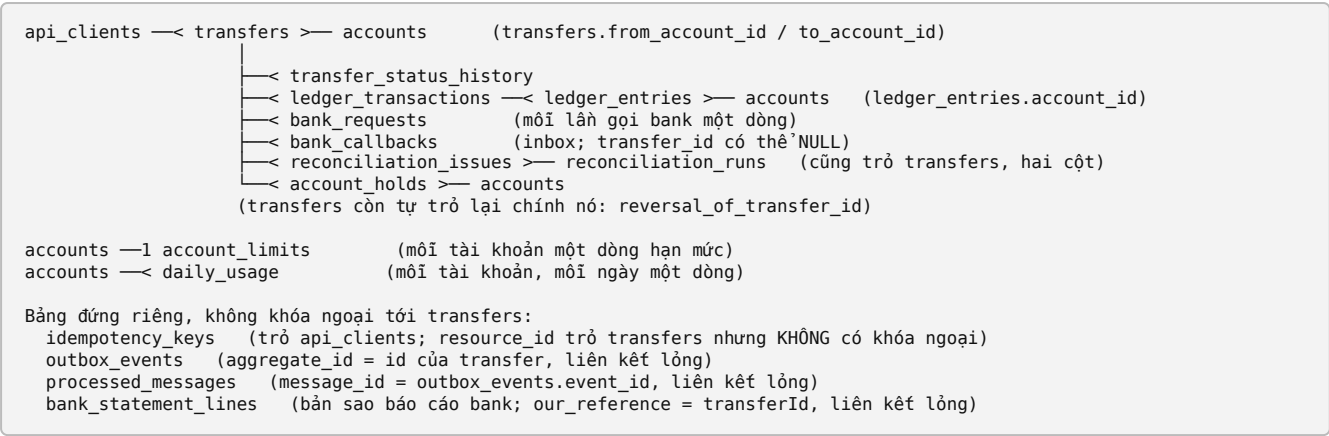
Tự kiểm tra: có bug nào mà **chỉ** test crash mới bắt được, còn test thường bỏ sót không? Nếu chưa tìm ra bug nào, kịch bản của bạn có thể chưa đủ ác.

Giai đoạn 9: Tích hợp với dự án thương mại (client của wallet-service)

Bài toán: bạn ở phía ngược lại: **gọi** một hệ thống tiền tệ qua mạng. Học phía client: timeout, retry an toàn, xử lý "không biết kết quả", nhất quán giữa đơn hàng và giao dịch. (Cần GĐ 3; độc lập với GĐ 4.)

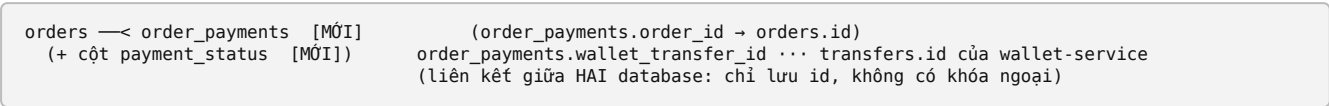
Trạng thái database trước khi bắt đầu giai đoạn này

Khi bắt đầu, database phải đã có **17 bảng** (không đánh dấu trong sơ đồ). Phần đánh dấu **[MỚI]** là thứ giai đoạn này sẽ thêm.



Ký hiệu: A —< B nghĩa là một A có nhiều B (B giữ khóa ngoại trỏ về A); A >— B là khóa ngoại từ A tới B; ... là liên kết lỏng, không có khóa ngoại.

Database của dự án thương mại (khác database wallet):



Bảng đã có khi bắt đầu	accounts, transfers, transfer_status_history, ledger_transactions, ledger_entries, api_clients, idempotency_keys, outbox_events, processed_messages, bank_requests, bank_callbacks, account_limits, daily_usage, account_holds, bank_statement_lines, reconciliation_runs, reconciliation_issues
Giai đoạn này thêm	bên dự án thương mại: bảng order_payments, cột orders.payment_status; bên wallet-service không đổi
Migration đã phải chạy	Như giai đoạn 8 (giai đoạn 8 không thêm bảng nào). Nếu làm giai đoạn 9 sớm hơn (xem mục 5), tối thiểu cần migration đến bậc 2.5 (schema của giai đoạn 1 và 2).
Dữ liệu cần có	Bảng orders của dự án thương mại (đã có sẵn); một dòng api_clients cho dự án thương mại cùng API key.

Kiểm tra database đã đúng trạng thái chưa (chạy trong `psql`):

```
SELECT string_agg(table_name, ' ' ORDER BY table_name COLLATE "C")
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'flyway_schema_history';
```

Kết quả phải đúng là:

```
account_holds, account_limits, accounts, api_clients, bank_callbacks, bank_requests, bank_statement_lines,
daily_usage, idempotency_keys, ledger_entries, ledger_transactions, outbox_events, processed_messages,
reconciliation_issues, reconciliation_runs, transfer_status_history, transfers
```

Thiếu hoặc thừa bảng so với kết quả trên thì quay lại các bậc của giai đoạn trước và chạy phần migration còn thiếu (nằm trong khối "Tài liệu schema" của từng bậc) **trước khi** làm giai đoạn này.

Bậc 9.1: Gọi wallet đồng bộ, không idempotency (bản ngây thơ)

- **Vấn đề:** chưa có gì.
- **Nâng cấp:** checkout trong monolith gọi `POST /transfers`, chờ kết quả, rồi đánh dấu `orders.payment_status = PAID`.

- **Giải quyết được:** chạy được khi mạng ổn.
- **Còn hờ:** làm hỏng bằng ba thí nghiệm: wallet chậm/timeout; khách bấm hai lần; monolith chết sau khi wallet đã trừ tiền.
- **✓ Bàn giao:** ba test đồ.

Tài liệu schema của bậc 9.1: không đổi so với bậc trước (schema vẫn như sau bậc (DB thương mại hiện có)). Chưa dựng database.

Bậc 9.2: Key ổn định và state machine thanh toán

- **Vấn đề:** timeout/bấm hai lần tạo hai giao dịch hoặc đơn ở trạng thái mơ hồ.
- **Nâng cấp:** thêm `order_payments` (migration và từ điển cột ở khối tài liệu schema bên dưới).
`idempotency_key` sinh từ `order_id + attempt_no`: retry cùng một lần thanh toán dùng **lại đúng key cũ**; chỉ khi khách thanh toán lại sau khi đã `FAILED` rõ ràng mới tăng `attempt_no` và sinh key mới. Trạng thái:

Trạng thái <code>order_payments</code>	Ý nghĩa	Sang
PENDING	Đã gửi/đang chờ wallet	SUCCEEDED / FAILED / UNKNOWN
SUCCEEDED	Wallet xác nhận	(cuối) → <code>orders.payment_status=PAID</code>
FAILED	Wallet từ chối rõ ràng	(cuối) → <code>PAYMENT_FAILED</code> , cho phép thử lại với <code>attempt_no</code> mới
UNKNOWN	Gọi wallet timeout	Hỏi lại bằng <code>GET</code> , không tạo thanh toán mới

Gọi wallet kèm `source: {type: ORDER, id: order_id}` để lớp idempotency nghiệp vụ (bậc 2.5) chặn thanh toán trùng.

- **Giải quyết được:** bấm hai lần và retry không tạo hai giao dịch.
- **Còn hờ:** phải nhận kết quả bất đồng bộ.
- **✓ Bàn giao:** `docs/integration-contract.md` (API `POST/GET /transfers`, event `transfer.completed|failed`).

Tài liệu schema của bậc 9.2

Bậc này đổi **database của dự án thương mại** (không phải wallet-service): thêm trạng thái thanh toán vào đơn hàng và bảng ghi từng lần thử thanh toán. Kiểu của `order_id` phải theo bảng `orders` của bạn.

Migration của DB thương mại (`V9_2__order_payments.sql`):

```
ALTER TABLE orders ADD COLUMN payment_status VARCHAR(20) NOT NULL DEFAULT 'UNPAID';
-- UNPAID | PAYMENT_PENDING | PAID | PAYMENT_FAILED | REFUNDED

CREATE TABLE order_payments (
  id                UUID PRIMARY KEY,
  order_id          BIGINT NOT NULL REFERENCES orders(id),    -- kiểu theo bảng orders của bạn
  attempt_no        INT NOT NULL,
  idempotency_key    VARCHAR(100) NOT NULL UNIQUE,             -- ổn định: sinh từ order_id + attempt_no,
  KHÔNG random mỗi lần retry
  wallet_transfer_id UUID,                                     -- id trả về từ wallet-service
  status            VARCHAR(20) NOT NULL,                     -- PENDING | SUCCEEDED | FAILED | UNKNOWN
  amount            BIGINT NOT NULL,
  failure_code       VARCHAR(50),
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (order_id, attempt_no)
);
-- Nếu monolith consume event từ Kafka, thêm bảng processed_messages giống bảng ở bậc 3.5.
```

Từ điển cột phía thương mại:

`orders.payment_status` : trạng thái thanh toán của đơn: `UNPAID` , `PAYMENT_PENDING` , `PAID` , `PAYMENT_FAILED` , `REFUNDED` . Tồn tại để đơn hàng **phản chiếu** kết quả từ ví (ví mới là nguồn sự thật về tiền).

`order_payments` : mỗi lần thử thanh toán một đơn.

Cột	Là gì	Vì sao tồn tại
<code>order_id</code>	Đơn hàng nào	Gắn thanh toán với đơn
<code>attempt_no</code>	Lần thanh toán thứ mấy của đơn	<code>UNIQUE (order_id, attempt_no)</code> ; chỉ tăng khi khách chủ động thanh toán lại sau khi đã thất bại rõ ràng
<code>idempotency_key</code>	Key gửi ví, sinh từ <code>order_id</code> + <code>attempt_no</code> , duy nhất	Ổn định : retry cùng một lần thanh toán dùng lại đúng key này. Key random mỗi lần retry sẽ làm vô hiệu cả cơ chế idempotency
<code>wallet_transfer_id</code>	Mã transfer ví trả về	Hỏi lại trạng thái bằng <code>GET /transfers/{id}</code> ; đối soát đơn với giao dịch. <code>NULL</code> khi chưa nhận được (timeout)
<code>status</code>	<code>PENDING</code> / <code>SUCCEEDED</code> / <code>FAILED</code> / <code>UNKNOWN</code>	<code>UNKNOWN</code> khi gọi ví bị timeout: chưa biết kết quả nên không tạo thanh toán mới
<code>amount</code>	Số tiền thanh toán	Đối chiếu với tổng đơn và với transfer
<code>failure_code</code>	Lý do thất bại	Hiển thị cho khách và quyết định có cho thử lại

Quy tắc `idempotency_key` : **retry cùng một lần thanh toán thì dùng lại đúng key cũ**; chỉ khi người dùng chủ động thanh toán lại sau khi đã thất bại rõ ràng mới tăng `attempt_no` và sinh key mới.

Bậc 9.3: Nhận kết quả bằng event

- **Vấn đề**: poll liên tục tốn và chậm.
- **Nâng cấp**: monolith consume `transfer.completed` / `transfer.failed` từ Kafka bằng consumer idempotent (`processed_messages` phía monolith), cập nhật `order_payments` và `orders.payment_status` .
- **Giải quyết được**: nhận kết quả nhanh, không poll.
- **Còn hở**: event có thể tới trễ/mất; monolith có thể chết trước khi nhận.
- **✓ Bàn giao**: test consumer nhận trùng cùng event chỉ cập nhật một lần.

Tài liệu schema của bậc 9.3: không đổi so với bậc trước (schema vẫn như sau bậc 9.2).

Bậc 9.4: Xử lý `UNKNOWN` và job quét

- **Vấn đề**: gọi wallet timeout: đơn ở trạng thái gì? Wallet trừ tiền nhưng monolith chết trước khi ghi nhận?
- **Nâng cấp**: timeout → `PAYMENT_PENDING` , hỏi lại bằng `GET` , **không** tạo thanh toán mới. Job quét `order_payments` ở `PENDING` / `UNKNOWN` quá lâu và hỏi lại wallet. Job đối soát phía thương mại: đơn `PAID` nhưng không có `wallet_transfer_id` , hoặc ngược lại.
- **Giải quyết được**: không đơn nào bị trừ tiền mà không ghi nhận, hay ghi nhận mà không trừ tiền.
- **Còn hở**: hủy đơn/hoàn tiền.
- **✓ Bàn giao**: test tắt wallet-service giữa checkout rồi bật lại: mỗi đơn hoặc `PAID` đúng một lần, hoặc `PAYMENT_FAILED` rõ ràng.

Tài liệu schema của bậc 9.4: không đổi so với bậc trước (schema vẫn như sau bậc 9.2).

Bậc 9.5: Hoàn tiền khi hủy đơn

- **Vấn đề**: hủy đơn cần trả tiền lại mà không sửa giao dịch cũ.
- **Nâng cấp**: tạo transfer mới (`source_type='ORDER_REFUND'` , `reversal_of_transfer_id` trở về giao dịch gốc). Hủy khi thanh toán còn `PENDING` : chờ kết quả rồi mới hoàn tiền nếu đã thu.
- **Giải quyết được**: hoàn tiền truy vết được, không sửa dữ liệu cũ.

- **Còn hờ:** hướng mở rộng.
- **✓ Bàn giao:** test hủy đơn ở ba trạng thái (chưa thanh toán, đang chờ, đã thanh toán).

Bảng tình huống lỗi phía client:

Tình huống	Đúng	Sai
Gọi wallet bị timeout	<code>PAYMENT_PENDING</code> , hỏi lại bằng <code>GET</code>	Tạo thanh toán mới với key mới
Khách bấm thanh toán 2 lần	Cùng <code>attempt_no</code> → cùng key → wallet trả kết quả cũ	Tạo hai giao dịch
Wallet trừ tiền, monolith chết trước khi ghi nhận	Nhận lại event khi khởi động; job quét <code>PENDING</code> quá lâu	Để đơn ở <code>UNPAID</code> vĩnh viễn
Hủy đơn khi thanh toán còn <code>PENDING</code>	Chờ kết quả rồi mới hoàn tiền nếu đã thu	Hoàn tiền khi chưa chắc đã thu

Tài liệu schema của bậc 9.5: không đổi so với bậc trước (schema vẫn như sau bậc 9.2).

Tổng kết giai đoạn 9

Bạn vừa làm	Gọi đồng bộ ngay thơ → key ổn định + state machine → event → <code>UNKNOWN</code> + job quét → hoàn tiền
Bài toán đã giải	Nhất quán giữa đơn hàng và giao dịch tiền dù có timeout, gửi trùng, crash
Còn hờ	Mở rộng: đa tiền tệ, hold/capture (giữ tiền rồi thu sau), webhook thay polling, sharding cho hot account
Hướng đi tiếp	Chọn một hướng mở rộng ở mục 3 (danh sách vấn đề khó) để đào sâu

Tự kiểm tra: ai là nguồn sự thật về "đơn đã thanh toán"? (Wallet là nguồn sự thật về tiền; monolith chỉ phản chiếu trạng thái.)

3. Danh sách vấn đề khó cần suy nghĩ

Về đúng đắn

1. **Double-spend do race condition:** hai request cùng đọc số dư 100, cùng trừ 80. Chọn khóa bị quan/lạc quan/serializable và chấp nhận đánh đổi hiệu năng.
2. **Deadlock:** $A \rightarrow B$ và $B \rightarrow A$ đồng thời. Cần thứ tự khóa nhất quán.
3. **Kiểm tra hạn mức cũng có race:** "50 triệu/ngày" bị vượt nếu kiểm tra và ghi không nguyên tử.
4. **Làm tròn và phí:** phí tính thế nào, tiền lẻ đi về đâu để tổng vẫn cân?
5. **Chuyển cho chính mình, số tiền 0/âm, tràn số `long`.**

Về lỗi mạng và tính không chắc chắn

6. **Timeout không có nghĩa là thất bại.** Gọi bank bị timeout, tiền có thể đã đi. Không được retry mù quáng cũng không được hoàn tiền vội. Cần trạng thái `UNKNOWN`, inquiry API và đối soát.
7. **Exactly-once là ảo tưởng.** Thực tế là at-least-once + idempotency. Xác định rõ từng ranh giới (client→API, API→DB, DB→Kafka, Kafka→consumer, mình→bank) và cơ chế idempotent ở mỗi ranh giới.
8. **Partial failure:** đã trừ tiền nhưng bước sau chết. Ai phát hiện, ai sửa, sửa bằng cách nào?
9. **Dual write:** ghi DB rồi gửi Kafka. Outbox giải quyết, nhưng hãy chắc bạn hiểu vì sao và hiệu hậu quả (message trùng).
10. **Callback trùng, trễ, sai thứ tự** từ đối tác: xử lý bằng state machine chỉ cho phép chuyển tiến hợp lệ.
11. **Rebalance Kafka giữa chừng:** consumer đang xử lý thì partition bị chuyển sang consumer khác, message có thể được xử lý hai lần.
12. **Poison message:** một message luôn lỗi có thể chặn cả partition nếu retry tại chỗ không giới hạn.

Về dữ liệu và vận hành

13. **Không xóa, không sửa:** sai sót xử lý bằng reversal/adjustment. Thiết kế nghiệp vụ hoàn tiền/chargeback thế nào?
14. **Balance cache lệch ledger:** phát hiện và tự sửa thế nào?
15. **Hot account:** một tài khoản (như tài khoản phí hoặc merchant lớn) bị khóa liên tục làm nghẽn hệ thống. Cân nhắc sub-account/batch/ghi phí bất đồng bộ.
16. **Migration khi đang có giao dịch bay, và đổi state machine** khi vẫn còn transfer ở phiên bản cũ.
17. **Thời gian:** dùng thời gian DB hay app? Múi giờ, cutoff ngày cho hạn mức và đối soát.
18. **Bảo mật:** replay attack, chỉnh sửa `amount` giữa đường (ký request), lộ dữ liệu trong log.
19. **Sắp xếp/phân trang lịch sử giao dịch** với dữ liệu lớn (index, keyset pagination).
20. **Multi-currency** (mở rộng): tỷ giá khóa lúc nào, ai chịu chênh lệch?

4. Kịch bản kiểm thử bắt buộc

Danh sách dưới đây là bản tóm tắt. **Mô tả chi tiết từng kịch bản (gây lỗi bằng gì, phải thấy gì, bắt bug nào), kèm cách chạy không cần tự viết test, nằm ở chương 7.** Cách xem kết quả bằng giao diện nằm ở chương 8.

Concurrency (Testcontainers + nhiều thread)

- ☐ 1000 request song song từ cùng một tài khoản (tổng vượt số dư) → không âm.
- ☐ A→B và B→A song song hàng nghìn lần → không deadlock, tổng tiền không đổi.
- ☐ 50 request cùng Idempotency-Key song song → đúng 1 lần trừ tiền.

Property-based (jqwik)

- ☐ Sinh ngẫu nhiên chuỗi thao tác (chuyển, chuyển trùng key, chuyển thất bại, reversal) → sau cùng cả 4 invariants đúng.

Crash test (dừng process tại từng điểm)

- ☐ Sau khi ghi transfer, trước khi ghi outbox (không thể xảy ra nếu cùng transaction: hãy chứng minh).
- ☐ Sau khi ghi outbox, trước khi gửi Kafka.
- ☐ Sau khi gửi Kafka, trước khi đánh dấu `published`.
- ☐ Sau khi trừ tiền, trước khi gọi bank.
- ☐ Sau khi gọi bank, trước khi nhận response.
- ☐ Sau khi nhận callback, trước khi commit DB.
- ☐ Ở mỗi điểm: khởi động lại → hệ thống hội tụ về trạng thái đúng, invariants đúng.

Fault injection

- ☐ Fake Bank ở từng chế độ lỗi trong ma trận ở Giai đoạn 4, bậc 4.7.
- ☐ Toxiproxy đặt giữa app và Postgres/Kafka/Fake Bank: thêm độ trễ, ngắt kết nối, chặn gói tin.
- ☐ Dừng Kafka broker vài phút rồi bật lại.
- ☐ Postgres chậm/đầy connection pool.

Load test (k6 hoặc Gatling)

- ☐ Đo throughput, latency, tìm hot account, quan sát circuit breaker mở/đóng.

Kiểm chứng cuối mỗi kịch bản: chạy các câu SQL bất biến (Giai đoạn 0, bước 0.2). Nếu chỉ một câu trả về dòng lệch, đó là bug nghiêm trọng.

5. Thứ tự làm và mốc hoàn thành

Bước	Nội dung	Mốc
1	Giai đoạn 0: phạm vi, bất biến, môi trường	Test rỗng chạy được với Postgres thật; các câu SQL bất biến viết xong
2	Giai đoạn 1: bậc 1.1 đến 1.10 (từ CRUD đến ledger), tự thiết kế schema trong khi đi	Schema của bạn đạt các mục 1-9 của rubric (bậc 1.10); 1000 request song song vẫn đúng bất biến
3	Giai đoạn 2: idempotency (tự thiết kế schema idempotency trước)	50 request trùng key chỉ trừ 1 lần
4	Giai đoạn 3: state machine + outbox + Kafka (tự thiết kế outbox và processed_messages trước)	Kill giữa chừng vẫn hội tụ đúng
5a	Giai đoạn 9: nối dự án thương mại (thanh toán đơn hàng)	Tắt wallet-service giữa checkout, không đơn nào trừ tiền sai
5b	Giai đoạn 4: Fake Bank + chuyển ra/nạp vào (độc lập với 5a, làm trước hay sau đều được)	Mọi chế độ lỗi đều về trạng thái xác định
6	Giai đoạn 5: rate limit, hạn mức, bảo mật	429 hoạt động, giao dịch hợp lệ vẫn qua
7	Giai đoạn 6: log/MDC + SQL sức khỏe	Truy được mọi bước của một transfer
8	Giai đoạn 7: recovery + reconciliation (đối soát bank chỉ khi đã làm 5b)	Phát hiện được sai lệch cố tình tạo
9	Giai đoạn 8: chaos + load test	Báo cáo: hỏng gì, vì sao, sửa ra sao
Song song	Chương 8 (giao diện quan sát) và chương 7 (thẻ kịch bản)	Làm theo từng giai đoạn: bảng UI.1 đến UI.5 (mục 8.7) và lịch chạy thẻ (mục 7.8); không để dồn đến cuối

Mẹo học: ở mỗi giai đoạn hãy **phá trước khi coi là xong**. Với mỗi lỗi bạn tự tái hiện và sửa, ghi lại vào docs/ theo mẫu: *triệu chứng* → *nguyên nhân gốc* → *cách sửa* → *cách phòng ngừa*. Đây là phần giá trị nhất của dự án.

6. Checklist tổng

Đúng đắn

- ☐ Schema tự thiết kế đã đối chiếu rubric (bậc 1.10 liệt kê đủ 15 tiêu chí)
- ☐ Ledger double-entry, append-only (ép ở DB), tiền là số nguyên
- ☐ Tách transfer (trạng thái) khỏi ledger transaction (bút toán)
- ☐ Khóa có thứ tự, không deadlock
- ☐ 4 invariants có test và câu SQL kiểm tra
- ☐ Idempotency ở API, consumer, và khi gọi bank

Bất đồng bộ

- ☐ State machine với chuyển trạng thái có điều kiện
- ☐ Transactional outbox
- ☐ Kafka: `acks=all`, idempotent producer, manual ack, consumer idempotent, DLT
- ☐ Phân biệt lỗi tạm thời và lỗi vĩnh viễn

Chống chịu

- ☐ Timeout ở mọi lời gọi ra ngoài
- ☐ Retry có backoff + jitter, chỉ với lỗi tạm thời
- ☐ Circuit breaker + bulkhead
- ☐ Trạng thái `UNKNOWN` + inquiry API
- ☐ Rate limit + hạn mức nghiệp vụ

Phục hồi

- ☐ Recovery worker cho transfer kẹt
- ☐ Reconciliation nội bộ và với Fake Bank
- ☐ Reversal/adjustment thay vì sửa dữ liệu
- ☐ Runbook cho từng loại sự cố

Quan sát tối thiểu

- ☐ Log kèm `transferId` (MDC)
- ☐ `transfer_status_history`
- ☐ Câu SQL sức khỏe hệ thống
- ☐ Actuator health

Kiểm thử

- ☐ Concurrency, property-based, crash test, fault injection, load test
- ☐ Bốn viên gạch dùng chung (công tắc lỗi, chờ hội tụ, bộ đèn, ảnh chụp trước/sau) ở mục 7.3
- ☐ Các thẻ kịch bản ở mục 7.5 chạy xanh theo lịch ở mục 7.8

Giao diện quan sát (chương 8)

- ☐ Tài khoản `ops_reader` chỉ đọc; view `ops_1` đến `ops_3` đã tạo
- ☐ Sửa tay số dư thì đèn đỏ hiện ngay trên màn tổng quan
- ☐ Kể lại hành trình một transfer chỉ từ trang chi tiết, không mở `psql`

7. Logic kiểm thử: mô tả từng kịch bản để chạy được mà không phải tự viết test

Chương này dành cho ai thấy viết test là khó và tốn thời gian. Bạn không cần viết 40 lớp test. Bạn dựng **bốn viên gạch dùng chung** một lần (mục 7.3), sau đó mỗi kịch bản chỉ là một **thẻ mô tả**: gây lỗi gì, phải thấy kết quả gì, bắt được loại bug nào (mục 7.5). Kết quả được đọc trên giao diện ở chương 8 (đèn xanh/đỏ), không phải mò vào database.

Trạng thái kiểm chứng của chương này (đọc trước khi tin):

- **Đã chạy thật trên PostgreSQL 16:** mọi câu SQL trong chương (đèn bất biến, chờ hội tụ, ảnh chụp trước/sau, các lệnh phá dữ liệu D1 đến D6) và bốn kịch bản `pgbench` ở mục 7.6, kèm số liệu đo được.
- **Chưa chạy được vì ứng dụng chưa tồn tại:** các thẻ ở nhóm B (mất mạng), C (sập process), E (ngẫu nhiên), và phần gọi HTTP của nhóm A. Chúng là **đặc tả hành vi đúng**, suy ra từ các cơ chế bạn sẽ xây ở Giai đoạn 2 đến 7. Tên công tắc `Toxiproxy` ở mục 7.3 chưa chạy thử. Đoạn mã Java chưa biên dịch.

7.1 Vì sao test khó, và cách tách nhỏ

Một test độ tin cậy thường gộp **bốn việc** vào một chỗ, nên rối:

1. **Gây lỗi** (cắt mạng, giết process, làm bank trả lời mất).
2. **Chờ** hệ thống tự sửa mình (retry, recovery chạy nền).
3. **Kiểm tra** mọi thứ vẫn đúng.
4. **So sánh** trạng thái trước và sau.

Chương này tách bốn việc thành bốn viên gạch tái sử dụng (mục 7.3). Có ba cách chạy kịch bản, chọn tùy độ lười:

Cách	Cần gì	Khi nào dùng
A. SQL và <code>pgbench</code> thuần	Chỉ cần Postgres	Ngay từ Giai đoạn 1, trước khi có API (mục 7.6)
B. Nút bấm "Scenario Lab"	Giao diện chương 8 + bốn viên gạch	Từ Giai đoạn 3 trở đi; đây là cách chính
C. Dịch thẻ thành JUnit	Bốn viên gạch gọi từ code test	Nếu sau này muốn chạy trong CI; mỗi thẻ thành đúng một test

7.2 Mẫu một thẻ kịch bản

Mọi thẻ ở mục 7.5 có cùng các trường:

Trường	Ý nghĩa
Mã	Nhóm và số, ví dụ <code>A1</code> , <code>B3</code>
Đời thật	Chuyện này xảy ra ở đâu ngoài đời, một câu
Gây ra bằng	Công tắc hoặc lệnh cụ thể (mục 7.3)
Kết quả đúng	Điều phải thấy sau khi hệ thống ổn định; thiếu một ý là đỏ
Bắt lỗi nào	Kiểu bug mà thẻ này tồn tại để phát hiện

Quy tắc chung cho mọi thẻ (không lặp lại trong từng thẻ): sau khi hệ thống hội tụ, **tất cả 8 đèn** ở `v_invariant_checks` phải bằng 0 vi phạm. Thẻ chỉ ghi thêm những gì **riêng** của nó.

7.3 Bốn viên gạch dùng chung

Gạch 1: công tắc gây lỗi. Làm một lần, dùng cho mọi thê.

Loại lỗi	Công tắc	Cách dùng
Ngân hàng lỗi, chậm, mất response, callback trùng/trễ/mâu thuẫn, sập	Bảng <code>chaos_rules</code> của Fake Bank (bậc 4.1)	Bật/tắt một dòng bằng SQL hoặc nút ở chương 8
Process chết đúng một điểm	Biến môi trường <code>CRASH_AT</code>	Khởi động app với <code>CRASH_AT=<điểm></code> , app tự chết khi chạm điểm đó
Mạng chậm chờn, Postgres/Kafka/bank chậm hoặc đứt	Toxiproxy (đặt giữa app và đích)	Gọi HTTP API của Toxiproxy để thêm/xóa "độc"
Hạ tầng sập hẳn	<code>docker compose stop / start</code>	Dừng Kafka, Postgres vài phút rồi bật lại
Nhiều request song song	<code>pgbench</code> (mức DB) hoặc <code>xargs -P</code> (mức HTTP)	Mục 7.6 và ví dụ bên dưới

Công tắc của Fake Bank, đã chạy thử trên schema ở bậc 4.1. Tạo sẵn một lần, mặc định tắt hết:

```
INSERT INTO chaos_rules (name, enabled, mode, probability, params) VALUES
('slow',          false, 'DELAY',          1.0, '{"delay_ms":[6000,8000]}'),
('five-xx',       false, 'ERROR_5XX',      1.0, NULL),
('lose-response', false, 'LOSE_RESPONSE',  1.0, NULL),
('callback-flip', false, 'CALLBACK_FAIL_AFTER_SUCCESS', 1.0, NULL),
('callback-dup',  false, 'CALLBACK_DUPLICATE', 1.0, '{"copies":3}'),
('callback-late', false, 'CALLBACK_LATE',   1.0, '{"delay_s":120}'),
('down',          false, 'DOWNTIME',        1.0, NULL),
('reject',        false, 'REJECT_ACCOUNT',  1.0, '{"account":"0000"}');

-- Bật một lỗi:
UPDATE chaos_rules SET enabled = true WHERE name = 'lose-response';
-- Tắt hết (luôn làm bước này trước khi chờ hội tụ):
UPDATE chaos_rules SET enabled = false;
```

Điểm dừng cho process chết. Lớp nhỏ này gọi từ các chỗ đã chọn trong code nghiệp vụ. `halt` làm process chết **ngay**, không chạy `finally` hay shutdown hook, giống `kill -9`, nên mô phỏng đúng việc mất điện giữa chừng (đoạn mã chưa biên dịch):

```
@Component
public class CrashPoints {
    private final String crashAt = System.getenv("CRASH_AT"); // đọc một lần lúc khởi động; mặc định null = tắt

    public void hit(String point) {
        if (point.equals(crashAt)) {
            Runtime.getRuntime().halt(137);
        }
    }
}

// Dùng: crashPoints.hit("AFTER_DEBIT_BEFORE_BANK_CALL");
```

Chạy lại sau khi chết **không** có `CRASH_AT`, nếu không app chết lại ở đúng điểm đó. Chỉ bật trong môi trường học (profile `lab`), đừng để công tắc này tồn tại ở bản chạy thật.

Các điểm dừng nên đặt (khớp ma trận crash ở bậc 3.7 và luồng ngân hàng ở Giai đoạn 4):

Tên điểm	Đặt ở đâu
<code>API_AFTER_COMMIT</code>	Sau commit của API, trước khi trả 202
<code>RELAY_AFTER_SEND</code>	Relay gửi Kafka xong, chưa đánh dấu <code>published_at</code>
<code>CONSUMER_BEFORE_COMMIT</code>	Consumer xử lý xong, chưa commit DB

Tên điểm	Đặt ở đâu
CONSUMER_AFTER_COMMIT	Consumer commit DB xong, chưa ack Kafka
AFTER_DEBIT_BEFORE_BANK_CALL	Đã giữ tiền (FUNDS_RESERVED), chưa gọi bank
AFTER_BANK_CALL_BEFORE_RESPONSE	Đã gửi cho bank, chưa nhận response
AFTER_CALLBACK_BEFORE_COMMIT	Đã lưu callback, đang xử lý, chưa commit
RECOVERY_MID_RUN	Recovery worker đang xử lý một transfer kẹt

Toxiproxy (chưa chạy thử, tên trường theo tài liệu Toxiproxy): tạo proxy cho từng đích, rồi thêm "độc" qua HTTP API cổng 8474. Ví dụ làm mọi gói tới bank chậm thêm 5 giây rồi gỡ:

```
curl -s -X POST localhost:8474/proxies/bank/toxics \
-d '{"name":"slow","type":"latency","attributes":{"latency":5000}}'
curl -s -X DELETE localhost:8474/proxies/bank/toxics/slow
```

Bắn 50 request cùng một Idempotency-Key song song bằng xargs (chưa chạy thử vì API chưa tồn tại; thay URL, token, body.json theo hợp đồng ở bậc 1.9):

```
seq 1 50 | xargs -P 50 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
-X POST http://localhost:8080/transfers \
-H "Content-Type: application/json" -H "Idempotency-Key: same-key-1" \
-d @body.json | sort | uniq -c
```

Gạch 2: chờ hệ thống hội tụ. Sau khi gây lỗi xong và gỡ lỗi, đừng kiểm tra ngay, hãy chờ cho đến khi ba con số này cùng bằng 0 trong 3 lần hỏi liên tiếp, cách nhau 1 giây. Quá thời hạn (ví dụ 90 giây) mà chưa về 0 thì kịch bản **đỏ vì "không hội tụ"**, và đó là lỗi nghiêm trọng của recovery:

```
SELECT
  (SELECT count(*) FROM v_open_transfers WHERE status <> 'PENDING_REVIEW') AS transfer_dang_do,
  (SELECT count(*) FROM outbox_events WHERE published_at IS NULL) AS outbox_chua_gui,
  (SELECT count(*) FROM bank_callbacks WHERE status = 'RECEIVED') AS callback_chua_xu_ly;
```

PENDING_REVIEW bị loại trừ vì đó là trạng thái **cố ý** chờ người xử lý (thẻ B7). Giai đoạn 1 và 2 chưa có outbox và callback: bỏ hai dòng tương ứng.

Gạch 3: bộ đèn bất biến. Một câu duy nhất, định nghĩa là view v_invariant_checks (mục 8.3). Kịch bản chỉ xanh khi câu này không trả dòng nào:

```
SELECT check_name, violations FROM v_invariant_checks WHERE violations > 0 ORDER BY sort_order;
```

Gạch 4: ảnh chụp trước và sau. Chụp ngay trước khi gây lỗi và ngay sau khi hội tụ, rồi so. Tiền của hệ thống chỉ được đổi khi kịch bản có nạp/rút thật:

```
SELECT type, count(*) AS so_tai_khoan, sum(balance) AS tong_so_du FROM accounts GROUP BY type ORDER BY type;
SELECT status, count(*) FROM transfers GROUP BY status ORDER BY status;
```

7.4 Quy trình chạy một kịch bản

Mọi thẻ chạy theo cùng 8 bước. Nếu bạn làm "Scenario Lab" (mục 8.5), chính là 8 bước này:

1. **Reset:** xóa dữ liệu thử (TRUNCATE) và nạp lại dữ liệu mẫu cố định (dữ liệu mẫu ở bậc 1.1; sau bậc 1.9 nạp lại bằng bút toán, xem ghi chú ở đó).
2. **Chụp trước** (gạch 4).
3. **Gây lỗi** (gạch 1), nếu thẻ cần.
4. **Tác động:** bắn các request mà thẻ mô tả.

5. **Gỡ lỗi:** tắt công tắc, hoặc khởi động lại process không có `CRASH_AT` .
6. **Chờ hội tụ** (gạch 2).
7. **Chụp sau** và chạy bộ đèn (gạch 3, 4).
8. **So với "Kết quả đúng"** trong thẻ. **Xanh khi và chỉ khi:** hội tụ được, 8 đèn bằng 0, và từng ý trong "Kết quả đúng" đều thấy.

7.5 Các thẻ kịch bản

Nhóm A: **đồng thời** (nhiều việc xảy ra cùng một lúc)

Mã	Đời thật	Gây ra bằng	Kết quả đúng	Bắt lỗi nào
A1	Hai người dùng cùng bấm trả tiền khi ví không đủ cho cả hai; hoặc app bị bot bắn dồn	Ví A có 100.000. 1.000 request chuyển 80.000 song song (8.6 cho mức DB, <code>xargs -P</code> cho mức HTTP)	Đúng 1 transfer <code>COMPLETED</code> , 999 bị từ chối <code>INSUFFICIENT_FUNDS</code> và ghi <code>FAILED</code> . Số dư A = 20.000	Đọc số dư rồi ghi lại không khóa (bậc 1.2, 1.3)
A2	A trả B và B trả A cùng lúc (hai chiều)	Hàng nghìn lần chuyển qua lại giữa hai ví, 16 luồng song song	Không có lỗi deadlock nào; tổng tiền hai ví không đổi	Khóa không có thứ tự (bậc 1.4)
A3	Client gửi lại cùng một yêu cầu khi mạng chậm	50 request song song cùng <code>Idempotency-Key</code> , cùng body	Đúng 1 dòng <code>transfers</code> cho key đó, tiền trừ một lần , cả 50 phản hồi cùng một <code>transferId</code>	Idempotency chỉ kiểm tra tuần tự (bậc 2.3, 2.4)
A4	Hai lần bấm "thanh toán" tạo hai key khác nhau cho cùng một đơn hàng	Hai request khác <code>Idempotency-Key</code> , cùng <code>source</code> (<code>ORDER</code> , <code>1234</code>)	Chỉ 1 khoản thanh toán cho đơn đó; lần hai bị chặn	Thiếu idempotency theo nghiệp vụ (bậc 2.5)
A5	Hot account: một ví nhận tiền từ hàng nghìn người (ví người bán lớn)	1.000 transfer cùng đích, nhiều luồng	Không lỗi dữ liệu; <code>account_seq</code> không hỏng (đèn 8); ghi lại thông lượng và thời gian chờ khóa	Điểm nghẽn do khóa (bậc 8.3)
A6	Hai giao dịch cùng kiểm tra hạn mức ngày trước khi ghi	Hạn mức 1.000.000/ngày; 20 request song song, mỗi cái 100.000	<code>daily_usage.total_amount</code> không bao giờ vượt 1.000.000; số request thành công ≤ 10	Kiểm tra rồi mới cộng (không nguyên tử) (bậc 5.2)
A7	Hai instance recovery cùng thấy một transfer kẹt	Chạy hai worker quét cùng lúc trên một transfer kẹt	Transfer được xử lý một lần : mỗi <code>(transfer_id, type)</code> trong <code>ledger_transactions</code> đúng một dòng; không có hai <code>bank_requests</code> trùng <code>attempt_no</code>	Thiếu khóa/chuyển trạng thái có điều kiện (bậc 7.2)

Nhóm B: **mất mạng và lỗi bên ngoài**

Mã	Đời thật	Gây ra bằng	Kết quả đúng	Bắt lỗi nào
B1	Bank đã chuyển tiền nhưng response bị mất giữa đường	<code>chaos_rules</code> : <code>lose-response</code> bật	Transfer đi qua <code>UNKNOWN</code> , hỏi lại (inquiry) thấy thành công, về <code>COMPLETED</code> . Không hoàn tiền . Bảng <code>bank_transactions</code> của bank có đúng 1 dòng cho <code>client_request_id</code> đó	Coi timeout là thất bại rồi hoàn tiền (bậc 4.4)
B2	Bank lỗi 5xx tạm thời	<code>five-xx</code> bật rồi tắt sau vài lần thử	<code>bank_requests</code> có nhiều dòng <code>attempt_no</code> 1, 2, 3 nhưng cùng <code>bank_request_id</code> ; cuối cùng <code>COMPLETED</code> ; bank chỉ ghi 1 giao dịch	Retry sinh khóa mới nên bank chuyển hai lần (bậc 4.3)

Mã	Đời thật	Gây ra bằng	Kết quả đúng	Bắt lỗi nào
B3	Bank từ chối vĩnh viễn (tài khoản nhận không tồn tại)	reject bật	Không retry. Đi FAILED rồi hoàn tiền, kết REFUNDED . Số dư người gửi về đúng như ban đầu. Có bút toán REFUND trở về bút toán gốc	Retry mù với lỗi vĩnh viễn; hoàn tiền sai (bậc 4.4, 4.7)
B4	Bank sập 10 phút	down bật 10 phút rồi tắt	Circuit breaker mở; transfer mới nằm ở FUNDS_RESERVED , không dồn lời gọi sang bank; bank sống lại thì tất cả về COMPLETED ; không ai bị trừ hai lần	Thiếu breaker/bulkhead (bậc 4.6)
B5	Bank gửi cùng callback nhiều lần	callback-dup bật (3 bản)	bank_callbacks chỉ có 1 dòng cho event_id ; trạng thái transfer đổi đúng một lần	Thiếu UNIQUE (bank_code, event_id) (bậc 4.5)
B6	Callback đến muộn, sau khi inquiry đã chốt kết quả	callback-late bật	Transfer đã ở trạng thái cuối thì không đổi	Cho phép chuyển trạng thái lùi (bậc 3.1, 4.5)
B7	Bank báo thành công rồi lại báo thất bại	callback-flip bật	Transfer sang PENDING_REVIEW , có dòng reconciliation_issues ; không tự hoàn tiền hay tự chốt; v_pipeline_health hiện đỏ	Tin callback sau cùng một cách mù quáng (bậc 4.7)
B8	AI đó gửi callback giả	POST callback với chữ ký sai	Lưu với signature_valid = false , trạng thái IGNORED , không đổi transfer nào	Không xác thực callback (bậc 4.5)
B9	Bank chậm hơn timeout cấu hình	slow bật (6 đến 8 giây, timeout 3 giây)	Transfer đi UNKNOWN , không bao giờ thẳng FAILED /hoàn tiền khi chưa biết bank đã làm hay chưa	Timeout bị xử lý như thất bại (bậc 4.4)
B10	Mất kết nối Postgres giữa lúc đang xử lý	Toxiproxy reset_peer hoặc docker compose stop postgres 30 giây	API trả 503 (lỗi hạ tầng), không để lại bút toán lè hay transfer nửa chừng; DB về thì client gửi lại cùng key và thành công đúng 1 lần	Ghi nhiều bước ngoài một transaction (bậc 1.2)
B11	Kafka sập 5 phút	docker compose stop kafka , vẫn gửi transfer, rồi bật lại	API vẫn nhận bình thường; outbox_chua_gui tăng rồi về 0 sau khi Kafka sống lại; mọi transfer hoàn tất, không mất, không trùng hiệu ứng	Gửi Kafka trong transaction DB / không có outbox (bậc 3.3, 3.4)
B12	Cùng một message Kafka được giao lại nhiều lần	Phát lại cùng event_id 3 lần cho consumer	processed_messages chặn, số ledger_entries không tăng thêm	Consumer không idempotent (bậc 3.5)
B13	Một message hòng (poison) làm consumer kẹt	Đẩy một message JSON sai định dạng vào topic	Thử lại theo backoff vài lần rồi vào DLT; các message sau nó vẫn được xử lý	Một message hòng chặn cả hàng đợi (bậc 3.6)
B14	Postgres chậm, hết pool kết nối	Toxiproxy latency 3 giây tới Postgres, tải vừa	Trả lỗi/giới hạn gọn (503, 429), không lỗi dữ liệu; gỡ độc thì hồi phục	Không có timeout/bulkhead (Giai đoạn 5)

Nhóm C: process chết đúng một điểm. Cách gây ra giống nhau: khởi động app với CRASH_AT=<điểm> , gửi một transfer, chờ app chết, **khởi động lại không có CRASH_AT** , chờ hội tụ.

Mã	Điểm (CRASH_AT)	Kết quả đúng sau khi khởi động lại	Bắt lỗi nào
C1	API_AFTER_COMMIT	Transfer CREATED có outbox; relay gửi tiếp; client gửi lại cùng key nhận lại đúng kết quả cũ, không tạo transfer thứ hai	Mất yêu cầu hoặc xử lý trùng
C2	RELAY_AFTER_SEND	Outbox gửi lại lần hai; consumer bỏ qua bản trùng nhờ processed_messages ; tiền đúng	Outbox không idempotent ở consumer
C3	CONSUMER_BEFORE_COMMIT	DB rollback sạch; Kafka giao lại; xử lý lại từ đầu, kết quả như chưa từng lỗi	Ghi dở một nửa
C4	CONSUMER_AFTER_COMMIT	Message giao lại, processed_messages chặn, chỉ ack; không ghi sổ lần hai	Ghi sổ trùng
C5	AFTER_DEBIT_BEFORE_BANK_CALL	Tiền đã được giữ nhưng bank chưa nhận; recovery nhận ra transfer kẹt ở FUNDS_RESERVED và gọi bank (hoặc hoàn tiền nếu hết hạn); không trừ tiền lần hai	Giữ tiền mà không ai lo tiếp
C6	AFTER_BANK_CALL_BEFORE_RESPONSE	Transfer về UNKNOWN , hỏi bank, ra kết quả thật; gửi lại bằng cùng bank_request_id , bank không chuyển hai lần	Gọi bank hai lần với hai khóa khác nhau
C7	AFTER_CALLBACK_BEFORE_COMMIT	Callback vẫn ở RECEIVED , được xử lý lại sau khi khởi động; không mất, không xử lý đôi	Callback bị nuốt
C8	RECOVERY_MID_RUN	Worker khác (hoặc chính nó sau khi khởi động lại) hoàn tất transfer; không có bước nào chạy hai lần	Recovery không an toàn khi bị gián đoạn

Nhóm D: dữ liệu bị phá, và đối soát

Mã	Đời thật	Gây ra bằng	Kết quả đúng	Bắt lỗi nào
D1	Ai đó sửa hoặc xóa sổ cái bằng SQL tay	UPDATE ledger_entries SET amount = 1 ... rồi DELETE FROM ledger_entries	Cả hai bị từ chối: ledger_entries is append-only	Ràng buộc chỉ nằm ở code (bậc 1.8)
D2	Bút toán lệch (chỉ có một chân)	BEGIN , thêm một ledger_transaction với đúng một dòng CREDIT 777, COMMIT	COMMIT thất bại: ledger_transaction ... is not balanced (diff=777)	Thiếu trigger cân bút toán (bậc 1.8)
D3	Số dư bị đặt âm	UPDATE accounts SET balance = -1 WHERE ... (ví người dùng)	Bị CHECK từ chối	Số dư âm chỉ chặn ở code
D4	Số dư lưu sẵn bị sửa tay lệch khỏi sổ cái	UPDATE accounts SET balance = balance + 500 WHERE ...	Hai đèn đỏ: "Số dư lưu sẵn khớp sổ cái" và "Tổng số dư mọi tài khoản = 0"; đối soát nội bộ báo BALANCE_CACHE_MISMATCH ; cách sửa đúng là bút toán ADJUSTMENT , không UPDATE ngược lại	Lệch tiền không ai biết (bậc 7.3)
D5	Số liệu hai bên lệch nhau	Nhập báo cáo bank có 3 dòng: một dòng ta không có, một dòng ta có mà bank không, một dòng lệch số tiền	reconciliation_issues có đúng 3 dòng: MISSING INTERNALLY , MISSING AT BANK , AMOUNT_MISMATCH	Đối soát bỏ sót (bậc 7.4)

Mã	Đời thật	Gây ra bằng	Kết quả đúng	Bắt lỗi nào
D6	Một bước ghi sổ bị ghi hai lần	INSERT thêm ledger_transactions cùng (transfer_id, type) đã có	Bị uq_ledger_txn_step từ chối	Chống ghi trùng chỉ ở code (bậc 1.8)

Các lệnh D1, D2, D3, D6 đã chạy thật và cho đúng thông báo lỗi nêu trên. Riêng D3: nếu đã chạy migration tùy chọn của bậc 5.2 (cột held_balance) thì CHECK từ chối mang tên ck_accounts_available thay vì ck_accounts_user_balance; cả hai đều chặn số dư âm.

Nhóm E: ngẫu nhiên (để bắt cái bạn không nghĩ ra)

Mã	Đời thật	Gây ra bằng	Kết quả đúng	Bắt lỗi nào
E1	Cuộc sống không theo kịch bản	"Bộ trộn": chọn ngẫu nhiên 1.000 thao tác từ danh sách: chuyển hợp lệ, chuyển quá số dư, gửi trùng key, hoàn tiền, bật/tắt một công tắc bank, giết process. Dùng một hạt giống (seed) ghi lại, và in ra	Hội tụ được; 8 đèn bằng 0. Nếu đồ: in seed để chạy lại đúng chuỗi thao tác đó và gỡ lỗi	Lỗi nằm ở tổ hợp bạn không nghĩ tới (bậc 8.2)

Bộ trộn chỉ cần khoảng 30 dòng: một vòng lặp chọn thao tác bằng bộ sinh số ngẫu nhiên có seed, gọi cùng các công tắc ở gạch 1, rồi chạy gạch 2 và 3. Không cần thư viện property-based nếu bạn chưa muốn học nó.

7.6 Chạy ngay không cần ứng dụng: bốn kịch bản pgbench

Bốn kịch bản này mô phỏng A1 và A2 ở mức database, chạy được ngay sau bậc 1.9 (schema sau bậc 1.9), và đã chạy thật trên PostgreSQL 16 với 16 kết nối song song. pgbench đi kèm gói PostgreSQL (postgresql-contrib trên một số bản phân phối). Chuẩn bị:

```
-- reset.sql: dữ liệu mẫu của phòng thí nghiệm (schema sau bậc 1.9)
TRUNCATE accounts CASCADE;
DROP TABLE IF EXISTS lab_log;
CREATE TABLE lab_log (n INT);
INSERT INTO accounts (id, account_number, owner_ref, type, balance) VALUES
('00000000-0000-0000-0000-0000000000a1', 'LAB-1', 'lab-1', 'USER', 1000000),
('00000000-0000-0000-0000-0000000000a2', 'LAB-2', 'lab-2', 'USER', 1000000);
```

Kịch bản 1 (A2, sai): khóa hai ví không theo thứ tự

```
-- deadlock_naive.pgb
\set dir random(0,1)
BEGIN;
SELECT 1 FROM accounts WHERE account_number = CASE WHEN :dir = 0 THEN 'LAB-1' ELSE 'LAB-2' END FOR UPDATE;
SELECT pg_sleep(0.002);
SELECT 1 FROM accounts WHERE account_number = CASE WHEN :dir = 0 THEN 'LAB-2' ELSE 'LAB-1' END FOR UPDATE;
END;
```

Kịch bản 2 (A2, đúng): khóa có thứ tự

```
-- deadlock_ordered.pgb
BEGIN;
SELECT 1 FROM accounts WHERE account_number IN ('LAB-1','LAB-2') ORDER BY id FOR UPDATE;
SELECT pg_sleep(0.002);
END;
```

Kịch bản 3 (A1, sai): đọc số dư, kiểm tra trong app, rồi ghi

```
-- overdrew_naive.pgb (đặt LAB-1 = 100 trước khi chạy)
BEGIN;
SELECT balance AS b FROM accounts WHERE account_number = 'LAB-1' \gset
SELECT pg_sleep(0.001);
\if :b >= 80
UPDATE accounts SET balance = :b - 80 WHERE account_number = 'LAB-1';
INSERT INTO lab_log VALUES (1);
\endif
END;
```

Kịch bản 4 (A1, đúng): một câu UPDATE có điều kiện

```
-- overdrew_safe.pgb (đặt LAB-1 = 100 trước khi chạy)
BEGIN;
WITH u AS (UPDATE accounts SET balance = balance - 80
            WHERE account_number = 'LAB-1' AND balance >= 80 RETURNING 1)
INSERT INTO lab_log SELECT 1 FROM u;
END;
```

Cách chạy mỗi kịch bản (đổi tên tệp .pgb ; kịch bản 1 và 2 dùng -T 4 thay cho -t 20):

```
psql -d lab -f reset.sql
psql -d lab -c "UPDATE accounts SET balance = 100 WHERE account_number = 'LAB-1'" # chỉ cho kịch bản 3 và 4
pgbench -n -c 16 -j 4 -t 20 --failures-detailed -f overdrew_naive.pgb lab
psql -d lab -c "SELECT (SELECT count(*) FROM lab_log) AS lệnh_bao_thanh_cong, balance FROM accounts WHERE
account_number = 'LAB-1'"
```

Kết quả đo được (PostgreSQL 16.15, 16 kết nối, mỗi dòng là một lần chạy):

Kịch bản	Kết quả đo	Cách đọc
1. Khóa không thứ tự (4 giây)	5 giao dịch xong, 34 thất bại do deadlock (87%)	Hai chiều khóa ngược nhau thì gần như chỉ còn deadlock
2. Khóa có thứ tự (4 giây)	1.193 giao dịch xong, 0 thất bại	Thứ tự cố định làm deadlock biến mất
3. Đọc rồi ghi (320 lệnh rút 80 từ ví có 100)	15 lệnh báo "thành công" (tức đã báo rút 1.200), số dư cuối 20, tiền thực rời ví chỉ 80	Hệ thống báo đã rút nhiều hơn số tiền thật rời ví: tiền "bốc hơi" trong sổ sách (lost update)
4. UPDATE có điều kiện (320 lệnh)	1 lệnh thành công , số dư cuối 20	Đúng như A1

Con số kịch bản 1 và 3 thay đổi theo máy và lần chạy; điều cần thấy là **chiều hướng** (rất nhiều deadlock; nhiều hơn 1 lệnh "thành công"), không phải con số chính xác.

7.7 Khi một kịch bản đổ: nhìn vào đâu

Thấy gì	Nghĩ ngờ trước	Xem ở
Đèn "Số dư lưu sẵn khớp sổ cái" đỏ	Có đoạn code đổi <code>accounts.balance</code> ngoài <code>LedgerPostingService</code>	Màn tài khoản (9.2): cột chênh lệch; tìm lần sửa bằng sổ phụ
Đèn "Thứ tự bút toán không bị hỏng" đỏ	Ghi sổ song song mà quên khóa tài khoản	Sổ phụ tài khoản: chỗ nhảy số <code>account_seq</code>
Đèn "Tổng debit = tổng credit" hoặc "Mỗi bút toán đều cân" đỏ	Một bước ghi sổ chỉ ghi một chân; trigger chưa cài	Chi tiết transfer: khối "Sổ cái"
"Không hội tụ" (còn transfer đang dở)	Recovery không quét trạng thái đó, hoặc ngưỡng chờ quá dài	Danh sách transfer đang dở; <code>stuck_for</code> lớn nhất
<code>outbox_chua_gui</code> không giảm	Relay đã chết hoặc Kafka chưa sống lại	Trang tổng quan: "Sự kiện outbox chưa gửi"

Thấy gì	Nghĩ ngờ trước	Xem ở
Bank có hai giao dịch cho một transfer	Mỗi lần retry sinh <code>bank_request_id</code> mới	Chi tiết transfer: khối "Gọi bank", so cột <code>bank_request_id</code>
Tiền bị hoàn dù bank đã chuyển	Timeout bị coi là thất bại thay vì <code>UNKNOWN</code>	Dòng thời gian transfer: có đi qua <code>UNKNOWN</code> không
1.000 request, nhiều hơn 1 cái thành công (A1)	Kiểm tra số dư ngoài khóa	Màn tài khoản + danh sách transfer lọc <code>COMPLETED</code>

Mỗi lỗi bạn tìm ra, ghi một postmortem (bậc 8.4): *triệu chứng* → *nguyên nhân gốc* → *cách phát hiện* → *cách sửa* → *ràng buộc/kịch bản nào chặn nó lần sau*. Thẻ kịch bản đã làm bạn thấy lỗi chính là "kịch bản chặn nó lần sau".

7.8 Chạy thẻ nào ở giai đoạn nào

Đừng dồn tất cả đến Giai đoạn 8. Chạy thẻ ngay khi cơ chế tương ứng xuất hiện:

Sau giai đoạn	Chạy các thẻ	Mục đích
1 (bậc 1.4 đến 1.9)	8.6 (bốn kịch bản <code>pgbench</code>), A1, A2, D1 đến D4, D6	Chứng minh khóa và ràng buộc DB thật sự chặn lỗi
2	A3, A4, B10	Idempotency chịu được song song và mất kết nối
3	A7 (nếu đã có worker), B11, B12, B13, C1 đến C4	Outbox, Kafka, consumer idempotent
4	B1 đến B9, C5 đến C7	Mọi cách bank "nói dối" hoặc "im lặng"
5	A6, B14	Hạn mức và chịu tải
7	A7, C8, D4, D5	Recovery và đối soát bắt đúng cái bị cố ý làm hỏng
8	A5, E1, và toàn bộ nhóm A đến D chạy lại một lượt	Bảng chứng tổng; báo cáo load test

Quy tắc: thẻ nào đã xanh thì **giữ nguyên trong danh sách** và chạy lại mỗi khi sửa lỗi (khóa, ledger, state machine). Một thẻ xanh hôm nay mà đỏ sau khi bạn "dọn code" chính là giá trị của chương này.

8. Giao diện quan sát nghiệp vụ (Ops Console): xem kết quả thay vì mò vào database

Mục tiêu: sau mỗi thao tác hoặc mỗi kịch bản ở chương 7, bạn nhìn **một màn hình** là biết hệ thống đúng hay sai, giao dịch nào đang ở đâu, tiền nằm ở đâu. Không phải mở `psql`, không phải nhớ câu SQL. Đây là "observability nghiệp vụ": trả lời **"chuyện gì đã xảy ra với tiền"**, khác với metrics kỹ thuật (CPU, latency) mà bạn đã quyết định tạm bỏ qua.

Trạng thái kiểm chứng: mọi view và câu SQL trong chương đã chạy thật trên PostgreSQL 16 với dữ liệu mẫu (gồm cả trường hợp cố ý làm hỏng để xem đèn đỏ). Phần giao diện (màn hình, công cụ ở mục 8.6) là thiết kế, **chưa dựng thành trang thật** vì ứng dụng chưa tồn tại; mô tả tính năng công cụ ngoài (Metabase, Grafana) dựa trên hiểu biết chung, chưa chạy thử ở đây.

8.1 Nguyên tắc thiết kế

- Chỉ đọc.** Giao diện đọc database qua một tài khoản chỉ có quyền `SELECT` (mục 8.3). Nó không thể làm hỏng dữ liệu, kể cả khi có lỗi. Ngoại lệ duy nhất là "Scenario Lab" và công tắc lỗi (mục 8.5), chỉ tồn tại trong môi trường học.
- Màu có nghĩa, không trang trí.** Xanh = đúng, vàng = đáng chú ý (đang chờ, đang dở), đỏ = sai/cần người xử lý. Không dùng màu cho việc khác để mất bạn tin được màu.
- Mỗi con số bấm vào được.** "2 transfer đang dở" phải dẫn tới danh sách hai transfer đó; mỗi transfer dẫn tới trang chi tiết. Đi từ tổng quan xuống nguyên nhân không quá ba lần bấm.
- Một trang trả lời một câu hỏi.** "Hệ thống đúng không?" (tổng quan), "Giao dịch này đi qua những đâu?" (chi tiết), "Tiền của ví này từ đâu ra?" (tài khoản).
- Hiển thị cả cái nên bằng nhau.** Số dư lưu sẵn **cạnh** số dư tính lại từ số cái; nếu khác nhau, tô đỏ ngay. Đó là cách nhanh nhất thấy lỗi tiền.

8.2 Các màn hình

Có bảy màn hình. Mỗi màn dưới đây ghi rõ nó trả lời câu hỏi gì và lấy dữ liệu từ đâu (mục 8.4 có câu SQL).

Màn	Câu hỏi nó trả lời	Làm sau giai đoạn
S1. Tổng quan	Hệ thống có đúng và khỏe không?	1 (đèn), 6 (sức khỏe)
S2. Danh sách transfer	Có những giao dịch nào, đang ở trạng thái gì?	1
S3. Chi tiết transfer	Giao dịch này đã đi qua những bước nào, tiền chạy ra sao?	1, mở rộng ở 3, 4
S4. Tài khoản	Số dư của ví này đúng không, nó biến động thế nào?	1
S5. Ngân hàng và hàng đợi	Bank và Kafka có đang kẹt gì không?	3, 4
S6. Đối soát	Có sai lệch nào cần xử lý?	7
S7. Scenario Lab	Chạy kịch bản chương 7 bằng nút bấm	3 đến 8

S1. Tổng quan. Trang bạn mở đầu tiên và mở lại sau mỗi lần thử:

```

TỔNG QUAN ————— tự làm mới mỗi 3 giây
BẮT BIẾN TIẾN (8/8 phải xanh)
● Tổng debit = tổng credit toàn hệ thống ..... OK
● Mỗi bút toán đều cân ..... OK
● Không có bút toán rỗng ..... OK
● Không tài khoản USER nào âm ..... OK
● Số dư lưu sẵn khớp với số cái ..... ĐỎ 1 ►
● Tổng số dư mọi tài khoản = 0 ..... ĐỎ 1 ►
● Transfer nội bộ COMPLETED đều có bút toán ..... OK
● Thứ tự bút toán từng tài khoản không bị hỏng ..... OK

ĐƯỜNG ĐI CỦA GIAO DỊCH
Transfer đang chờ: 2 (WARN) UNKNOWN: 1 (WARN) Cảnh người xem: 0
Outbox chưa gửi Kafka: 1 (WARN) Callback chưa xử lý: 0 Callback chữ ký sai: 1 (ĐỎ)
Sai lệch đối soát chưa xử lý: 1 (ĐỎ)

TRANSFER THEO TRẠNG THÁI (1 giờ qua) ██████████ COMPLETED 120 █ FAILED 4 █ UNKNOWN 1

```

Dấu ► bấm vào dẫn tới danh sách vi phạm (ví dụ tài khoản bị lệch ở S4). Dữ liệu: `v_invariant_checks` và `v_pipeline_health`.

S3. Chi tiết transfer. Màn quan trọng nhất cho việc học: một trang kể **toàn bộ câu chuyện** của một giao dịch, gom từ nhiều bảng:

```

TRANSFER f0000000-...-a2 OUTBOUND 20.000 VND ► SENT_TO_BANK (đã chờ 20 phút)
Từ: W-B (user-B)      Tối: ngân hàng VCB / 0011223344      Key: k-2      Nguồn: ORDER 1234

DÒNG THỜI GIAN
05:45:20 TRẠNG THÁI (mới) → CREATED API
05:45:20 KAFKA transfer.created đã gửi lúc 05:45:20
05:46:20 TRẠNG THÁI CREATED → FUNDS_RESERVED KAFKA_CONSUMER
05:46:20 SỐ CÁI TO_CLEARING W-B DEBIT 20000, CLEARING CREDIT
05:47:20 TRẠNG THÁI FUNDS_RESERVED → SENT_TO_BANK KAFKA_CONSUMER
05:47:20 GỌI BANK TRANSFER #1 → TIMEOUT READ_TIMEOUT
05:48:20 GỌI BANK TRANSFER #2 → SENT
(chưa có callback)

BÚT TOÁN (kiểu chữ T)          GỌI BANK
TO_CLEARING W-B DEBIT 20.000 #1 bank_request_id br-2 TIMEOUT
CLEARING CREDIT 20.000 #2 bank_request_id br-2 SENT

```

Trang này làm được hai việc mà đọc bảng không làm được: thấy **thứ tự thật** các sự kiện từ năm nguồn khác nhau (trạng thái, Kafka, sổ cái, gọi bank, callback), và so các dòng cần giống nhau (hai lần gọi bank phải cùng `bank_request_id`). Dữ liệu: `v_transfer_timeline` cộng hai câu SQL ở mục 8.4.

S4. Tài khoản. Số dư cạnh số dư tính lại; sổ phụ có số dư chạy; biểu đồ bậc thang theo `account_seq`:

```

TÀI KHOẢN W-A (user-A) USER —————
Số dư lưu sẵn: 60.500      Tính lại từ số cái: 60.000      CHÉNH LỆCH: +500 ● ĐỎ

seq loại      chiều số tiền số dư sau số dư tính lại
1 TOPUP CREDIT 100.000 100.000 100.000
2 PRINCIPAL DEBIT 30.000 70.000 70.000
3 TO_CLEARING DEBIT 10.000 60.000 60.000

Số dư theo thời gian (mỗi bậc là một dòng số):
100k | █████
70k  | █████
60k  | █████

```

Dữ liệu: `v_account_ledger_check` (đầu trang) và `v_account_statement` (bảng và biểu đồ). Cột "số dư sau" so với "số dư tính lại": nếu hai cột này khác nhau ở dòng nào, **đó là dòng đầu tiên bị ghi sai**.

S2, S5, S6 là các bảng có lọc, mỗi dòng bấm vào để xuống S3:

- **S2 Danh sách transfer:** lọc theo trạng thái, loại, khoảng thời gian; tìm theo `id`, `idempotency_key`, `source_id` (mã đơn hàng).
- **S5 Ngân hàng và hàng đợi:** transfer kẹt lâu nhất, outbox chưa gửi (sự kiện cũ nhất bao nhiêu giây), callback chưa xử lý, callback chữ ký sai, trạng thái các công tắc lỗi của Fake Bank.
- **S6 Đối soát:** các lần chạy và các sai lệch OPEN; mỗi dòng có ghi chú xử lý và bút toán điều chỉnh đã tạo.

8.3 Các view và quyền đọc

Thay vì để giao diện tự ghép SQL phức tạp, đặt logic vào **view** trong database. Lợi ích: câu SQL kiểm tra nằm một chỗ, cả giao diện, kịch bản chương 7 và bạn gõ tay đều dùng chung. View không thêm bảng, không thêm dữ liệu. Tạo theo ba đợt, đúng lúc các bảng nó cần đã tồn tại. Cả ba đợt đã chạy thật: đợt 1 trên một database chỉ có V1, các đợt sau trên schema đầy đủ V1 đến V6.

Đợt ops_1 : sau bậc 1.9 (chỉ cần năm bảng của giai đoạn 1)

```
-- Số dư lưu sẵn so với số dư tính lại từ số cái, từng tài khoản
CREATE VIEW v_account_ledger_check AS
SELECT a.id AS account_id, a.account_number, a.type, a.owner_ref,
       a.balance AS cached_balance,
       COALESCE(SUM(CASE e.direction WHEN 'CREDIT' THEN e.amount ELSE -e.amount END), 0) AS ledger_balance,
       a.balance - COALESCE(SUM(CASE e.direction WHEN 'CREDIT' THEN e.amount ELSE -e.amount END), 0) AS diff,
       a.entry_seq, COUNT(e.id) AS entry_count
FROM accounts a LEFT JOIN ledger_entries e ON e.account_id = a.id
GROUP BY a.id;

-- Bảng đèn: mỗi dòng là một bất biến; violations = 0 thì OK
CREATE VIEW v_invariant_checks AS
SELECT 'Tổng debit = tổng credit toàn hệ thống' AS check_name, 1 AS sort_order,
       (SELECT CASE WHEN COALESCE(SUM(CASE direction WHEN 'DEBIT' THEN amount ELSE -amount END), 0) = 0 THEN 0
        ELSE 1 END
        FROM ledger_entries) AS violations
UNION ALL
SELECT 'Mỗi bút toán đều cân', 2,
       (SELECT COUNT(*) FROM (SELECT 1 FROM ledger_entries GROUP BY ledger_transaction_id
                               HAVING SUM(CASE direction WHEN 'CREDIT' THEN amount ELSE -amount END) <> 0) x)
UNION ALL
SELECT 'Không có bút toán rỗng', 3,
       (SELECT COUNT(*) FROM ledger_transactions l
        LEFT JOIN ledger_entries e ON e.ledger_transaction_id = l.id WHERE e.id IS NULL)
UNION ALL
SELECT 'Không tài khoản USER nào âm', 4,
       (SELECT COUNT(*) FROM accounts WHERE type = 'USER' AND balance < 0)
UNION ALL
SELECT 'Số dư lưu sẵn khớp với số cái', 5,
       (SELECT COUNT(*) FROM v_account_ledger_check WHERE cached_balance <> ledger_balance)
UNION ALL
SELECT 'Tổng số dư mọi tài khoản = 0', 6,
       (SELECT CASE WHEN COALESCE(SUM(balance), 0) = 0 THEN 0 ELSE 1 END FROM accounts)
UNION ALL
SELECT 'Transfer nội bộ COMPLETED đều có bút toán', 7,
       (SELECT COUNT(*) FROM transfers t
        LEFT JOIN ledger_transactions l ON l.transfer_id = t.id AND l.type = 'PRINCIPAL'
        WHERE t.type = 'INTERNAL' AND t.status = 'COMPLETED' AND l.id IS NULL)
UNION ALL
SELECT 'Thứ tự bút toán từng tài khoản không bị hỏng', 8,
       (SELECT COUNT(*) FROM (SELECT account_seq,
                                     LAG(account_seq) OVER (PARTITION BY account_id ORDER BY account_seq) AS prev
                               FROM ledger_entries) x WHERE prev IS NOT NULL AND account_seq <> prev + 1);

-- Transfer chưa xong; lọc theo tuổi ở phía ứng dụng, ví dụ WHERE stuck_for > interval '5 minutes'
CREATE VIEW v_open_transfers AS
SELECT id, type, status, amount, from_account_id, to_account_id, retry_count,
       created_at, updated_at, now() - updated_at AS stuck_for
FROM transfers
WHERE status NOT IN ('COMPLETED', 'FAILED', 'REFUNDED');

-- Số phụ một tài khoản, kèm số dư chạy; lọc WHERE account_id = ... ORDER BY account_seq
CREATE VIEW v_account_statement AS
SELECT e.account_id, e.account_seq, e.created_at, lt.transfer_id, lt.type AS ledger_type,
       e.direction, e.amount, e.balance_after,
       SUM(CASE e.direction WHEN 'CREDIT' THEN e.amount ELSE -e.amount END)
       OVER (PARTITION BY e.account_id ORDER BY e.account_seq) AS recomputed_balance
FROM ledger_entries e JOIN ledger_transactions lt ON lt.id = e.ledger_transaction_id;
```

Đền số 6 ("tổng số dư mọi tài khoản = 0") đúng vì mỗi bút toán có tổng DEBIT bằng tổng CREDIT, và số dư mỗi tài khoản bằng tổng CREDIT trừ tổng DEBIT của nó: cộng tất cả lại phải ra 0. Nếu tài khoản người dùng dương thì tài khoản `SYSTEM_BANK` (đối ứng với tiền nạp vào) phải âm bằng đúng số đó.

Đợt ops_2 : sau bậc 4.5 (cần `outbox_events` của bậc 3.4, `bank_requests` của bậc 4.3 và `bank_callbacks` của bậc 4.5)

```
-- Hành trình của MỘT transfer: gộp mọi nguồn vào một dòng thời gian
CREATE VIEW v_transfer_timeline AS
SELECT transfer_id, created_at AS at, 'TRẠNG THÁI' AS source,
       COALESCE(from_status, '(mới)') || ' → ' || to_status AS what,
       actor || COALESCE(' · ' || reason, '') AS detail
FROM transfer_status_history
UNION ALL
SELECT lt.transfer_id, lt.created_at, 'SỐ CÁI', lt.type,
       (SELECT string_agg(a.account_number || ' ' || e.direction || ' ' || e.amount, ', ' ORDER BY e.id)
        FROM ledger_entries e JOIN accounts a ON a.id = e.account_id
        WHERE e.ledger_transaction_id = lt.id)
FROM ledger_transactions lt
UNION ALL
SELECT transfer_id, sent_at, 'GỬI BANK', operation || ' #' || attempt_no || ' → ' || status,
       COALESCE('http ' || http_status || ' ', '') || COALESCE(error_code, '')
FROM bank_requests
UNION ALL
SELECT transfer_id, received_at, 'CALLBACK', event_type || ' (' || status || ')',
       CASE WHEN signature_valid THEN 'chữ ký hợp lệ' ELSE 'CHỮ KÝ SAI' END
FROM bank_callbacks WHERE transfer_id IS NOT NULL
UNION ALL
SELECT aggregate_id, created_at, 'KAFKA (OUTBOX)', event_type,
       CASE WHEN published_at IS NULL THEN 'chưa gửi, đã thử' || attempts || ' lần'
       ELSE 'đã gửi lúc ' || to_char(published_at, 'HH24:MI:SS') END
FROM outbox_events WHERE aggregate_type = 'TRANSFER';
```

Đợt ops_3 : sau bậc 7.4 (cần `reconciliation_issues`)

```
-- Các con số cho trang tổng quan: tên, giá trị, mức độ (OK / WARN / FAIL)
CREATE VIEW v_pipeline_health AS
SELECT 'Transfer đang dở (chưa COMPLETED/FAILED/REFUNDED)' AS metric, COUNT(*)::bigint AS value,
       CASE WHEN COUNT(*) FILTER (WHERE updated_at < now() - interval '5 minutes') > 0 THEN 'WARN' ELSE 'OK'
END AS level
FROM transfers WHERE status NOT IN ('COMPLETED', 'FAILED', 'REFUNDED')
UNION ALL
SELECT 'Transfer ở trạng thái UNKNOWN', COUNT(*), CASE WHEN COUNT(*) > 0 THEN 'WARN' ELSE 'OK' END
FROM transfers WHERE status = 'UNKNOWN'
UNION ALL
SELECT 'Transfer cần người xem (PENDING_REVIEW)', COUNT(*), CASE WHEN COUNT(*) > 0 THEN 'FAIL' ELSE 'OK' END
FROM transfers WHERE status = 'PENDING_REVIEW'
UNION ALL
SELECT 'Sự kiện outbox chưa gửi lên Kafka', COUNT(*),
       CASE WHEN COUNT(*) FILTER (WHERE created_at < now() - interval '1 minute') > 0 THEN 'WARN' ELSE 'OK'
END
FROM outbox_events WHERE published_at IS NULL
UNION ALL
SELECT 'Callback từ bank chưa xử lý', COUNT(*),
       CASE WHEN COUNT(*) FILTER (WHERE received_at < now() - interval '1 minute') > 0 THEN 'WARN' ELSE 'OK'
END
FROM bank_callbacks WHERE status = 'RECEIVED'
UNION ALL
SELECT 'Callback có chữ ký sai', COUNT(*), CASE WHEN COUNT(*) > 0 THEN 'FAIL' ELSE 'OK' END
FROM bank_callbacks WHERE NOT signature_valid
UNION ALL
SELECT 'Sai lệch đối soát chưa xử lý', COUNT(*), CASE WHEN COUNT(*) > 0 THEN 'FAIL' ELSE 'OK' END
FROM reconciliation_issues WHERE status IN ('OPEN', 'INVESTIGATING');
```

Các ngưỡng "5 phút", "1 phút" là con số chọn tạm; chỉnh theo thời gian thật của hệ thống bạn.

Tài khoản chỉ đọc cho giao diện. Đã thử: tài khoản này đọc được view, nhưng `UPDATE` bị từ chối, và bảng `api_clients` (chứa hash khóa API) bị che:

```
CREATE ROLE ops_reader LOGIN PASSWORD 'đổi-mật-khẩu-này';
GRANT CONNECT ON DATABASE wallet TO ops_reader;           -- đổi 'wallet' thành tên database của bạn
GRANT USAGE ON SCHEMA public TO ops_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO ops_reader;
REVOKE SELECT ON api_clients FROM ops_reader;
-- Bảng tạo SAU này (migration mới) cần cấp lại, hoặc đặt mặc định:
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO ops_reader;
```

Nếu dùng `ALTER DEFAULT PRIVILEGES` thì nhớ `REVOKE` lại cho bảng nhạy cảm mới. Giao diện kết nối bằng `ops_reader` , **không** dùng tài khoản của ứng dụng.

8.4 Câu SQL cho từng màn hình

Tham số viết dạng `:tên` (kiểu Spring, xem mục 2.0). Mọi câu đã chạy thật.

S1. Tổng quan

```
SELECT check_name, violations FROM v_invariant_checks ORDER BY sort_order;
SELECT metric, value, level FROM v_pipeline_health;

-- Transfer theo phút và trạng thái (cho biểu đồ cột chồng)
SELECT date_trunc('minute', created_at) AS minute, status, count(*)
FROM transfers WHERE created_at > now() - interval '1 hour'
GROUP BY 1, 2 ORDER BY 1;

-- Từ lúc tạo đến lúc xong mất bao lâu (chỉ giao dịch đã COMPLETED)
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY completed_at - created_at) AS p50,
       percentile_cont(0.95) WITHIN GROUP (ORDER BY completed_at - created_at) AS p95
FROM transfers WHERE status = 'COMPLETED' AND completed_at IS NOT NULL;
```

S2. Danh sách transfer (lọc tùy chọn; `CAST` cần thiết để PostgreSQL biết kiểu khi tham số có thể là `NULL`):

```
SELECT id, type, status, amount, created_at, updated_at
FROM transfers
WHERE (CAST(:status AS text) IS NULL OR status = CAST(:status AS text))
ORDER BY created_at DESC
LIMIT 50;
```

S3. Chi tiết transfer

```
-- Dòng thời gian (gộp năm nguồn)
SELECT at, source, what, detail FROM v_transfer_timeline WHERE transfer_id = :id ORDER BY at;

-- Bút toán kiểu chữ T
SELECT lt.type, a.account_number, e.direction, e.amount, e.balance_after
FROM ledger_transactions lt
JOIN ledger_entries e ON e.ledger_transaction_id = lt.id
JOIN accounts a ON a.id = e.account_id
WHERE lt.transfer_id = :id
ORDER BY lt.created_at, e.id;

-- Các lần gọi bank (so cột bank_request_id: phải giống nhau qua các lần retry)
SELECT attempt_no, operation, bank_request_id, bank_reference, status, http_status, error_code, sent_at,
       responded_at
FROM bank_requests WHERE transfer_id = :id ORDER BY sent_at;
```

S4. Tài khoản

```
-- Đầu trang: lưu sẵn số với tính lại
SELECT cached_balance, ledger_balance, diff FROM v_account_ledger_check WHERE account_id = :account_id;

-- Số phụ và số liệu cho biểu đồ bậc thang
SELECT account_seq, created_at, ledger_type, direction, amount, balance_after, recomputed_balance,
transfer_id
FROM v_account_statement WHERE account_id = :account_id ORDER BY account_seq;

-- Tài khoản đang lệch (dẫn từ đèn đỏ ở S1)
SELECT account_number, cached_balance, ledger_balance, diff FROM v_account_ledger_check WHERE diff <> 0;
```

S5. Ngân hàng và hàng đợi

```
SELECT id, type, status, amount, stuck_for FROM v_open_transfers ORDER BY stuck_for DESC LIMIT 20;

SELECT id, event_type, attempts, now() - created_at AS cho_bao_lau
FROM outbox_events WHERE published_at IS NULL ORDER BY id LIMIT 20;

SELECT id, event_type, status, signature_valid, now() - received_at AS cho_bao_lau
FROM bank_callbacks WHERE status = 'RECEIVED' OR NOT signature_valid ORDER BY id DESC LIMIT 20;
```

S6. Đối soát

```
SELECT i.created_at, i.issue_type, i.status, i.transfer_id, i.expected_amount, i.actual_amount,
i.resolution_note
FROM reconciliation_issues i
WHERE i.status IN ('OPEN', 'INVESTIGATING')
ORDER BY i.created_at DESC;
```

8.5 Scenario Lab: chạy kịch bản bằng nút bấm

Đây là phần biến chương 7 thành thứ **bấm được**, để bạn không phải viết test. Màn S7 liệt kê các thẻ kịch bản; mỗi thẻ có nút **Chạy**:

```
SCENARIO LAB _____ chỉ bật ở profile "lab"
A1 Rút vượt số dư song song (1000 request) [ Chạy ] ● ĐẠT
A3 50 request cùng Idempotency-Key [ Chạy ] ● ĐẠT
B1 Bank xử lý xong nhưng mất response [ Chạy ] ● ĐỎ
C6 Chết sau khi gọi bank, trước khi nhận response [ Chạy ] chưa

KẾT QUẢ B1 (chạy lúc 14:02:11, mất 38 giây)
Hội tụ: ĐẠT (sau 12 giây) Bộ đèn: 8/8 xanh
Kết quả đúng của thẻ:
✓ Transfer đi qua UNKNOWN rồi COMPLETED
✗ Số dòng bank_transactions cho client_request_id = 2 (phải là 1) ▶ xem dòng thời gian
Trước → Sau: USER 120.000 → 100.000 COMPLETED 3 → 4 UNKNOWN 0 → 0
```

Bên dưới mỗi lần chạy, Lab thực hiện đúng 8 bước ở mục 7.4 và hiển thị:

- **Đạt/Đỏ** cho từng ý trong "Kết quả đúng" của thẻ, ghi **số liệu thật** bên cạnh (ví dụ "= 2, phải là 1"), để bạn biết sai ở đâu mà không cần đoán.
- **Trước → Sau**: hai ảnh chụp của gạch 4, hiển thị song song.
- **Liên kết vào dòng thời gian** (S3) của transfer liên quan để đọc câu chuyện.
- **Lịch sử các lần chạy**, để thấy thẻ nào từng xanh rồi đỏ trở lại sau khi bạn sửa code.

Cũng trong Lab: **bảng công tắc lỗi** của Fake Bank (danh sách `chaos_rules`, mỗi dòng có nút bật/tắt) và nút **"Chạy lại bộ trộn với seed này"** cho thẻ E1.

An toàn: Lab và công tắc lỗi **ghi** vào database và có thể `TRUNCATE`, nên (1) chỉ bật khi biến môi trường/profile `lab` được đặt, (2) chỉ lắng nghe `localhost`, (3) nút Reset luôn hỏi xác nhận. Các màn S1 đến S6 vẫn chỉ đọc.

8.6 Dựng bằng cách nào

Cách	Công sức	Được	Không được
A. Metabase (chạy bằng Docker, nối vào database bằng <code>ops_reader</code>)	Khoảng vài giờ; gần như không có code	Dựng S1, S2, S4, S5, S6 bằng cách chọn view và bảng; có lọc, biểu đồ, làm mới tự động	Không có nút hành động (S7); trang chi tiết transfer (S3) phải làm tay bằng "dashboard có tham số"
B. Grafana (nguồn dữ liệu PostgreSQL)	Vài giờ; bạn đã có Grafana nên đỡ cài	Panel bảng và ô số, tô màu theo ngưỡng (xanh/vàng/đỏ) rất hợp với đèn bất biến; biểu đồ theo thời gian	Cũng không có nút hành động; lần này đọc bảng nghiệp vụ , khác với việc phân tích metrics Prometheus mà bạn thấy khó
C. Ops Console tự viết trong wallet-service (Spring Boot + Thymeleaf, thêm htmx cho phần tự làm mới)	Vài ngày cho S1 đến S6; thêm cho S7	Có đủ bảy màn, có nút bấm, trang chi tiết kể được câu chuyện, cùng một dự án Java	Phải viết code giao diện (nhưng chỉ HTML phía server, không cần học framework JavaScript)

Khuyến nghị: đi từ A hoặc B ngay sau bậc 1.9 (vài giờ, đủ để thấy đèn xanh/đỏ và số dư lịch), rồi chuyển sang C khi đến Giai đoạn 3 và 4, lúc bạn cần trang chi tiết transfer và nút chạy kịch bản. Các view ở mục 8.3 dùng chung cho cả ba cách nên công đã bỏ ra không phí.

Nếu muốn xem nhanh mà **chưa dựng gì**, `psql` có sẵn lệnh lặp lại một câu hỏi mỗi vài giây:

```
SELECT check_name, violations FROM v_invariant_checks ORDER BY sort_order \watch 2
```

(`\watch 2` chạy lại câu lệnh ngay trước nó mỗi 2 giây, trong `psql` tương tác.)

8.7 Lộ trình dựng theo từng giai đoạn

Bậc	Sau giai đoạn	Việc làm	Bàn giao (kiểm tra được)
UI.1	1 (sau bậc 1.9)	Chạy <code>ops_1</code> , tạo <code>ops_reader</code> ; dựng S1 (chỉ phần đèn), S2, S4 bằng Metabase/Grafana	Sửa tay <code>accounts.balance</code> thì đèn số 5 đỏ trong lần tải trang kế tiếp, và S4 chỉ ra tài khoản nào lịch bao nhiêu
UI.2	2 đến 3	Thêm S3 bản đầu (dòng thời gian từ <code>transfer_status_history</code> , bút toán chữ T, outbox); thêm khối outbox chưa gửi vào S5	Với một transfer bất kỳ, kể lại hành trình chỉ từ một trang
UI.3	4 (sau bậc 4.5)	Chạy <code>ops_2</code> ; S3 thêm khối gọi bank và callback; S5 thêm bank; bảng công tắc lỗi của Fake Bank	Bật <code>lose-response</code> , thấy trên S3 transfer đi qua <code>UNKNOWN</code> mà không dựng database
UI.4	6 đến 7 (sau bậc 7.4)	Chạy <code>ops_3</code> ; hoàn thiện S1 (khối "đường đi của giao dịch"), dựng S6	Cố ý tạo một sai lệch đối soát, nó hiện đỏ ở S1 và có trong S6
UI.5	8	Dựng S7 Scenario Lab theo mục 8.5 (cần Cách C)	Bấm Chạy một thẻ và thấy Đạt/Đỏ kèm số liệu, không mở <code>psql</code>

Đạt chuẩn chung của chương: **sau mỗi lần thử, bạn biết kết quả nhờ nhìn giao diện, không cần mở `psql`** . Nếu phải mở `psql` để hiểu chuyện gì xảy ra, giao diện còn thiếu một thứ, và đó là việc cần thêm.